
A Greedy PDE Router for Blending Neural Operators and Classical Methods

Anonymous Author(s)

Affiliation

Address

email

1 **Revision note.** All text, tables and figures added in this revision are typeset in red (new figures
2 carry a red frame); everything in black is unchanged from the submitted version. The iteration-based
3 experiments of the submitted version on 31^2 and 63^2 grids with the LSTM router, and their appendix
4 analyses, have been removed: they are superseded by the wall-clock study of Sections E and 7, which
5 covers the same solvers and equations (plus anisotropic and variable-coefficient diffusion) on 128^2 –
6 512^2 grids and also reports the iteration-based metrics. The Dirichlet experiment of the submitted
7 version has been withdrawn (Section D); two black sentences of the abstract (the greedy-guarantee
8 sentence and the AUC sentence) were edited and are now marked in red.

9 **Status of the confirmatory re-run (draft for review).** In response to five rounds of review, every
10 wall-clock result is being regenerated under a revised, frozen protocol (Section E.1): compiled
11 classical kernels, a corrected forcing sampler, per-operation costs measured in the form in which
12 they are charged, the deployed rule equal to Algorithm 1 on macro-actions, HINTS with its two-
13 dimensional period $\tau = 15$, residual-decay controls executed live, classical baselines timed in the
14 same session as the policies with validated Krylov replays, a drift guard with re-timing, two-sided
15 tests with bootstrap intervals and explicit censoring classes, a fresh test seed and three warmed-up
16 timed replays. At the time of this draft the following are *final*: the theory corrections (Sections 4
17 and 5 and their proofs); the wording of every claim is macro-driven and will be recomputed from the
18 complete tables. The 128^2 pairwise cells of Poisson and convection–diffusion were produced by an
19 earlier revision of the driver (before the live decay rule, the Krylov validation, the stored decision
20 sequences and the in-cell re-timing existed), and every one of their instances is re-timed with the final
21 driver once the other 128^2 cells are complete; those of anisotropic diffusion were produced by the
22 final driver except for two details of the drift guard (it slept rather than ran the reference operation
23 while waiting, and it compared an instance only with the reference at the time of its check), and the
24 re-timing pass re-times any of their instances that were slow against the cell’s final reference. Until
25 the re-timing, the macros that depend on those fields (decision-sequence agreement, Krylov validity,
26 re-timing counts) print as pending. Everything marked **[pending]** in the text or shown as a **[results
27 pending]** table is still running, in the order of `run_all.sh`, which completes every 128^2 result
28 before any larger grid: the remaining 128^2 pairwise cells and the 128^2 ensembles, variable-coefficient
29 diffusion at 128^2 (Section E.5), the re-timing pass, the 128^2 usage traces, seed trials and assumption,
30 screening and theorem checks, and the decision-granularity ablation (Section E.6); then all 256^2 cells
31 with their ensembles, all 512^2 cells, the checks at those grids, overheads, the discretisation study and
32 variable-coefficient diffusion at 256^2 . Summary ranges quoted in the abstract and in Section 7 are
33 computed from the cells available at build time. Where a first confirmatory run (before the protocol
34 revision) had produced a number that is quoted in the text, it is labelled as such.

Abstract

35 When solving PDEs, classical numerical solvers are often computationally ex-
36 pensive, while machine learning methods can suffer from spectral bias, failing
37 to capture high-frequency components. Designing an optimal hybrid iterative
38 solver—where, at each iteration, a solver is selected from an ensemble of solvers to
39 leverage their complementary strengths—poses a challenging combinatorial prob-

lem. While greedy selection is desirable for its **guarantee of a constant fraction of the optimal error reduction** under Lipschitz and shared-eigenbasis assumptions, it requires knowledge of the true error at each step, which is unavailable in practice. We address this by proposing an approximate greedy router that efficiently mimics a greedy approach to solver selection. **Empirical results on the Poisson, convection–diffusion and anisotropic diffusion equations compare our method with single-solver baselines and with existing hybrid approaches such as HINTS on the area-under-the-curve (AUC) of the error trajectory: over the 128^2 cells the mean-AUC ratios of HINTS and of the solver alone to our method are $3\text{--}174709\times$ and $3\text{--}2666251\times$ (one-sided paired t -tests on $\log \text{AUC}$, $p \leq 10^{-10}$).** With a cost-aware instantiation of the greedy rule, a lightweight router, and a band-limited scale-equivariant corrector, we also measure wall-clock time to a target accuracy (the time charged to each method in a replay of its decisions, Section E.1) on 128^2 to 512^2 grids with compiled classical kernels, four equations (Poisson, convection–diffusion, anisotropic and variable-coefficient diffusion) and classical solvers from Jacobi to geometric multigrid, against HINTS, the best fixed schedule and the best residual-decay rule chosen per setting on the test set, and multigrid, preconditioned Krylov and direct solves timed in the same session. At truncation-level accuracy the learned router executes the same decision sequence as the best static schedule found by a test-set search on 67–100% of the instances, and its paired time ratios are $0.94\times\text{--}0.99\times$ against that schedule and $0.95\times\text{--}5.11\times$ against HINTS with its published period; below truncation level the ratios are $0.92\times\text{--}3.07\times$ against HINTS, $0.97\times\text{--}1373\times$ against the one-shot schedule and $0.83\times\text{--}1.69\times$ against the best fixed schedule (the router is faster in 8 of 24 pairings), and the ratio of multigrid alone to the router is $0.03\times\text{--}20.5\times$ (all at 128^2 ; a ratio above one favours the router). Routing over ensembles of classical solvers, the larger grids and the finer decision granularity are **[pending]**.

1 Introduction

Natural phenomena and engineered systems are often governed by ordinary and partial differential equations (PDEs). Solving these equations enables a variety of tasks - predicting the evolution of the system through simulation (e.g., forecasting weather using the Navier-Stokes equations) [Kalnay, 2003, Bauer et al., 2015, Staniforth and Côté, 1991], addressing control problems (e.g., optimizing heat shield design for spacecrafts using heat transfer equations) [Anderson, 1989, Tröltzsch, 2010], and tackling inverse problems based on external measurements (e.g., reconstructing brain activity from EEG data using electrophysiological models) [Baillet et al., 2001, Grech et al., 2008].

Traditionally, PDEs are solved using finite difference methods [Smith, 1985, LeVeque, 2007], which discretize the spatial and/or temporal domain and approximate the solution by solving a system of equations at the discrete grid points. Similarly, finite element methods [Hughes, 2003, Bathe, 2006, Brenner, 2008] construct a system of equations based on local basis functions, and then rely on numerical linear algebra techniques—such as the Jacobi and Gauss-Seidel method [Saad, 2003]—to compute the solution. These iterative methods, however, **must be re-run from scratch for every new** initial condition, boundary condition, or forcing function, as even minor changes to any of these parameters require re-solving the entire system of equations from scratch.

These challenges have motivated the development of neural operators [Kovachki et al., 2023]—a class of machine learning models that aim to learn the solution operator directly, enabling fast inference across a range of parameters. Neural operators lift the classical linear layer in neural networks to an operator—typically a kernel integral operator—between function spaces. Despite strong approximation guarantees [Chen and Chen, 1995], neural operators are prone to spectral bias [Liu and Cai, 2021, Liu et al., 2024, You et al., 2024, Khodakarami et al., 2025, Xu et al., 2025], similar to traditional neural networks [Rahaman et al., 2019, Xu et al., 2019, Luo et al., 2019]. As a result, they favor learning low-frequency components of the solution function and often struggle to capture high-frequency features that iterative solvers excel at.

Recognizing this complementarity, Zhang et al. [2024] proposed HINTS, a hybrid solver that interleaves a neural operator pass every τ^{th} iteration of a classical solver, with τ fixed a priori. While HINTS reports significant empirical gains in convergence and accuracy over the standalone neural

operator and Jacobi solver, this fixed schedule can be detrimental: a poorly timed neural correction may increase the error and undo recent progress.

Although state-dependent solver schedules are desirable, this naturally suggest a reinforcement learning formulation for learning routing policies. However, in our setting, the structure of iterative PDE solvers allows for a simpler approach: each solver induces a known update, and its effect can be evaluated through error-based signals. Leveraging this, we adopt a greedy rule that selects, at each iteration, the solver with the largest immediate error reduction. Such a rule can be near-optimal when the final error satisfies (weak) supermodularity—so that applying a beneficial solver earlier is at least as valuable as applying it later. To support this approach, we make the following contributions:

- We present a general hybrid PDE solver in which a routing rule chooses, at each iteration, from a set of classical solvers and neural operators. With oracle access to the true error at every step, we show that a greedy routing rule achieves **a constant fraction of the optimal error reduction** for linear PDEs, provided the updates are **non-expansive (Lipschitz with constant ≤ 1 , at least one of those in the optimal sequence strictly contractive)** and zero-preserving—conditions under which weak supermodularity holds—and **share a unitary eigenbasis, which gives the postfix monotonicity the guarantee also needs; for other ensembles the slack μ is measured, over short horizons only.**
- Given that the true error is not observed at test time, a convex surrogate loss is introduced, which, when minimized, enables the learned model to imitate the greedy router without access to true error information. This alignment is guaranteed through Bayes consistency **of the surrogate for the full-information router (Section 5).**
- We evaluate the approximate greedy router against HINTS and individual solvers on the Poisson, convection-diffusion, anisotropic and variable-coefficient diffusion equations, **in iterations and in wall-clock time on 128^2 to 512^2 grids with compiled classical kernels and with the classical baselines timed in the same session. At truncation-level accuracy the router executes the same decision sequence as the best static schedule on 67–100% of the instances without a search, and its paired time ratios are $0.94\times$ – $0.99\times$ against that schedule and $0.95\times$ – $5.11\times$ against HINTS with its published period; below truncation level the ratios are $0.92\times$ – $3.07\times$ against HINTS, $0.97\times$ – $1373\times$ against the one-shot schedule and $0.83\times$ – $1.69\times$ against the best fixed schedule chosen on the test set (faster in 8 of 24 pairings at 128^2), while the ratio of multigrid alone to the router is $0.03\times$ – $20.5\times$ there. The construction extends to ensembles of classical solvers (Section E.7; **[pending]**).**

2 Background

Consider the following linear PDE:

$$\mathcal{L}_x^a(u) = f, x \in D; \quad \mathcal{B}_x(u) = b, x \in \partial D \quad (1)$$

where $\mathcal{L}_x^a$ is a differential operator with respect to the spatial variable $x \in D$ parameterized by the coefficient function a , f is a forcing function, u is the PDE solution, $\mathcal{B}_x$ is the boundary operator, and b is the boundary term. Such PDEs can be solved numerically using various discretization strategies, such as finite differences [Smith, 1985, LeVeque, 2007], finite element, and spectral methods [Boyd, 2001, Canuto et al., 2006]. Here, we focus on finite differences, which replace derivatives with difference quotients (e.g., $\partial_x u(x) \approx (u(x+h) - u(x))/h$) on a uniform grid.

Formally, let h be the grid size and $G_h(D)$ be a uniform grid with spacing h over D . For a function $g : D \rightarrow \mathbb{R}$, we denote its restriction to $G_h(D)$ by $g_h \in \mathbb{R}^N$, where $N = |G_h(D)|$. If $\mathcal{L}_h^a \in \mathbb{R}^{N \times N}$ is the discretized operator, the PDE reduces to a linear system of equations

$$\mathcal{L}_h^a u_h = f_h, \quad (2)$$

with the boundary conditions incorporated in $\mathcal{L}_h^a$. Unlike much of the neural operator literature, which retains function space formulations for discretization invariance, we follow the conventions of classical numerical analysis and represent functions and operators as vectors and matrices. Discretization invariance is not essential here, since we focus on fixed-grid problems.

2.1 Iterative PDE Solvers

Direct methods like Gaussian elimination and the Thomas algorithm can be computationally expensive in high-dimensional domains when solving systems like Equation (2). In contrast, iterative methods like Jacobi and Gauss-Seidel offer computational speedups by iteratively updating the solution, gradually converging to the true solution for the PDE. The general iterative update is

$$u^{(t+1)} = u^{(t)} + C \left(f_h - \mathcal{L}_h^a u^{(t)} \right), \quad (3)$$

where $u^{(t)}$ is the t^{th} iterate of the solution and C is a preconditioning matrix. Jacobi uses $C = D^{-1}$, where D is the diagonal of $\mathcal{L}_h^a$, and Gauss-Seidel uses $C = (D + L)^{-1}$, where L denotes the strict lower triangular part. However, these methods tend to damp low frequency parts of the error slowly.

2.2 Neural Operators

Suppose we observe samples $\{(a_i, f_i, u_i)\}_{i=1}^N$ such that $(a_i, f_i) \sim \mathcal{P}$ are i.i.d. samples, and $u_i = \mathcal{G}^*(a_i, f_i)$ be generated by a deterministic solution operator $\mathcal{G}^*$. A neural operator (NO) $\mathcal{G}_\theta$ seeks to approximate $\mathcal{G}^*$ by minimizing the expected squared $L^2(D)$ error:

$$\mathcal{R}_{\text{NO}}(\mathcal{G}_\theta) = \mathbb{E}_{(a,f) \sim \mathcal{P}} \left[\int_D \|\mathcal{G}^*(a, f)(x) - \mathcal{G}_\theta(a, f)(x)\|_2^2 dx \right]$$

Neural operators use discretization-invariant layers. Prominent instantiations include Fourier Neural Operators (FNO), which use spectral transforms [Li et al., 2020a], Graph Neural Operators, which perform graph-based aggregation over sampled points [Li et al., 2020b], and DeepONets, which employ a trunk-branch decomposition to map input functions to output functions [Lu et al., 2021].

2.3 Greedy Optimization

Consider maximizing $g(S)$ over $S \subseteq \Omega$ with $|S| \leq T$ where $g : 2^\Omega \rightarrow \mathbb{R}$ is a set function defined on subsets of a set Ω . An exhaustive search is infeasible, so an efficient alternative is the greedy algorithm. It builds the solution subset iteratively by adding ω^* that yields the largest marginal gain, i.e. $\omega^* = \arg \max_{\omega \in \Omega \setminus S} g(S \cup \omega)$. When g is non-negative, monotone (adding elements never decreases the value), and submodular (diminishing returns), the greedy algorithm achieves a $(1 - e^{-1})$ approximation to the optimal solution [Nemhauser et al., 1978]. Formally, a function g is submodular if $g(A \cup \{\omega\}) - g(A) \geq g(B \cup \{\omega\}) - g(B)$ for all $A \subseteq B \subseteq \Omega$ and $\omega \in \Omega \setminus B$; intuitively, delaying an addition cannot increase its benefit. For set function minimization problems, supermodularity, where the inequality flips, plays an analogous role, and the greedy rule achieves constant-factor approximation guarantees [Liberty and Sviridenko, 2017].

The suboptimality of the greedy algorithm has been extensively studied for both set-function minimization [Bounia and Koriche, 2023] and maximization [Das and Kempe, 2018, Feige et al., 2011, Bian et al., 2017, Harshaw et al., 2019] when standard assumptions—non-negativity, monotonicity, and sub/supermodularity—are weakened. In contrast, results on greedy sequence maximization [Streeter and Golovin, 2008, Alaei et al., 2021, Zhang et al., 2015, Bernardini et al., 2020, Van Over et al., 2024, Tschitschek et al., 2017]—where the ordering of the elements affects the function—have led to sequential analogues of submodularity and monotonicity. We leverage these tools to analyze the suboptimality of the greedy solution to minimizing the final error.

3 General framework for Hybrid Solvers

To solve Equation (2), consider the following hybrid iterative update:

$$u_h^{(t+1)} = u_h^{(t)} + C_{S_t} \left(f_h - \mathcal{L}_h^a u_h^{(t)} \right) \quad (4)$$

where $\mathcal{C} = \{C_j\}_{j=1}^K$ is a set of preconditioning functions and $S_t \in [K] = \{1, \dots, K\}$ indexes the function chosen at step t . Here, we use “preconditioning function” broadly to include classical linear updates ($C_j(x) = C_j x$ where C_j is a matrix), and learned models, such as neural operators. This update generalizes the classical iterative update in Equation (3), by allowing C_j to be non-linear and enabling the preconditioning function to be adaptively chosen at every step. $\mathcal{C}$ can also accommodate

parameterized solver families (e.g., different Jacobi relaxation weights), enabling adaptive selection of solver parameters. As the number of solvers K grows, the chance of selecting a more effective update increases. Furthermore, HINTS [Zhang et al., 2024] is a special case of this hybrid solver with $K = 2$, where C_1 is a neural operator and C_2 is a classical preconditioner. Its routing rule is given by $S_t = \mathbf{1}_{(t+1) \bmod \tau > 0} + 1$, which selects C_1 every τ steps (first at iteration τ , after $\tau - 1$ classical steps, as in Algorithm S3 of Zhang et al. [2024]) and C_2 otherwise.

Let $e_h^{(t)} = u_h - u_h^{(t)}$ denote the error at step t . Then,

$$e_h^{(t+1)} = u_h - u_h^{(t)} - C_{S_t} \left(\mathcal{L}_h^a u_h - \mathcal{L}_h^a u_h^{(t)} \right) = (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(t)})$$

where I is the identity map and $I - C_j \circ \mathcal{L}_h^a$ is the error propagation function for solver j . Here, “ $\circ$ ” denotes function composition (matrix multiplication for linear updates). The objective is to select a sequence $S = (S_1, \dots, S_T)$ that minimizes the error norm after T steps or $\|e_h^{(T)}\|_2^2$:

$$\min_{S_t \in [K], |S| \leq T} h(S) := \left\| (I - C_{S_{|S|}} \circ \mathcal{L}_h^a) \circ \dots \circ (I - C_{S_1} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \quad (5)$$

with compositions applied from right to left, so that the S_1 update acts on the initial error.

4 Greedy Algorithm

Equation (5) defines a combinatorial optimization problem with a search space exponential in T , making exact optimization intractable for large T or K . As a starting point, we consider an “omniscient” greedy algorithm that assumes access to the true initial error $e_h^{(0)}$. This is unrealistic since knowledge of $e_h^{(0)}$ could directly recover the solution via $u_h = u_h^{(0)} + e_h^{(0)}$, but it provides a useful benchmark. In Section 5, we relax this assumption using a practical learning strategy that is Bayes consistent with this omniscient rule: for the full-information router, in the infinite-sample limit, it drives the normalised routing risk to its Bayes value, i.e. it recovers the greedy decision in the risk sense on the state distribution it is trained on, which is a per-decision statement (Section 5 states what the feature-based router inherits); the sequential guarantee below is stated for the rule with access to the true error.

As discussed in Section 2.3, when supermodularity and monotonicity hold, the greedy rule—such as the one described in Algorithm 1—enjoys constant-factor approximation guarantees. However, classical results focus on *set* functions, with only recent extensions made to sequences. Building on this line, we introduce a sequence-based notion of weak supermodularity.

Algorithm 1 Greedy Algorithm for a Hybrid PDE solver

Require: $\{C_j\}_{j=1}^K, T, \mathcal{L}_h^a, e_h^{(0)}$
 $S^0 \leftarrow \emptyset$
for $t < T$ **do**
 $S^{t+1} \leftarrow S^t \oplus \arg \min_{j \in [K]} \|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$
 $e_h^{(t+1)} \leftarrow (I - C_{S_{t+1}} \circ \mathcal{L}_h^a)(e_h^{(t)})$
end for
return S^T

Let Ω^* denote the space of sequences with elements in Ω . For $S, S' \in \Omega^*$, we denote their concatenation as $S \oplus S'$. S is a prefix of S' or $S \preceq S'$ if $S' = S \oplus L$ for some $L \in \Omega^*$. A function $g : \Omega^* \rightarrow \mathbb{R}$ is prefix monotonically non-increasing if $g(S \oplus S') \leq g(S)$ for all $S, S' \in \Omega^*$, and postfix monotonically non-increasing if $g(S' \oplus S) \leq g(S)$ for all $S, S' \in \Omega^*$. A prefix non-increasing function g is sequence supermodular if, for all $S', S \in \Omega^* : S \preceq S'$, it holds that

$$g(S) - g(S \oplus \omega) \geq g(S') - g(S' \oplus \omega), \quad \forall \omega \in \Omega \quad (6)$$

However, $h(S)$ (defined in Equation (5)) may not satisfy this property in general. Therefore, we introduce weak sequence supermodularity. A prefix non-increasing function g is weakly supermodular with respect to $S' \in \Omega^*$ if there exists a single constant $\alpha(S') \geq 1$ such that, for every $S \in \Omega^*$,

$$g(S) - g(S \oplus S') \leq \alpha(S') \sum_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \quad (7)$$

218 The parameter $\alpha(S')$ or the supermodularity ratio quantifies deviation from exact sequence super-
 219 modularity. Expanding the $g(S) - g(S \oplus S')$ as a telescoping sum $\sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) -$
 220 $g(S \oplus (S'_1, \dots, S'_i))$ shows that the marginal decrease from appending S'_i after its predecessors is
 221 controlled by the effect of appending S'_i directly to S . Thus, postponing the inclusion of S'_i cannot
 222 yield a significantly larger benefit than adding it earlier. The supermodularity ratio $\alpha(S)$ quantifies
 223 the extent to which future gains from delays may exceed immediate gains.

224 **Postfix monotonicity enters the analysis only through the greedy prefixes, so we also allow a slack:**
 225 **g is μ -postfix monotone with respect to O along a sequence S^T if $g(S^t \oplus O) \leq \mu g(O)$ for every**
 226 **prefix S^t , $t < T$; a postfix non-increasing function satisfies this with $\mu = 1$, and μ can be computed**
 227 **exactly for short horizons (Section E.10). Having introduced these notions, we now characterize the**
 228 **suboptimality of greedy solutions of weakly supermodular and postfix monotonic sequence functions**
 229 **in Theorem 4.1.**

230 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a non-negative, prefix non-increasing function that is weakly*
 231 *supermodular with respect to the optimal solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity*
 232 *ratio of $\alpha(O)$, and let the greedy solution of length T be S^T . Suppose g is μ -postfix monotone with*
 233 *respect to O along S^T ($\mu = 1$ if g is postfix non-increasing). If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then*

$$g(S^T) \leq \mu(1 - \phi_T(\alpha(O)))g(O) + \phi_T(\alpha(O))g(\emptyset), \quad \text{i.e.,} \quad g(\emptyset) - g(S^T) \geq (1 - \phi_T(\alpha(O))) (g(\emptyset) - \mu g(O)).$$

234 The proof of Theorem 4.1 appears in Section A.2. As $T \rightarrow \infty$, the factor $1 - \phi_T(\alpha(O))$ decreases
 235 to $1 - e^{-1/\alpha(O)}$, so, for a fixed α , the worst-case performance of the greedy rule saturates rather
 236 than degrades with horizon. Since $\alpha(O)$ and μ are defined through the optimal sequence of length
 237 T , a horizon-uniform statement needs uniform bounds $\alpha(O_T) \leq A$ and $\mu_T \leq M$, under which the
 238 guaranteed reduction is at least $(1 - e^{-1/A}) [g(\emptyset) - M g(O_T)]$ whenever that bracket is nonnegative,
 239 and only the trivial nonnegative reduction of prefix monotonicity otherwise. The bound concerns
 240 the reduction of the error over T steps relative to the best sequence of T steps: with $\alpha = \mu = 1$ it
 241 guarantees a $(1 - e^{-1})$ fraction of the optimal reduction, uniformly over ensembles, but it is not a
 242 convergence rate—a single convergent solver applied T times already reduces the error by ρ^{2T} , which
 243 for large T is far below the $e^{-1}g(\emptyset)$ term. Section E.10 reports, for short horizons, how far the greedy
 244 sequence is from the exhaustive optimum in the settings of Section E. Additionally, larger $\alpha(O)$,
 245 which indicates higher reward for delayed inclusions, loosens the suboptimality bound. Theorem 4.1
 246 requires that the sequence objective h be weakly supermodular with respect to the optimal solution
 247 and postfix monotone—properties established in Proposition 4.2. We defer the proof to Section A.3.

248 **Proposition 4.2.** *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is*
 249 *ρ_j -Lipschitz continuous with $\rho_j \leq 1$, that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$, and that $\sum_{i=1}^T (1 - \rho_{O_i}^2) > 0$,*
 250 *i.e., at least one action of the optimal sequence O is a strict contraction. Then, the function h is*
 251 *weakly supermodular with respect to the optimal solution O , with*

$$\alpha(O) = \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

252 *Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by a*
 253 *unitary matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also postfix*
 254 *non-increasing; invertibility alone does not suffice (Section A.3).*

255 The conditions of Proposition 4.2 are natural. The corrector of Section 7 leaves the modes outside its
 256 band untouched and has $\rho = 1$ exactly; the proposition permits this provided the optimal sequence
 257 contains at least one action that is a strict contraction in the norm of h (the Euclidean norm) on
 258 a subspace that contains the initial error and is invariant under every action. For the Jacobi-type
 259 smoothers the mean-free, Nyquist-free subspace is such a subspace and they are strict contractions
 260 on it; for Gauss–Seidel, SOR and multigrid neither the invariance of that subspace nor contraction
 261 in the Euclidean norm is established in our settings (SOR(1.5) expands on it, Section E.10), so
 262 for them Proposition 4.2 in the norm of h is not established. For the exact common-unitary case
 263 Proposition 4.3 applies without any strict contraction. For classical solvers, the Lipschitz constant is
 264 $\|I_N - C_j \mathcal{L}_h^a\|$, which is typically less than 1 for well-posed linear elliptic PDEs (i.e., the update is
 265 damping errors). For neural networks, Lipschitz continuity can be enforced via weight regularization
 266 [Gouk et al., 2021] and a sufficiently trained model should approximate $(\mathcal{L}_h^a)^{-1}$ well enough to make

the Lipschitz constant small. The requirement $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$ is both natural and desirable: it precludes spurious updates when the residual $f_h - \mathcal{L}_h^a u_h^{(t)}$ is 0. This holds by design for classical schemes, and it can be enforced for any learned model by excluding bias terms.

The form of $\alpha(O)$ in Proposition 4.2 highlights that the suboptimality factor in Theorem 4.1 is governed by the collective contraction factors of the solvers chosen by the optimal solution: as $\sum_{i=1}^T \rho_{O_i}^2 \rightarrow T$, $\alpha(O)$ grows, resulting in a weaker bound. Postfix monotonicity ($\mu = 1$) holds when the error propagation maps share a unitary eigenbasis—constant-coefficient periodic operators with Jacobi-type smoothers and band-limited spectral correctors—but it can fail for non-commuting contractions even when all of them are invertible, and a Gauss–Seidel sweep or a learned corrector need not be invertible at all. For the non-commuting solvers of our experiments no a priori bound on μ is available; Section E.10 computes μ , $\alpha(O)$ and the bound of Theorem 4.1 by exhaustive enumeration for short horizons ($T \leq 4$ macro-actions) on every pairing and on four representative ensembles, including ones with Gauss–Seidel, SOR and multigrid, and reports the largest μ ([pending]), the largest candidate $\alpha(O)$ over greedy prefixes ([pending]) and the number of instance–ensemble pairs on which the on-path premises of the proof were verified numerically ([pending]); at the horizons of tens to hundreds of macro-actions at which the oracle actually runs, neither α in the Euclidean norm nor μ is established for the non-normal sweeps, and no guarantee is claimed for them.

Theorem 4.1 thus indicates that the approximation guarantee is strongest when the sequence is nearly supermodular ($\alpha(O) \approx 1$). Generally, if all error propagation maps share an eigenbasis, supermodularity is established. This occurs, for example, for linear, constant-coefficient PDEs with periodic boundary conditions when the solver ensemble consists of (damped) Jacobi iterations and band-limited spectral correctors such as a single-layer linear Fourier neural operator, all of which are diagonal in the Fourier basis; lexicographic Gauss–Seidel and SOR are not, because their sweep order breaks translation invariance. The proof of Theorem 4.3 is deferred to Section A.5.

Proposition 4.3. *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common unitary matrix P (orthogonal in the real symmetric case; the Jacobi matrix of the convection–diffusion operator is a real circulant with complex eigenvalues, hence unitarily but not orthogonally diagonalisable). Then, h is supermodular.*

Cost-aware greedy routing. Algorithm 1 treats every update as equally expensive, whereas in practice a neural-operator evaluation and a classical sweep have very different costs, and the quantity of interest is the error reached per unit of wall-clock time. We make the greedy rule cost-aware by changing its action set, not its comparison: let c_j denote the (measured) cost of one application of C_j , take the cost u of one neural-operator call as the unit, and replace each preconditioner by the macro-action $C_j^{(m_j)}$ that applies C_j for $m_j = \max\{1, \text{round}(u/c_j)\}$ consecutive iterations, so that every action not dearer than the unit costs u up to rounding ($m_j c_j / u$ within $1/(2m_j - 1)$ of 1). For a linear update, m consecutive sweeps are again a stationary iteration, $(I - C_j \circ \mathcal{L}_h^a)^m = I - C_j^{(m)} \mathcal{L}_h^a$ with $C_j^{(m)} = \sum_{i=0}^{m-1} (I - C_j \mathcal{L}_h^a)^i C_j$, and for a nonlinear C_j the macro-action is the m -fold composition of its error propagation map. Hence $\{C_j^{(m_j)}\}_{j=1}^K$ is again a set of preconditioning functions in the sense of Section 4: if $I - C_j \circ \mathcal{L}_h^a$ is ρ_j -Lipschitz and zero-preserving, then so is its m_j -fold composition (with constant $\rho_j^{m_j}$), a shared eigenbasis is inherited by powers, and Theorem 4.1, Propositions 4.2 and 4.3, and the consistency result of Section 5 apply verbatim to an ensemble of macro-actions, all of which are equalised to the unit cost. The deployed rule is exactly Algorithm 1 on these macro-actions—the plain comparison of the errors after one macro-action—and a horizon of T macro-actions corresponds to a wall-clock budget of $\approx Tu$; it selects, at every decision epoch, the action with the largest error reduction per unit cost up to the rounding of the m_j . The one exception is an action dearer than the unit ($c_j > u$, hence $m_j = 1$), whose error reduction is compared per unit of cost, $(\|e_j\|/\|e^{(t)}\|)^{u/c_j}$: this is a geometric-rate heuristic rather than an equal-cost step, and the analysis does not cover it. Which actions are dearer than the unit is a fact of the measured cost tables (Tables 7, 13 and 19): on the 128^2 grids measured so far (Poisson, convection–diffusion, anisotropic diffusion) none is, so those cells run Algorithm 1 on macro-actions, which is also the rule that the exhaustive check of Section E.10 verifies (variable-coefficient diffusion [pending]); on the larger grids only a multigrid cycle can be dearer than the unit ([pending]), and in the granularity ablation of Section E.6 the corrector itself is. Even with all exponents equal to one, the wall-clock reading of a horizon of T macro-actions is approximate: the rounding of the m_j and the router’s decision

cost are not part of the equal-count analysis. (An earlier version applied the exponent $u/(m_j c_j)$ to every action, including the rounding error of the cheap ones; that rule is not Algorithm 1, and it was changed so that the theory covers what is deployed.) Section E uses this cost-aware instantiation throughout.

5 Approximate Greedy Router

The results in Section 4 indicate that the error reduction from the greedy solution closely matches that of the optimal sequence, but this is predicated on having access to the initial error, $e_h^{(0)}$. Inaccurate error estimates can result in poor solver decisions, causing errors to amplify. To remedy this, we learn a router r that selects solvers myopically, as in Algorithm 1, without access to true errors.

We adopt the following learning setup. Let $\mathcal{A}, \mathcal{F}, \mathcal{U} \subseteq \mathbb{R}^N$ denote the spaces of coefficient, forcing, and solution functions on the grid $G_h(D)$. We assume an application-specific data distribution $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$ over $\mathcal{A} \times \mathcal{F}$ that reflects test time conditions. During training, (a_h, f_h) is drawn from $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$, and a high-accuracy reference solution u_h is computed, providing the true per-step error. The router r is learned offline on this data; at test time, it operates without access to true errors.

At iteration t , router r selects a solver using the coefficient $a_h \in \mathcal{A}$, forcing $f_h \in \mathcal{F}$, and the current iterate $u_h^{(t)} \in \mathcal{U}$. If $r(a_h, f_h, u_h^{(t)}) = j$, the next iterate is computed with solver j , resulting in an error of $\|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$. Learning such a router requires minimizing the following loss:

$$l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) = \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)})=j} \quad (8)$$

with risk $\mathcal{R}_{\text{route}}(r) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} [l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h)]$. For analysis, we fix the iterate generation via teacher-forcing [Williams and Zipser, 1989, Lamb et al., 2016]: during training, the iterates fed to the router are produced by an oracle greedy rollout, not by the router's own past choices. Formally, with $u_h^{(0)} = 0_N$, for $t \geq 1$,

$$j_t^* \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t-1)}) \right\|_2^2, \quad u_h^{(t)} = u_h^{(t-1)} + C_{j_t^*} \left(f_h - \mathcal{L}_h^a u_h^{(t-1)} \right)$$

Thus, $\{u_h^{(t)}\}$ is a deterministic function of (a_h, f_h) . At each t , l_{route} is evaluated on $r(a_h, f_h, u_h^{(t)})$ which takes in the teacher-forced iterate. Finally, the greedy choice j_{t+1}^* is used to advance $u_h^{(t+1)}$. Thus the input distribution seen by r depends on (a_h, f_h) , not on the router's predictions. However, full teacher forcing induces a distributional mismatch at test time (exposure bias). In experiments, we mitigate this with scheduled sampling [Bengio et al., 2015]; see Section C for details.

Minimizing $\mathcal{R}_{\text{route}}(r)$ yields the following Bayes-Optimal Router:

$$r^*(a_h, f_h, u_h^{(t)}) \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 \quad (9)$$

which aligns with the update in Algorithm 1. Hence, l_{route} is consistent with learning the greedy rule.

5.1 Surrogate Loss

Since Equation (8) is discontinuous and non-convex, direct minimization is intractable in practice. We instead introduce a surrogate loss that (1) is convex in the scores; (2) upper bounds the original loss; and (3) is Bayes consistent, ensuring the Bayes optimal decision of the original loss is preserved upon minimization. Formally, a surrogate ϕ is considered to be Bayes consistent with respect to the loss l if

$$\lim_{n \rightarrow \infty} \mathcal{R}_\phi(f_n) - \mathcal{R}_\phi^* = 0 \implies \lim_{n \rightarrow \infty} \mathcal{R}_l(f_n) - \mathcal{R}_l^* = 0$$

where $\mathcal{R}_l(f) = \mathbb{E}[l(f(X), Y)]$ and $\mathcal{R}_l^* = \inf_f \mathcal{R}_l(f)$. In other words, in the limit of infinite data, if the risk of a sequence of learned hypotheses $\{f_n\}$ converges to the optimal risk under ϕ , it also converges to the optimal risk with respect to the original loss l .

To define a surrogate loss for the routing problem, consider a set of scoring functions $\mathbf{g} = \{g_j\}_{j=1}^K$ with $g_j : \mathcal{A} \times \mathcal{F} \times \mathcal{U} \rightarrow \mathbb{R}$ and we define the router as $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$. For

example, $\mathbf{g}$ can be a neural network with K outputs. Then, minimizing the following surrogate loss yields the same decision as minimizing Equation (8):

$$\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h) = - \sum_{j=1}^K \sum_{k=1}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h) \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{m=1}^K \exp(g_m(a_h, f_h, u_h^{(t)}))} \right) \quad (10)$$

where $\tilde{c}_j(a_h, u_h^{(t)}, u_h) = \|(I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)})\|_2^2$. The Ψ -risk is denoted by $\mathcal{R}_\Psi(\mathbf{g}) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}}[\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)]$. The convexity of Ψ with respect to the score vector $\mathbf{g}$ follows from the convexity of the negative log-softmax function in its inputs (the surrogate is convex in the scores, not in the parameters of a network that produces them). Moreover, Ψ upper bounds l_{route} up to a constant factor, and we refer the reader to Section B.2 for the proof. Finally, Theorem 5.1 shows that the per-state normalised surrogate $\hat{\Psi}$ is Bayes consistent with respect to the normalised routing loss $\hat{l}_{\text{route}}$, which has the same Bayes decision as l_{route} at every state; the proof can be found in Section B.3.

Theorem 5.1. *Let $K \geq 2$ and let the per-step costs be normalised state by state, $\hat{c}_j = \tilde{c}_j / \sum_{k=1}^K \tilde{c}_k$, at every state at which $\sum_k \tilde{c}_k > 0$, i.e., at which at least one solver does not annihilate the error (states at which every solver annihilates it carry no routing loss and are discarded), and let $\hat{l}_{\text{route}}$ and $\hat{\Psi}$ denote the routing loss and the surrogate with $\tilde{c}_j$ replaced by $\hat{c}_j$; the normalisation does not change the Bayes-optimal decision equation 9 at any state. Then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, $\hat{\Psi}$ is a Bayes consistent surrogate for $\hat{l}_{\text{route}}$.*

Theorem 5.1 is a cost-sensitive analogue of the classical Bayes-consistency of multiclass cross-entropy for 0-1 loss: in the infinite-sample limit, minimizing cross-entropy recovers the true conditional class probabilities, so the induced decision is Bayes optimal. Our result extends this to cross-entropy with instance-dependent weights $\sum_{k \neq j} \hat{c}_k(a_h, u_h^{(t)}, u_h)$. The normalisation is what the router is trained with (Section E.1) and what makes the calibration constants meaningful: an earlier version assumed uniform absolute bounds $E_{\min} \leq \max_j \tilde{c}_j \leq \bar{E}$, which cannot hold along a convergent solve, where every $\tilde{c}_j \rightarrow 0$; with the normalised weights the only hypothesis is that at least one solver leaves a positive fraction of the error at every visited state, which Section E.10 checks. Under this condition, minimizing $\hat{\Psi}$ recovers the Bayes-optimal router of Equation (9), i.e., the greedy solution of Equation (5). The proof bounds the conditional routing excess by the square root of the conditional surrogate excess through Pinsker’s inequality, a direct calibration argument in the spirit of Mao et al. [2023], and takes expectations; it concerns unrestricted measurable scores, not a fixed network class or optimiser. This is a per-decision statement on the state distribution the router is trained on: it says that minimising $\hat{\Psi}$ drives the normalised routing risk to its Bayes value, i.e. recovers the greedy decision in the risk sense, not that the induced sequence, its time to tolerance, or its distance to the optimal sequence are controlled; the latter would require a sequential argument about the router’s own state distribution, which we do not provide and mitigate empirically with DAgger (Section E). The deployed router, moreover, does not see $(a_h, f_h, u_h^{(t)})$ but a few scalar features of the residual history (Section E.1); with respect to those features the Bayes-optimal router minimises the conditional expectation of the normalised costs given the features, which can differ from the full-information greedy decision when states with different error spectra share the same features, and the normalisation preserves the decision state by state but not necessarily after that averaging. Theorem 5.1 therefore concerns the full-information router, and the price of the feature compression is measured rather than proved, by the agreement between the deployed router and the oracle (Section E).

6 Related Works

Hybrid PDE Solvers: Early data-driven solvers sought convergence guarantees by predicting parameters—e.g., preconditioning matrices, multi-grid smoothers or restriction matrices—within iterative schemes [Taghibakhshi et al., 2021, Caldana et al., 2024, Kopaničáková and Karniadakis, 2025, Huang et al., 2022, Katrutsa et al., 2020]. These works, however, do not leverage the neural surrogates’ ability to generalize across varying coefficients and forcings highlighted in Section 2.2 and thus, offer only modest speedups over classical solvers. HINTS [Zhang et al., 2024] introduced hybrid solvers that interleave a classical method with a pre-trained DeepONet on a fixed schedule (e.g., 24 Jacobi

steps, then one DeepONet correction), achieving faster convergence than the standalone numerical solver. Subsequent works extend this idea to new geometries [Kahana et al., 2023] and analyze error mode damping, replacing DeepONet with alternatives such as MIONet [Jin et al., 2022] [Hu and Jin, 2025] or FNO [Li et al., 2020a, Cui et al., 2022]. However, these hybrid solvers are limited in two ways: they, firstly, only combine the trained surrogate with a *single* numerical solver, and secondly rely on a fixed, heuristic schedule. Our proposed method addresses these shortcomings, resulting in significant empirical improvements (Section 7).

Model Routing: Routing [Shnitzer et al., 2023, Hu et al., 2024, Ding et al., 2024, Huang et al., 2025] selects, for each input, a model from a fixed set to optimize a task metric under cost or latency constraints. Simple heuristics—e.g., thresholding a cheap model’s uncertainty estimates [Chuang et al., 2024, 2025]—often poorly balance the cost-accuracy tradeoff, motivating many systems to instead learn a router that maps inputs to the best model [Hari and Thomson, 2023, Mohammadshahi et al., 2024, Šakota et al., 2024]. We similarly learn a router, but over numerical and neural solvers for PDEs at the iteration level. Our work is the first to apply routing to the problem of learning hybrid PDE solvers, in contrast to prior approaches that use fixed schedules. Additionally, by exploiting the algebraic structure of PDE solvers, we derive theoretical guarantees for our routing strategy (Section 4), unlike typical model-routing settings.

Mixture of Experts: Classical mixture-of-experts (MoE) [Jacobs et al., 1991, Jordan and Jacobs, 1994] uses a gating network to combine the outputs of multiple experts, trained jointly with the gate. In modern LLMs, MoE instead performs sparse, token-level routing to a subset of in-layer experts, enabling large capacity without proportional compute, typically with load-balancing mechanisms [Shazeer et al., 2017, Lepikhin et al., 2020, Fedus et al., 2022, Jiang et al., 2024]. Our method can be regarded as gating of a different kind: “experts” (solvers) are not jointly trained with the router, and a single router is reused across iterations, unlike MoE which commonly employs layer-specific gates.

7 Experiments

We evaluate the approximate greedy router on four PDEs posed on the unit domain $D = [0, 1]^2$ with periodic boundary conditions:

$$\text{Poisson: } -\Delta u = f, \quad \text{ConvDiff: } -\Delta u + 20 \partial_{x_1} u + 20 \partial_{x_2} u = f, \quad \text{AnisoDiff: } -\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f, \quad \epsilon = 10^{-2}, \quad \text{VarCoef}$$

where f is centred to satisfy the compatibility condition $\int_D f(\mathbf{x}) d\mathbf{x} = 0$ and is drawn from a zero-mean hierarchical Gaussian random field with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$ and sample-dependent (α, β, γ) (Section C). Anisotropic diffusion is the standard problem on which point relaxation fails to smooth: an error mode oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep. Variable-coefficient diffusion uses a fixed smooth coefficient field $a(\mathbf{x}) = \exp(\kappa z(\mathbf{x}))$ whose contrast $\max a / \min a$ is at most 10 by construction (9.99 measured): its discrete operator is not diagonalised by the FFT, so no direct spectral solve exists, the corrector’s band-limited inverse has no closed form and is fitted, and the fitted corrector is only approximately accurate (Section E.5). The domain is discretised on 128^2 , 256^2 and 512^2 grids; Dirichlet boundary conditions are not covered by this study (Section D). Because iteration counts favour methods whose iterations are expensive, our primary metric is the *wall-clock time to a target relative error* charged to each method in a replay of its decisions (Section E.1) $\varepsilon = \|e_h^{(t)}\| / \|u_h\|$ on 64, 32 and 16 test instances per grid, with $\varepsilon = h^2$ —a grid-dependent tolerance chosen to lie below the discretisation error of the test instances (the smallest ratio of the estimated discretisation error to h^2 over the confirmatory instances is [pending], Section E.1)—highlighted; paired comparisons of times use two-sided Wilcoxon signed-rank tests with a Holm correction and bootstrap intervals, the AUC comparisons one-sided paired t -tests on log AUC. We also report the iteration-based metric of the original submission, the area under the error curve, $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\| / \|u_h\|$ over $T = 300$ iterations. Section E gives the full protocol and collects every result per equation.

Setup. The classical solvers are Jacobi, damped Jacobi ($\omega = 0.67$), Gauss–Seidel (GS), symmetric GS (SymGS), SOR ($\omega = 1.5$), line Gauss–Seidel (anisotropic diffusion only: every grid line along the strongly coupled axis solved exactly, the classical remedy for anisotropy) and geometric multigrid (V(2,2)-cycles with GS smoothing; line-GS smoothing for anisotropic diffusion), all implemented as

compiled single-threaded C stencil kernels (a GS sweep costs $61 \mu\text{s}$ and a V-cycle $326 \mu\text{s}$ at 128^2 , measured in the form in which the benchmark charges them), so the classical baselines are not slowed by the implementation. The neural operator is a DeepONet corrector that acts on the residual through a 64×64 sensor grid with a Fourier trunk basis and is applied scale-equivariantly: its deployed form is a linear, Fourier-structured map (a least-squares fit of the band-limited inverse of the discrete operator, which it reproduces to $\approx 10^{-6}$ on the constant-coefficient equations), so one call solves the smooth band of the spectrum at the cost of about 32 Jacobi or 11 GS sweeps at 128^2 (and [pending] Jacobi sweeps at 512^2 , since its cost is nearly grid-independent) and never injects error outside its band. The router is made *cost-aware* through the cost-equalised macro-actions of Section 4 and selects from $6+K$ scalar features (log residual, its recent decay rate, the previous action and call counts) with a two-layer MLP whose decision costs 16–21 μs in situ, independently of N ; it is trained with the surrogate of Section 5.1 to imitate the cost-aware greedy oracle on oracle rollouts followed by DAGger rounds. We compare against the classical solver alone; HINTS with the period $\tau=15$ that Zhang et al. [2024] use in all of their two-dimensional experiments ($\tau=25$, their one-dimensional period, is reported in Section E); the best fixed schedule chosen *a posteriori* on the test set for each cell from HINTS with $\tau \in \{2, 5, 10, 15, 25, 50\}$, phase-shifted HINTS (first call at iteration 0) and the static one-shot schedule (corrector once at iteration 0, then the solver); the best of three residual-decay rules—deterministic controls at the router’s granularity that call the corrector again whenever the residual’s per-iteration contraction over the last macro-action was worse (closer to one) than a threshold, also chosen on the test set; the cost-aware greedy oracle (Algorithm 1 with the true error; not deployable); and the classical methods a practitioner would use on these problems without a corrector—multigrid alone (line-smoothed for anisotropic diffusion), multigrid-preconditioned CG or BiCGSTAB, and the FFT or sparse direct solve—timed inside the same replay loop as the policies. The forcings are sampled from the hierarchical GRF with the Hermitian symmetry of a real field and without Nyquist modes (Section E.1). All configurations were fixed on a development test set, and every number reported is a single confirmatory evaluation on a fresh test set with three timed replays in random order and a drift guard per instance (Section E.9); paired comparisons use two-sided Wilcoxon tests with a Holm correction and bootstrap intervals.

Paired solvers. Table 1 reports, for every pairing $\{\text{NO}, \text{solver}\}$, equation and grid, the median wall-clock time to $\varepsilon = h^2$ of HINTS, of the best fixed schedule, and of the learned router, together with the classical methods without corrector. Every ratio below is a paired median time ratio (baseline over router; above one favours the router; the one exception, the router-to-oracle ratio, is a ratio of medians) computed over the 128^2 pairwise cells unless a grid is named, and “faster in n of m ” counts pairings whose ratio exceeds one irrespective of significance (the tests are in Section E). Two regimes appear. At *truncation level* the router makes exactly one corrector call on 80–100% of the instances (Table 6), executes the same decision sequence to h^2 as the cost-aware oracle on 50–100% of the instances and as the best static schedule ($\tau=10$ (first call at 0) (7), $\tau=50$ (first call at 0) (5), one-shot (5), $\tau=15$ (first call at 0) (4), $\tau=25$ (first call at 0) (2), $\tau=5$ (first call at 0) (1) at h^2 , pairings in parentheses) on 67–100%, and its ratios against the best fixed schedule found by a test-set search are $0.94 \times$ – $0.99 \times$ at 128^2 ([pending]–[pending] at 256^2 , [pending]–[pending] at 512^2), a difference consistent with its decision overhead (Section E.8); against HINTS with its published period $\tau=15$, whose first call comes $\tau - 1$ sweeps too late, they are $0.95 \times$ – $5.11 \times$ ([pending]–[pending] at 256^2 , [pending]–[pending] at 512^2 ; $0.95 \times$ – $7.08 \times$ against $\tau=25$); against the best residual-decay rule, which also calls the corrector first, $0.95 \times$ – $0.97 \times$; and against the classical solver alone $0.95 \times$ – $7083 \times$ (censored runs entered at the cap; the low end is the Multigrid pairing; anisotropic diffusion [pending]). Below *truncation level* the schedule matters and the picture is pairing-dependent: down to $\varepsilon = 10^{-8}$ the ratios are $0.92 \times$ – $3.07 \times$ against HINTS ($\tau=15$; the router is faster in 22 of 24 pairings) and $0.97 \times$ – $1373 \times$ against the one-shot schedule (faster in 21), because the router calls the corrector again when the smooth error has re-emerged (Figure 1), but $0.83 \times$ – $1.69 \times$ against the best fixed schedule (faster in 8) and $0.98 \times$ – $2.07 \times$ against the best residual-decay rule (faster in 21). The best fixed schedules at 10^{-8} are $\tau=15$ (first call at 0) (9), one-shot (4), $\tau=5$ (first call at 0) (4), $\tau=10$ (first call at 0) (3), $\tau=25$ (first call at 0) (2), $\tau=50$ (1), $\tau=10$ (1) (number of pairings in parentheses): a periodic schedule with a short period calls the corrector more often than a cost-equalised macro-action of 11 Gauss–Seidel or 32 Jacobi sweeps allows the router to, and Section E.6 tests whether that granularity explains the losses [pending]. The ratio of the router’s median time to the cost-aware oracle’s is $1.01 \times$ – $1.09 \times$, and its decision sequence to 10^{-8} is identical to the oracle’s on 30–94% of the instances; the stability of its decisions over five independent training seeds is reported in Section E [pending]. Against the classical methods

without corrector, the ratio of multigrid alone to the router is $0.95 \times -11.2 \times$ at h^2 and $0.03 \times -20.5 \times$ at 10^{-8} , that of multigrid-preconditioned CG or BiCGSTAB $0.95 \times -3.00 \times$ and $0.03 \times -3.46 \times$, and routing over {NO, multigrid} has ratios $0.94 \times -2.30 \times$ and $1.11 \times -1.63 \times$ against multigrid alone; on the constant-coefficient equations the router takes $3.08 \times -3.25 \times$ times as long as the FFT direct solve, and on variable-coefficient diffusion, which has no such solve, it is compared with the sparse direct solve (Section E.5). For the iteration-based metric of the original submission, at 128^2 the mean AUC over $T = 300$ iterations of HINTS and of the solver alone are $3-174709 \times$ and $3-2666251 \times$ the router’s (one-sided paired t -tests on log AUC, $p \leq 10^{-10}$; Tables 8, 14, 20 and 26).

Solver ensembles. The same construction routes over ensembles $\text{NO} \cup \mathcal{W}$ of several classical solvers. Since the greedy rule is a per-decision comparison, a larger ensemble can only help where the error passes through regimes that favour *different* members; an oracle-level screening of every subset of up to three members (Section E.7) showed, in the development runs, that this never happens at $\varepsilon = h^2$ —one corrector call and one macro-action of the cheapest smoother already reach truncation level—and happens at 10^{-8} only through a nonstationary Jacobi schedule (undamped Jacobi on the band-edge modes, damped Jacobi on the near-Nyquist modes). We evaluate the learned router over the nested sets {Jacobi}, {Jacobi (0.67)}, {Jacobi, Jacobi (0.67)} and {Jacobi, Jacobi (0.67), GS} and the four larger sets of Table 30 on every equation, with the single-member routers evaluated in the same session on the same instances. The revised-protocol ensemble cells are [pending]. In the first confirmatory run (revision 1, 39-sweep Jacobi macro-actions) no ensemble was significantly faster than the best single-member router at 128^2 and the ensemble oracle was within 6% of the best single-member oracle, which we attributed to the decision granularity: a single damped-Jacobi macro-action of 32 sweeps contracts the band-edge modes by $0.665^{32} \approx 2 \times 10^{-6}$, which suggests that the band the corrector leaves at 128^2 is removed within one or two macro-actions, leaving a nonstationary schedule little to exploit. Whether this holds under the revised protocol, on the 256^2 grids (twice as wide a band) and at a finer granularity (Section E.6) is what the pending cells decide; the ensemble routers’ ratios against the best single-member router at h^2 and 10^{-8} will be reported as $1.00 \times -1.00 \times$ and $0.74 \times -1.00 \times$. Finally, Section E.10 checks numerically which assumptions of Sections 4 and 5 hold in which setting: exactly for the Jacobi-type ensembles with the exact band-limited inverse on the constant-coefficient equations, and only as finite diagnostics for the fitted corrector, the non-normal sweeps and multigrid, for which no guarantee is claimed.

Figure 4 shows the individual operation sequences of the learned router on every test instance, next to HINTS on the same instances. Every result per equation and grid, the ensemble study, the accounting of overheads and training cost, and the verification of the theoretical assumptions are collected in Section E; Section D reports the Dirichlet experiment of the original submission.

8 Discussion

We have introduced an adaptive method for selecting solvers that efficiently minimizes the final error of a PDE in iterative methods, opening many directions for future work. Our current DeepONet is trained without accounting for its downstream role in correction term prediction; jointly training the ML solver and router could yield larger gains. Another avenue is to use reinforcement learning to learn a cost-aware routing strategy that optimizes error under compute budgets; Sections 4 and E show that a myopic cost-aware variant of the greedy rule—Algorithm 1 on cost-equalised macro-actions—executes the decision sequence of the best static schedule at truncation level on 67–100% of the instances and gives pairing-dependent results below it (Section 7), and RL could extend it to non-myopic budgeted routing. RL would additionally enable extending the ensemble routing to continuous parameterization of solvers. Finally, framing routing as an online learning problem could enable adaptation to unknown deployment conditions.

References

- Eugenia Kalnay. *Atmospheric modeling, data assimilation and predictability*. Cambridge university press, 2003.
- Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.

Table 1: Median wall-clock time to truncation-level relative error $\varepsilon = h^2$ for every solver pairing $\{\text{NO, solver}\}$, equation and grid ($64, 32$ and 16 test instances at $128^2, 256^2$ and 512^2): HINTS with its published two-dimensional period $\tau=15$, the best fixed schedule chosen per cell on the test set itself from HINTS ($\tau \in \{2, 5, 10, 15, 25, 50\}$), phase-shifted HINTS ($\tau \in \{5, 10, 15, 25, 50\}$, first call at 0) and the one-shot schedule (subscript), and the learned router (paired median speedup over HINTS-15/over the best fixed schedule in parentheses; bold: both ratios at least one). The paired ratio of the classical solver alone to the router is $0.95 \times -7083 \times$ at 128^2 (above one favours the router; censored runs entered at the cap; the low end is the Multigrid pairing; superscripts count censored and invalid runs as in Section E.1; intervals and tests are in the appendix speed tables, this table being descriptive; Section E). The last rows of each block give geometric multigrid alone (point-smoothed; line-smoothed as well for anisotropic diffusion), the best multigrid-preconditioned Krylov method and the direct solve, without corrector, timed in the same session on the same instances. Anisotropic diffusion is not run at 512^2 : its corrector keeps full resolution along x_1 , and the resulting $65,536 \times 65,536$ branch matrix does not fit in memory (Section E). $\dagger n$: n instances did not reach h^2 within the iteration cap.

Equation	Pairing	128 $\times$ 128			256 $\times$ 256			512 $\times$ 512		
		HINTS-15	best schedule	router (ours)	HINTS-15	best schedule	router (ours)	HINTS-15	best schedule	router (ours)
Poisson	Jacobi	977 μ s	702 μ s $_{\tau=50, t_0=0}$	723 μ s (1.40 $\times$ / 0.97 $\times$)	—	—	—	—	—	—
	Jacobi (0.67)	1.01 ms	718 μ s $_{\tau=50, t_0=0}$	763 μ s (1.32 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	GS	1.58 ms	712 μ s $_{\tau=10, t_0=0}$	758 μ s (2.09 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	SymGS	1.99 ms	725 μ s $_{\text{one-shot}}$	793 μ s (2.57 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	SOR (1.5)	3.06 ms	718 μ s $_{\tau=25, t_0=0}$	777 μ s (3.65 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	Multigrid	1.01 ms	850 μ s $_{\tau=10, t_0=0}$	880 μ s (0.97 $\times$ / 0.97 $\times$)	—	—	—	—	—	—
	Multigrid alone (no corrector)	—	—	1.01 ms	—	—	—	—	—	—
	PCG (MG) (no corrector)	—	—	982 μ s	—	—	—	—	—	—
	FFT direct solve (exact)	—	—	235 μ s	—	—	—	—	—	—
	—	—	—	—	—	—	—	—	—	—
ConvDiff	Jacobi	986 μ s	712 μ s $_{\text{one-shot}}$	746 μ s (1.39 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	Jacobi (0.67)	1.03 ms	750 μ s $_{\tau=10, t_0=0}$	813 μ s (1.26 $\times$ / 0.97 $\times$)	—	—	—	—	—	—
	GS	1.88 ms	794 μ s $_{\text{one-shot}}$	853 μ s (2.30 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	SymGS	2.15 ms	807 μ s $_{\tau=25, t_0=0}$	864 μ s (2.51 $\times$ / 0.94 $\times$)	—	—	—	—	—	—
	SOR (1.5)	3.14 ms	847 μ s $_{\tau=15, t_0=0}$	906 μ s (3.46 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	Multigrid	1.05 ms	1.04 ms $_{\text{one-shot}}$	1.09 ms (0.95 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	Multigrid alone (no corrector)	—	—	1.02 ms	—	—	—	—	—	—
	BiCGSTAB (MG) (no corrector)	—	—	1.46 ms	—	—	—	—	—	—
	FFT direct solve (exact)	—	—	235 μ s	—	—	—	—	—	—
	—	—	—	—	—	—	—	—	—	—
AnisoDiff	Jacobi	1.04 ms	727 μ s $_{\tau=10, t_0=0}$	764 μ s (1.37 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	Jacobi (0.67)	970 μ s	676 μ s $_{\tau=15, t_0=0}$	700 μ s (1.38 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	GS	1.54 ms	672 μ s $_{\tau=10, t_0=0}$	710 μ s (2.18 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	SymGS	1.98 ms	677 μ s $_{\tau=50, t_0=0}$	714 μ s (2.72 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	SOR (1.5)	1.53 ms	663 μ s $_{\tau=15, t_0=0}$	690 μ s (2.20 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	Line GS	2.00 ms	656 μ s $_{\tau=10, t_0=0}$	686 μ s (2.81 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	Multigrid alone (no corrector)	—	—	8.19 ms	—	—	—	—	—	—
	PCG (MG) (no corrector)	—	—	2.08 ms	—	—	—	—	—	—
	FFT direct solve (exact)	—	—	245 μ s	—	—	—	—	—	—
	—	—	—	—	—	—	—	—	—	—
VarCoeff	Jacobi	1.24 ms	730 μ s $_{\tau=5, t_0=0}$	784 μ s (1.65 $\times$ / 0.99 $\times$)	—	—	—	—	—	—
	Jacobi (0.67)	1.20 ms	705 μ s $_{\tau=50, t_0=0}$	752 μ s (1.59 $\times$ / 0.95 $\times$)	—	—	—	—	—	—
	GS	2.58 ms	742 μ s $_{\tau=50, t_0=0}$	792 μ s (3.19 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	SymGS	3.59 ms	780 μ s $_{\text{one-shot}}$	831 μ s (4.32 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	SOR (1.5)	4.90 ms	798 μ s $_{\tau=15, t_0=0}$	848 μ s (5.11 $\times$ / 0.96 $\times$)	—	—	—	—	—	—
	Multigrid	2.53 ms	989 μ s $_{\tau=10, t_0=0}$	1.06 ms (2.42 $\times$ / 0.97 $\times$)	—	—	—	—	—	—
	Multigrid alone (no corrector)	—	—	2.66 ms	—	—	—	—	—	—
	PCG (MG) (no corrector)	—	—	2.01 ms	—	—	—	—	—	—
	Sparse LU direct solve	—	—	932 μ s	—	—	—	—	—	—
	—	—	—	—	—	—	—	—	—	—

- 567 Andrew Staniforth and Jean Côté. Semi-lagrangian integration schemes for atmospheric models—a
568 review. *Monthly weather review*, 119(9):2206–2223, 1991.
- 569 John David Anderson. *Hypersonic and high temperature gas dynamics*. Aiaa, 1989.
- 570 Fredi Tröltzsch. *Optimal control of partial differential equations: theory, methods, and applications*,
571 volume 112. American Mathematical Soc., 2010.
- 572 Sylvain Baillet, John C Mosher, and Richard M Leahy. Electromagnetic brain mapping. *IEEE Signal*
573 *processing magazine*, 18(6):14–30, 2001.
- 574 Roberta Grech, Tracey Cassar, Joseph Muscat, Kenneth P Camilleri, Simon G Fabri, Michalis
575 Zervakis, Petros Xanthopoulos, Vangelis Sakkalis, and Bart Vanrumste. Review on solving the
576 inverse problem in eeg source analysis. *Journal of neuroengineering and rehabilitation*, 5:1–33,
577 2008.
- 578 Gordon D Smith. *Numerical solution of partial differential equations: finite difference methods*.
579 Oxford university press, 1985.
- 580 Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-*
581 *state and time-dependent problems*. SIAM, 2007.
- 582 Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*.
583 Courier Corporation, 2003.

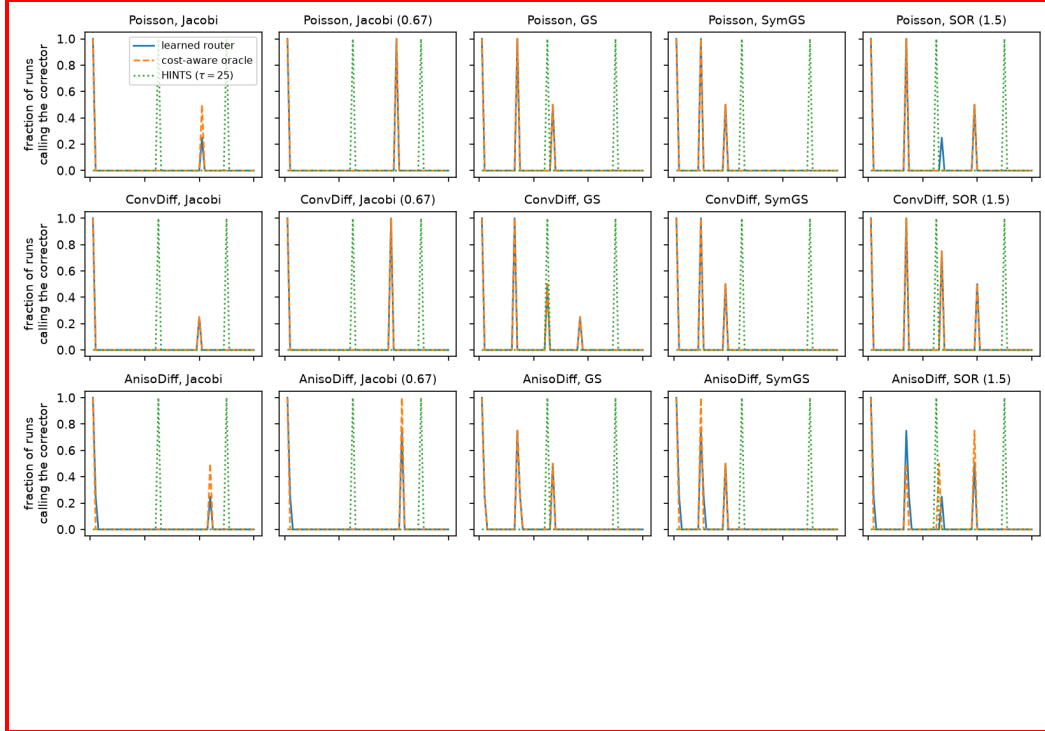


Figure 1: Corrector usage on the 128×128 grid: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=15$), for every solver pairing (columns) and all four equations (rows); the same figure for the 256^2 and 512^2 grids is in Section E.

- 584 Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.
- 585 Susanne C Brenner. *The mathematical theory of fnite element methods*. Springer, 2008.
- 586 Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- 587 Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew
588 Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with
589 applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- 590 Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks
591 with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on
592 neural networks*, 6(4):911–917, 1995.
- 593 Lizuo Liu and Wei Cai. Multiscale deeponet for nonlinear operators in oscillatory function spaces for
594 building seismic wave responses. *arXiv preprint arXiv:2111.04860*, 2021.
- 595 Xinliang Liu, Bo Xu, Shuhao Cao, and Lei Zhang. Mitigating spectral bias for the multiscale operator
596 learning. *Journal of Computational Physics*, 506:112944, 2024.
- 597 Zhilin You, Zhenli Xu, and Wei Cai. Mscalefno: Multi-scale fourier neural operator learning for
598 oscillatory function spaces. *arXiv preprint arXiv:2412.20183*, 2024.
- 599 Siavash Khodakarami, Vivek Oommen, Aniruddha Bora, and George Em Karniadakis. Mitigating
600 spectral bias in neural operators via high-frequency scaling for physical systems. *arXiv preprint
601 arXiv:2503.13695*, 2025.
- 602 Zhi-Qin John Xu, Lulu Zhang, and Wei Cai. On understanding and overcoming spectral biases of
603 deep neural network learning methods for solving pdes. *arXiv preprint arXiv:2501.09987*, 2025.

604 Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua
605 Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference*
606 *on machine learning*, pages 5301–5310. PMLR, 2019.

607 Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle:
608 Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.

609 Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for
610 general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019.

611 Enrui Zhang, Adar Kahana, Alena Kopaničáková, Eli Turkel, Rishikesh Ranade, Jay Pathak, and
612 George Em Karniadakis. Blending neural operators and relaxation methods in pde numerical
613 solvers. *Nature Machine Intelligence*, pages 1–11, 2024.

614 John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.

615 Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods:*
616 *Fundamentals in Single Domains*. Springer, 2006. ISBN 9783540307264.

617 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew
618 Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations.
619 *arXiv preprint arXiv:2010.08895*, 2020a.

620 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik
621 Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial
622 differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020b.

623 Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning
624 nonlinear operators via deeponet based on the universal approximation theorem of operators.
625 *Nature machine intelligence*, 3(3):218–229, 2021.

626 George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for
627 maximizing submodular set functions—i. *Mathematical programming*, 14(1):265–294, 1978.

628 Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions.
629 In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*
630 *(APPROX/RANDOM 2017)*, pages 19–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

631 Louenas Bounia and Frederic Koriche. Approximating probabilistic explanations via supermodular
632 minimization. In *Uncertainty in Artificial Intelligence*, pages 216–225. PMLR, 2023.

633 Abhimanyu Das and David Kempe. Approximate submodularity and its applications: Subset selection,
634 sparse approximation and dictionary selection. *Journal of Machine Learning Research*, 19(3):
635 1–34, 2018.

636 Uriel Feige, Vahab S Mirrokni, and Jan Vondrák. Maximizing non-monotone submodular functions.
637 *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

638 Andrew An Bian, Joachim M Buhmann, Andreas Krause, and Sebastian Tschieschek. Guarantees for
639 greedy maximization of non-submodular functions with applications. In *International conference*
640 *on machine learning*, pages 498–507. PMLR, 2017.

641 Chris Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. Submodular maximization beyond
642 non-negativity: Guarantees, fast algorithms, and applications. In *International Conference on*
643 *Machine Learning*, pages 2634–2643. PMLR, 2019.

644 Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions.
645 *Advances in Neural Information Processing Systems*, 21, 2008.

646 Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. Maximizing sequence-submodular functions
647 and its application to online advertising. *Management Science*, 67(10):6030–6054, 2021.

648 Zhenliang Zhang, Edwin KP Chong, Ali Pezeshki, and William Moran. String submodular functions
649 with curvature constraints. *IEEE Transactions on Automatic Control*, 61(3):601–616, 2015.

650 Sara Bernardini, Fabio Fagnani, and Chiara Piacentini. Through the lens of sequence submodularity.
651 In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30,
652 pages 38–47, 2020.

653 Brandon Van Over, Bowen Li, Edwin KP Chong, and Ali Pezeshki. A performance bound for
654 the greedy algorithm in a generalized class of string optimization problems. *arXiv preprint*
655 *arXiv:2409.05020*, 2024.

656 Sebastian Tschiatschek, Adish Singla, and Andreas Krause. Selecting sequences of items via
657 submodular maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
658 volume 31, 2017.

659 Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J Cree. Regularisation of neural networks
660 by enforcing lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021.

661 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
662 neural networks. *Neural computation*, 1(2):270–280, 1989.

663 Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C
664 Courville, and Yoshua Bengio. Professor forcing: A new algorithm for training recurrent networks.
665 *Advances in neural information processing systems*, 29, 2016.

666 Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence
667 prediction with recurrent neural networks. *Advances in neural information processing systems*, 28,
668 2015.

669 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and
670 applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.

671 Ali Taghibakhshi, Scott MacLachlan, Luke Olson, and Matthew West. Optimization-based algebraic
672 multigrid coarsening using reinforcement learning. *Advances in neural information processing*
673 *systems*, 34:12129–12140, 2021.

674 Matteo Caldana, Paola F Antonietti, et al. A deep learning algorithm to accelerate algebraic multigrid
675 methods in finite element solvers of 3d elliptic pdes. *Computers & Mathematics with Applications*,
676 167:217–231, 2024.

677 Alena Kopaničáková and George Em Karniadakis. Deeponet based preconditioning strategies for
678 solving parametric linear systems of equations. *SIAM Journal on Scientific Computing*, 47(1):
679 C151–C181, 2025.

680 Ru Huang, Ruipeng Li, and Yuanzhe Xi. Learning optimal multigrid smoothers via neural networks.
681 *SIAM Journal on Scientific Computing*, 45(3):S199–S225, 2022.

682 Alexandr Katrutsa, Talgat Daulbaev, and Ivan Oseledets. Black-box learning of multigrid parameters.
683 *Journal of Computational and Applied Mathematics*, 368:112524, 2020.

684 Adar Kahana, Enrui Zhang, Somdatta Goswami, George Karniadakis, Rishikesh Ranade, and Jay
685 Pathak. On the geometry transferability of the hybrid iterative numerical solver for differential
686 equations. *Computational Mechanics*, 72(3):471–484, 2023.

687 Pengzhan Jin, Shuai Meng, and Lu Lu. Mionet: Learning multiple-input operators via tensor product.
688 *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.

689 Jun Hu and Pengzhan Jin. A hybrid iterative method based on mionet for pdes: Theory and numerical
690 examples. *Mathematics of Computation*, 2025.

691 Chen Cui, Kai Jiang, Yun Liu, and Shi Shu. Fourier neural solver for large sparse linear algebraic
692 systems. *Mathematics*, 10(21):4014, 2022.

693 Tal Shnitzer, Anthony Ou, Mírian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson,
694 and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint*
695 *arXiv:2309.15789*, 2023.

696 Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt
697 Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing
698 system. *arXiv preprint arXiv:2403.12031*, 2024.

699 Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS
700 Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query
701 routing. *arXiv preprint arXiv:2404.14618*, 2024.

702 Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and
703 Liang Lin. Routereval: A comprehensive benchmark for routing llms to explore model-level
704 scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.

705 Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki,
706 Xia Hu, and Helen Zhou. Learning to route llms with confidence tokens. *arXiv preprint
707 arXiv:2410.13284*, 2024.

708 Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui,
709 Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based
710 on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*,
711 2025.

712 Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to
713 large language models. *arXiv preprint arXiv:2308.11601*, 2023.

714 Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to
715 large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.

716 Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language
717 model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on
718 Web Search and Data Mining*, pages 606–615, 2024.

719 Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of
720 local experts. *Neural computation*, 3(1):79–87, 1991.

721 Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural
722 computation*, 6(2):181–214, 1994.

723 Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and
724 Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv
725 preprint arXiv:1701.06538*, 2017.

726 Dmitry Lepikhin, Hyounjoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang,
727 Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional
728 computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

729 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter
730 models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39,
731 2022.

732 Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris
733 Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al.
734 Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

735 Michael C Mozer. A focused backpropagation algorithm for temporal pattern recognition. In
736 *Backpropagation*, pages 137–169. Psychology Press, 2013.

737 Anthony J Robinson and Frank Fallside. *The utility driven dynamic error propagation network*,
738 volume 11. University of Cambridge Department of Engineering Cambridge, 1987.

739 Paul J Werbos. Generalization of backpropagation with application to a recurrent gas market model.
740 *Neural networks*, 1(4):339–356, 1988.

741 Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and
 742 George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical
 743 extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*,
 744 393:114778, 2022.

745 Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured
 746 prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference
 747 Proceedings, 2011.
 748

749 A Proofs for Section 4

750 A.1 Proof of Proposition A.1

751 **Proposition A.1.** Any prefix monotonically non-increasing sequence supermodular function g is
 752 weakly supermodular with respect to all sequences $S \in \Omega^*$ with $\alpha(S) = 1$

753 *Proof.* This proof is adapted from Liberty and Sviridenko [2017].

754 If g is sequence supermodular then,

$$\begin{aligned} & g(S) - g(S \oplus S') \\ &= \sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) - g(S \oplus (S'_1, \dots, S'_i)) \\ &\stackrel{(a)}{\leq} \sum_{i=1}^{|S'|} g(S) - g(S \oplus S'_i) \\ &\leq |S'| \max_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \end{aligned}$$

755 (a) by supermodularity

756

□

757 A.2 Proof of Theorem 4.1

758 **Theorem 4.1.** Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal
 759 solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity ratio of $\alpha(O)$ and μ -postfix monotonicity
 760 with respect to O along the greedy solution S^T of length T . If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then

$$g(S^T) \leq \mu(1 - \phi_T(\alpha(O)))g(O) + \phi_T(\alpha(O))g(\emptyset)$$

761 *Proof.* This proof strategy is inspired by Streeter and Golovin [2008] and Liberty and Sviridenko
 762 [2017]

$$\begin{aligned} g(S^t) - \mu g(O) &\stackrel{(a)}{\leq} g(S^t) - g(S^t \oplus O) \\ &= \sum_{i=1}^{|O|} g(S^t \oplus (o_1, \dots, o_{i-1})) - g(S^t \oplus (o_1, \dots, o_i)) \\ &\stackrel{(b)}{\leq} \alpha(O) \sum_{i=1}^{|O|} g(S^t) - g(S^t \oplus o_i) \\ &\leq \alpha(O)|O| \max_{i \in [|O|]} g(S^t) - g(S^t \oplus o_i) \\ &\leq \alpha(O)Tg(S^t) - \alpha(O)T \min_{\omega \in \Omega} g(S^t \oplus \omega) \\ &= \alpha(O)Tg(S^t) - \alpha(O)Tg(S^{t+1}) \end{aligned}$$

763 (a) by μ -postfix monotonicity along S^T , i.e., $g(S^t \oplus O) \leq \mu g(O)$; (b) by weak supermodularity with
 764 respect to O (applied to the prefix S^t).

765 After rearranging the inequality, we get:

$$\begin{aligned} g(S^{t+1}) &\leq \frac{1}{\alpha(O)T} (\mu g(O) + (\alpha(O)T - 1) g(S^t)) \\ &= \frac{\mu}{\alpha(O)T} g(O) + \left(1 - \frac{1}{\alpha(O)T}\right) g(S^t) \end{aligned}$$

766 When recursively applying this inequality, we get:

$$\begin{aligned} g(S^T) &\leq \frac{\mu}{\alpha(O)T} g(O) \sum_{i=0}^{T-1} \left(1 - \frac{1}{\alpha(O)T}\right)^i + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \\ &= \mu \left(1 - \left(1 - \frac{1}{\alpha(O)T}\right)^T\right) g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \end{aligned}$$

767

□

768 A.3 Proof of Proposition 4.2

769 **Proposition 4.2.** Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is
 770 ρ_j -Lipschitz continuous with $\rho_j \leq 1$, that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$, and that $\sum_{i=1}^T (1 - \rho_{O_i}^2) > 0$,
 771 i.e., at least one action of the optimal sequence O is a strict contraction. Then, the function h is
 772 weakly supermodular with respect to the optimal solution O , with

$$\alpha(O) = \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

773 Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by a
 774 unitary matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also postfix
 775 non-increasing; invertibility alone does not suffice (Section A.3).

776 *Proof.* For brevity, we use the notation $(g_1 \circ \dots \circ g_T)(x) = \circ_{t=1}^T g_t(x)$, where composition is
 777 applied from right to left so that g_T acts first. In this proof, we use a few properties of Lipschitz
 778 continuous functions:

- 779 • **Property 1:** If g is ρ -Lipschitz continuous and $g(0) = 0$, then $\|g(x)\|_2 = \|g(x) - 0\|_2 =$
 780 $\|g(x) - g(0)\|_2 \leq \rho \|x - 0\|_2 = \rho \|x\|_2$
- 781 • **Property 2:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
 782 and ρ_2 respectively, the Lipschitz constant of $g_1 + g_2$ and $g_1 - g_2$ is at most $\rho_1 + \rho_2$.
- 783 • **Property 3:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
 784 and ρ_2 respectively, the Lipschitz constant of $g_1 \circ g_2$ is at most $\rho_1 \rho_2$.

785 In order to prove weakly- α -supermodularity, we must first prove prefix monotonicity.

786 **Prefix monotonicity:** Let $S \preceq S'$ where $S' = S \oplus N$.

$$\begin{aligned} h(S') &= \left\| \circ_{t=|S'|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &= \left\| \circ_{t=|S'|+1}^{|S'|+1} (I - C_{S'_t} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\stackrel{(a)}{\leq} \left(\prod_{t=|S|+1}^{|S'|} \rho_{S'_t}^2 \right) \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\leq h(S) \end{aligned}$$

787 (a) by Property 1

788 **Weak supermodularity:** We will upper bound $\alpha(O)$ by providing an upper bound for $h(S) - h(S \oplus$
 789 $O)$ and a lower bound for $\sum_{i=1}^T h(S) - h(S \oplus O_i)$. Since $h \geq 0$, $h(S) - h(S \oplus O) \leq h(S) =$
 790 $\left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2$. (An earlier version of this step used $\|x\|^2 - \|Gx\|^2 \leq \|x - Gx\|^2$,
 791 which is false, e.g. for $G = \frac{1}{2}I$; the direct bound is sharper by a factor 4.)

792 To lower bound $\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i)$

$$\begin{aligned} \sum_{i=1}^{|O|} h(S) - h(S \oplus O_i) &= \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \left\| (I - C_{O_i} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\geq \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \rho_{O_i}^2 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &= \left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right) \end{aligned}$$

793 If $h(S) = 0$ then, by prefix monotonicity, every term above vanishes and the weak-supermodularity
 794 inequality holds for any $\alpha \geq 1$; assume $h(S) > 0$. The lower bound just obtained is positive because
 795 $T - \sum_i \rho_{O_i}^2 > 0$ by hypothesis (this is the only place where a strict contraction is needed; $\rho_j \leq 1$
 796 suffices everywhere else). Finally,

$$\begin{aligned} \frac{h(S) - h(S \oplus O)}{\sum_{i=1}^T (h(S) - h(S \oplus O_i))} &\leq \max \left\{ \frac{\left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2}{\left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right)}, 1 \right\} \\ &= \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\} \end{aligned}$$

797 **Postfix monotonicity.** Let $S' = S \oplus N$ and suppose that $I - C_j \mathcal{L}_h^a = P \Lambda_j P^*$ for all j with a
 798 common unitary P and $|\lambda_{ji}| \leq 1$ (the products below are of $|\lambda_{ji}|^2$ and $|z_i|^2$; we write them without
 799 moduli for the real case). By Theorem A.2,

$$h(S') = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \leq \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} = h(N),$$

800 since every factor $\lambda_{ji}^{2m_j(S)}$ lies in $[0, 1]$. The shared eigenbasis cannot be replaced by invertibility of
 801 the individual maps: for

$$A = \begin{bmatrix} 0.0674240 & -0.1864331 \\ -0.1473150 & -0.8707252 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7575738 & 0.4816284 \\ -0.1369844 & 0.3257283 \end{bmatrix}, \quad x = \begin{bmatrix} 0.2812107 \\ -0.5538228 \end{bmatrix},$$

802 both matrices are invertible with spectral norm 0.9, yet $\|BAx\|_2 = 0.330 > 0.225 = \|Bx\|_2$, i.e.,
 803 $h((A) \oplus (B)) > h((B))$. (An earlier version of this proof bounded $\|G^{-1}\|$ by the reciprocal of the
 804 Lipschitz constant of G , which is not valid.)

805

□

806 A.4 Proof of Theorem A.2

807 **Lemma A.2.** Let $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ where P is a unitary matrix and $\Lambda_j =$
 808 $\text{diag}(\lambda_{j1}, \dots, \lambda_{jN})$. If $P^{-1} e_h^{(0)} = z$, then the following equality holds (written for a real or-
 809 thogonal P and real eigenvalues; in general z_i^2 and λ_{ji}^{2m} are to be read as $|z_i|^2$ and $|\lambda_{ji}|^{2m}$ here and

810 in the proofs of Propositions 4.2 and 4.3, which use only $\|Px\| = \|x\|$ and $|\lambda_{ji}| \leq 1$):

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \quad (11)$$

811 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$

Proof.

$$\begin{aligned} h(S) &= \left\| \prod_{t=|S|}^1 (I_N - C_{S_t} \mathcal{L}_h^a) e_h^{(0)} \right\|^2 \\ &= \left\| \prod_{t=|S|}^1 (P \Lambda_{S_t} P^{-1}) e_h^{(0)} \right\|^2 \\ &= \left\| P \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\ &= \left\| \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\ &= \sum_{i=1}^N z_i^2 \prod_{t=|S|}^1 \lambda_{S_t i}^2 \\ &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \end{aligned}$$

812

□

813 A.5 Proof of Theorem 4.3

814 **Proposition 4.3.** Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common
815 unitary matrix P . Then, h is supermodular.

816 *Proof.* Let $S \preceq S'$ where $S' = S \oplus B$. By Theorem A.2,

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}$$

817 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$ be the number of times a sequence S calls the solver j . Recall that h is
818 considered sequence supermodular if $\forall S', S \in \Omega^*$ such that $S \preceq S'$, it holds that

$$h(S) - h(S \oplus \omega) \geq h(S') - h(S' \oplus \omega)$$

819

$$\begin{aligned} h(S) - h(S \oplus \omega) &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} - \sum_{i=1}^N z_i^2 \lambda_{\omega i}^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\ &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \end{aligned}$$

820 Similarly,

$$\begin{aligned}
h(S') - h(S' \oplus \omega) &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S')} \\
&\stackrel{(a)}{=} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2(m_j(S) + m_j(B))} \\
&\stackrel{(b)}{\leq} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\
&= h(S) - h(S \oplus \omega)
\end{aligned}$$

821 (a) Since $m_j(S') = \sum_{t=1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S'_t=j} + \sum_{t=|S|+1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j} +$
822 $\sum_{t=1}^{|B|} \mathbf{1}_{B_t=j} = m_j(S) + m_j(B)$, (b) since $|\lambda_{ji}| \leq 1$ (as $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ and P is unitary)
823 and $m_j(B) \geq 0$ for all $j \in [K]$

824 □

825 B Proofs for Section 5

826 B.1 Proof of Theorem B.1

827 **Lemma B.1.** *For any set of preconditioning functions $\mathcal{C}$, any discrete operator $\mathcal{L}_h^a$, any router r , any*
828 *$a_h, f_h, u_h^{(t)}, u_h \in \mathcal{A} \times \mathcal{F} \times \mathcal{U} \times \mathcal{U}$, the following equality holds true:*

$$\begin{aligned}
l_{route} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

829 *Proof.* Note that $\sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} = K - 1$

$$\begin{aligned}
l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) = j} \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad + (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad + \sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\quad - (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

830

$$\begin{aligned}
&= \sum_{j=1}^K \left(\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \right) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

831

□

832 **B.2 Proof of Theorem B.2**

833 **Proposition B.2.** *Let $K \geq 2$. For any router r defined by $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$,*

834 *any $a_h \in \mathcal{A}$, $f_h \in \mathcal{F}$, and $u_h^{(t)}, u_h \in \mathcal{U}$, the routing loss l_{route} satisfies:*

$$\log(2) l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) \leq \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)$$

835 *Proof.* By Theorem B.1, we know that

$$\begin{aligned}
\log(2)l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, u_h\right) &= \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - \log(2) (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\leq \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\stackrel{(a)}{\leq} - \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \\
&= \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)
\end{aligned}$$

836 (a) if $r(a_h, f_h, u_h^{(t)}) \neq j$, $\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \leq 0.5$ (with equality only at a tie for the largest
837 score), which implies that $-\log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \geq \log(2) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}$

838 $\square$

839 B.3 Proof of Theorem 5.1

840 **Theorem 5.1.** *Let $K \geq 2$ and let the per-step costs be normalised state by state, $\hat{c}_j = \tilde{c}_j / \sum_{k=1}^K \tilde{c}_k$,
841 at every state at which $\sum_k \tilde{c}_k > 0$, i.e., at which at least one solver does not annihilate the error
842 (states at which every solver annihilates it carry no routing loss and are discarded), and let $\hat{l}_{\text{route}}$
843 and $\hat{\Psi}$ denote the routing loss and the surrogate with $\tilde{c}_j$ replaced by $\hat{c}_j$; the normalisation does
844 not change the Bayes-optimal decision equation 9 at any state. Then, for any collection of solvers
845 $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, $\hat{\Psi}$ is a Bayes consistent surrogate for $\hat{l}_{\text{route}}$.*

846 **Proof.** Fix a state $x = (a_h, f_h, u_h^{(t)})$ at which $\sum_k \tilde{c}_k > 0$ and write $\hat{c}_j = \tilde{c}_j / \sum_k \tilde{c}_k$, so that
847 $\hat{c}_j \in [0, 1]$ and $\sum_j \hat{c}_j = 1$; let $K \geq 2$. Set $\hat{B}_j = \sum_{k \neq j} \hat{c}_k = 1 - \hat{c}_j$ and $q_j = \hat{B}_j / (K-1)$; then q is
848 a probability vector on $[K]$, and $\arg \max_j q_j = \arg \min_j \hat{c}_j$ is the Bayes decision equation 9. Let
849 $p = \text{softmax}(\mathbf{g}(x))$.

850 **Surrogate excess.** By the definition equation 10 of the surrogate with the weights $\hat{c}$, $\hat{\Psi}(\mathbf{g}, x) =$
851 $-\sum_j \hat{B}_j \log p_j = (K-1) \text{CE}(q, p) = (K-1) [H(q) + \text{KL}(q \| p)]$, with H the entropy and KL
852 the Kullback–Leibler divergence. Since the scores are unrestricted measurable functions, $\hat{\Psi}^*(x) =$
853 $\inf_{\mathbf{g}} \hat{\Psi}(\mathbf{g}, x) = (K-1) H(q)$ (attained at $p = q$ when $q > 0$, approached in the limit otherwise), so
854 the conditional surrogate excess is $\hat{\Psi}(\mathbf{g}, x) - \hat{\Psi}^*(x) = (K-1) \text{KL}(q \| p)$.

855 **Routing excess.** With $r(x) = \arg \max_j g_j(x) = \arg \max_j p_j$ and $j^* \in \arg \max_j q_j$, $\hat{l}_{\text{route}}(r, x) =$
856 $\hat{c}_{r(x)} = 1 - (K-1) q_{r(x)}$ and $\hat{l}_{\text{route}}^*(x) = 1 - (K-1) q_{j^*}$, so the conditional routing excess is
857 $(K-1) (q_{j^*} - q_{r(x)})$. Since $p_{r(x)} \geq p_{j^*}$,

$$q_{j^*} - q_{r(x)} \leq (q_{j^*} - p_{j^*}) + (p_{r(x)} - q_{r(x)}) \leq \|q - p\|_1 \leq \sqrt{2 \text{KL}(q \| p)}$$

858 by Pinsker's inequality, hence

$$\hat{l}_{\text{route}}(r, x) - \hat{l}_{\text{route}}^*(x) \leq (K-1) \sqrt{2 \text{KL}(q \| p)} = \sqrt{2(K-1) [\hat{\Psi}(\mathbf{g}, x) - \hat{\Psi}^*(x)]}.$$

859 **Risks.** Taking expectations over the state distribution and using Jensen's inequality for the concave
860 square root,

$$\mathcal{R}_{\hat{l}_{\text{route}}}(r_n) - \mathcal{R}_{\hat{l}_{\text{route}}^*}^* \leq \sqrt{2(K-1) (\mathcal{R}_{\hat{\Psi}}(\mathbf{g}_n) - \mathcal{R}_{\hat{\Psi}}^*)},$$

where $\mathcal{R}_{\hat{l}_{\text{route}}}^* = \mathbb{E}[\hat{l}_{\text{route}}^*]$ and $\mathcal{R}_{\hat{\Psi}}^* = \mathbb{E}[\hat{\Psi}^*]$ because the infima are over all measurable functions. Hence $\mathcal{R}_{\hat{\Psi}}(\mathbf{g}_n) - \mathcal{R}_{\hat{\Psi}}^* \rightarrow 0$ implies $\mathcal{R}_{\hat{l}_{\text{route}}}(\mathbf{r}_n) - \mathcal{R}_{\hat{l}_{\text{route}}}^* \rightarrow 0$, which is Bayes consistency. (An earlier version of this proof bounded the excess through the calibration function Γ of Mao et al. [2023], whose inverse is defined only on $[0, \log 2]$ while the surrogate excess is unbounded, and assumed uniform absolute bounds on the unnormalised costs, which cannot hold along a convergent solve; the argument above needs neither.) $\square$

C Training Details

The periodic experiments of Section 7 use the corrector and the lightweight router described in Section E.1; the data distribution below is shared by all experiments, and the DeepONet and LSTM router described afterwards are those of the original submission’s pipeline, whose Dirichlet experiment has been withdrawn (Section D); they are kept for the record.

Data for both DeepONet and the routers is sampled from a zero-mean Hierarchical Gaussian Random Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ are sampled as:

$$\begin{aligned}\alpha &\sim \text{Log-Uniform}(0.01, 100) \\ \beta &\sim \text{Log-Uniform}(0.1, 1000) \\ \gamma &\sim \text{Uniform}(\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\})\end{aligned}$$

Given (α, β, γ) , samples are generated in the Fourier space. For each non-zero frequency mode k , we draw an independent complex coefficient from a Gaussian distribution with mean 0 and variance $\alpha(4\pi^2\|k\|_2^2 + \beta)^{-\gamma}$. We enforce a Hermitian symmetry to obtain a real-valued field, set the zero-frequency (DC) mode to 0 to obtain a zero mean field, and apply the inverse Discrete Fourier Transform to obtain the field in physical space.

For each sample, we compute reference solutions with a least squares solver and treat them as ground truth.

This data is used to trained our DeepONet models and LSTM routers. All the models were implemented using PyTorch and all the models were trained on one Nvidia A40 GPU.

Table 2 contains all hyperparameter details for the DeepONet. DeepONet took 30 minutes to train. We then use the model with the best validation loss.

Table 2: Hyperparameter settings for DeepONet

Hyperparameter	Value
Learning rate	1e-3
Branch Dimension	128
Hidden dimension for branch net	256
No. of hidden layers in branch net	4
Hidden dimension for trunk net	256
No. of hidden layers in trunk net	4
Gradient Clipping Norm	1.0
Weight Decay	0.005
Batch size	256
Training samples	10000
Validation samples	2000
Epochs	1000

The routers are LSTM models. Along with the inputs specified in Section 5, namely $a_h, f_h, u_h^{(t)}$, we also supply the routers with the iteration index t , current residual $f_h - \mathcal{L}_h^a u_h^{(t)}$, and the routing decisions from the previous iteration. While the iteration index and residual are deterministic functions of $(a_h, f_h, u_h^{(t)})$ and align with the theory presented in Section 5, incorporating previous routing decisions is a mild deviation from the idealized setting. Nevertheless, this modification is motivated by the same considerations that justify the use of scheduled sampling: under deployment,

routing decisions are predicted autoregressively and influence future inputs, whereas training under teacher forcing assumes independence from past predictions. Including previous routing decisions bridges the gap induced by exposure bias by providing the model with trajectory-level information.

All the routers are trained with scheduled sampling. We use a warm-up of e_w epochs with teacher-forcing probability $p_{tf}(e) = ss_{start}$. After the warm-up, the p_{tf} decays geometrically by a factor of $\gamma_{tf} < 1$ per epoch and is floored by ss_{end} :

$$p_{tf}(e) = \begin{cases} ss_{start} & e \leq e_w \\ \max(ss_{start} \gamma_{tf}^{e-e_w}, ss_{end}) & e > e_w \end{cases}$$

At each time step, with probability $p_{tf}(e)$, we feed the teacher-forced greedy iterate; otherwise, we feed the router’s own predicted iterate.

Since LSTMs on long rollouts can suffer from exploding/vanishing gradients, we use truncated backpropagation through time (TBPTT) [Mozier, 2013, Robinson and Fallside, 1987, Werbos, 1988]: the forward pass unrolls the entire trajectory, but gradients are propagated only through the most recent $w_{bptt}(e)$ steps at epoch e . Hidden states are passed forward between segments, while earlier segments are treated as stop-gradient.

We employ a curriculum learning approach analogous to scheduled sampling. Let T_{max} be the horizon (300 iterations). With a warm-up of e_w epochs,

$$w_{bptt}(e) = \begin{cases} w_{start} & e \leq e_w \\ \min\left(T_{max}, w_{start} \gamma_{bptt}^{\lfloor \frac{e-e_w}{f_{bptt}} \rfloor}\right) & e > e_w \end{cases} \quad (12)$$

so the window grows geometrically by a factor of $\gamma_{bptt} > 1$ every f_{bptt} epochs and is capped at the full trajectory length.

Table 3 contains all hyperparameter details for the LSTM routers. The routers for the Paired Solver experiment took a maximum of 4 hours and 30 minutes to train while the routers for the Solver Ensemble experiment took a maximum of 6 hours to train. We then use the model with the best validation loss for testing. The architecture and dataset size is larger for the Solver ensemble experiments to encourage the model to learn some of the nuanced differences between the classes.

Table 3: Hyperparameter settings for routers in Paired solver and Solver Ensemble Experiments

Hyperparameter	Paired Solver	Solver Ensemble
Learning rate	1e-3	1e-3
Hidden dimension	256	512
No. of hidden layers	4	3
Gradient Clipping Norm	1.0	1.0
Weight Decay	0.005	0.005
Batch size	32	32
Training samples	256	1024
Validation samples	32	128
Epochs	200	200
ss_{start}	1.0	1.0
γ_{tf}	0.95	0.95
ss_{end}	0.0	0.0
w_{start}	50	50
γ_{bptt}	1.25	1.25
e_w	10	10
f_{bptt}	1	1

D Additional Experimental Results

The original submission’s Dirichlet experiment (31×31 grid, LSTM router, MLP-branch DeepONet), which this appendix previously kept, has been withdrawn. A review of its evaluation code found

that HINTS’ predictions had been mean-centred before comparison with the (uncentred) Dirichlet reference solution while the other methods’ predictions had not, which penalises HINTS by an amount we cannot reconstruct without the original per-instance predictions; in addition, its symmetric-GS rows used the mis-scaled SSOR reference noted in Section E.1. The evaluation code has been corrected (`results.py`), but the experiment has not been rerun, so the wall-clock study of Section E, which is restricted to periodic boundary conditions, is the paper’s only experimental evidence. Extending the corrector to Dirichlet boundaries requires a sine-transform trunk in place of the Fourier trunk and is left to future work.

E Cost-Aware Routing and Wall-Clock Evaluation

This appendix collects the complete wall-clock study that Section 7 summarises: the *wall-clock time to a target accuracy* (charged replay time, Section E.1) of the learned router against HINTS, the classical solver alone and the classical methods a practitioner would reach for on these problems (geometric multigrid and preconditioned Krylov methods), for all five classical solvers and three PDEs, on 128^2 , 256^2 and 512^2 grids, using the cost-aware instantiation of the greedy rule introduced at the end of Section 4. Besides the Poisson and convection–diffusion equations of Section 7 we add *variable-coefficient diffusion* (Section E.5), on which no Fourier diagonalisation exists and the corrector must be learned from data, and *anisotropic diffusion*, $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$ with $\epsilon = 10^{-2}$ (diffusion tensor $\text{diag}(\epsilon, 1)$), on the same periodic domain with the same forcing distribution. It is the standard model problem on which point-relaxation methods fail to smooth: an error component oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep, and geometric multigrid with point smoothing loses its grid-independent convergence (line relaxation or semi-coarsening would be needed). Section E.1 describes the protocol; Sections E.2 to E.5 collect all results per equation (every grid, every tolerance, every baseline, with significance tests and seed trials); Section E.7 studies ensembles of one, two and three classical solvers; Section E.8 accounts for overheads and training cost; and Section E.10 verifies numerically that the assumptions of Theorem 4.1, Propositions 4.2–4.3 and Theorem 5.1 hold in these settings.

E.1 Protocol

Fast, matched implementations. A wall-clock comparison is meaningful only if the classical baseline is not handicapped by implementation. All classical operations are therefore implemented as compiled, single-threaded C kernels on the periodic 5-point stencil (`stencil.c`, built with `cc -O3`): the residual together with its norm (the stopping test, in one pass), fused (damped) Jacobi sweeps, in-place lexicographic Gauss–Seidel, SOR and symmetric SOR sweeps (forward and backward), a lexicographic line Gauss–Seidel sweep in which every grid line along the strongly coupled axis is solved exactly by a cyclic Thomas algorithm (the line relaxation that anisotropic problems call for), and the full-weighting and bilinear transfers of geometric multigrid, whose V(2,2)-cycles use Gauss–Seidel smoothing—or line Gauss–Seidel smoothing for the anisotropic equation—with rediscritised coarse operators and a direct solve on the 32×32 coarsest grid (measured on one 128^2 instance, first cycle and asymptotic rate: 0.03 and 0.08 on Poisson with point smoothing, 0.005 and 0.04 on the anisotropic equation with line smoothing, against 0.25 and 0.86 with point smoothing there). At 128^2 a residual evaluation with its norm costs $14 \mu\text{s}$, a Gauss–Seidel sweep $61 \mu\text{s}$, a V-cycle $326 \mu\text{s}$ and one corrector call $672 \mu\text{s}$, all measured in the exact per-iteration form in which the benchmark charges them (Table 7); an earlier version of this study used numpy/scipy sweeps that were $4\text{--}5\times$ slower and correspondingly overstated the router’s margins (Section E.9). The compiled kernels agree with the numpy/scipy implementations they replace to 2×10^{-14} (relative), the line sweep agrees with a sparse block-triangular solve to 10^{-13} , and their operators and one-step iterates match the dense reference implementations of the original submission to the reference’s own precision (relative error below 10^{-7} , the reference assembling its matrices in single precision; 10^{-12} where the reference is exact); the comparison is an assertion in `validate_fast_pde.py`. In the course of this check we found that the original dense symmetric-GS reference mis-implemented the SSOR scaling (the factor $\omega/(2 - \omega)$ was added to D^{-1} instead of multiplying it); it has been corrected, and the wall-clock study uses the standard SSOR throughout. Reference solutions are computed by FFT diagonalization of the same discrete operator, i.e., they are exact solutions of the discrete system, so true errors are available for evaluation at no charged cost; the FFT solve is also reported as a baseline in every table (it is a direct method that exists because these test problems have constant coefficients and periodic

boundaries; at 128^2 the router takes $3.08 \times - 3.25 \times$ times as long as it, and the variable-coefficient problem of Section E.5 has no such solve). All timings are single-threaded CPU measurements (Apple M4 Pro, numpy/scipy/PyTorch, one thread). The benchmark guards against machine drift, which on this machine is caused by macOS background services (media analysis, Spotlight indexing and window-server rendering, which slow the memory-bound corrector call by up to $3 \times$ for minutes at a time) and by the scheduler, which moves an idle process to the efficiency cores: before the timed replays of every test instance it re-times a reference operation (one corrector call) and waits for up to 2 min, re-checking every 5 s and running the reference operation in between (a sleeping process stays on the efficiency cores, where the reference remains slow for the whole wait), while that time is more than 15% slower than the session’s reference (the 10th percentile of the reference measurements taken so far, seeded after 10 s of running the reference operation, which is robust to measurement noise and tightens only as genuinely faster states are observed); an instance whose reference is still too slow after the wait, or more than 15% slower than the cell’s final reference (the 10th percentile over all its instances, which catches a session that started in a slow state) or than the median final reference of the other cells of the same equation and grid (which catches a session that was slow throughout), is timed anyway and re-timed at the end of the cell, after waiting up to 5 min more (the Poisson and convection–diffusion 128^2 cells were produced before this in-cell pass existed, and every one of their instances is re-timed with the final driver in a separate pass once the other 128^2 cells are complete). The guard is a per-instance diagnostic; it does not certify that no drift occurred within a replay. Over the 1536 instance timings of the study the guard waited 2490 times, 791 instances were re-timed, 226 remained above the tolerance (largest accepted deviation 14.9%), and the per-instance ratios and load averages are stored with the results (an earlier confirmatory run had been contaminated by a 3.5 h stall of the machine inside one cell, which the harness could not detect). Two accounting checks are reported in Section E.8: the charged time of a router solve is 99–99% of the outer elapsed time of its replay, and its live time to h^2 is 0.92–1.17 times its work-unit time.

Corrector. The neural corrector is a DeepONet applied to the residual $r = f_h - \mathcal{L}_h^a u_h^{(t)}$ and evaluated on a 64×64 sensor grid, i.e., on the coarser grid obtained by band-limited (FFT) restriction to the modes $|k_1|, |k_2| \leq 31$ at every grid size (for the anisotropic equation the sensor grid is $N \times N/4$, i.e., full resolution along x_1 , so that the band contains every mode the point smoothers cannot damp; the sensor grid is the only PDE-specific hyperparameter of the pipeline; the variable-coefficient equation uses the isotropic 64×64 grid); its output is prolonged back to the fine grid by exact trigonometric interpolation, so the correction is band-limited by construction and never injects error into the high-frequency band that the classical smoother owns (Figure 2). The trunk net is the fixed orthonormal Fourier basis of that band ($p = 3969$ basis functions, as in POD-DeepONet [Lu et al., 2022]), and the branch net is a linear layer on the 4096 sensor values; it is fit by least squares on 32,000–64,000 exact (residual, error) pairs of the discrete operator drawn from a mixture of spectral shapes (forcing-like, residual-like, white and randomly tilted spectra), and reaches a validation relative error of $\approx 2 \times 10^{-6}$ on every mode of the band. The corrector is applied scale-equivariantly, $C_{\text{NO}}(r) = \|Rr\| \mathcal{G}_\theta(Rr/\|Rr\|)$, which preserves $C_{\text{NO}}(0) = 0$ (Proposition 4.2). Its deployed form is therefore a linear, Fourier-structured map—a least-squares approximation of the exact band-limited inverse $P(\mathcal{L}_h)_{\text{band}}^{-1} R$ of the discrete operator. Replacing it by that exact inverse changes nothing (development run): on the GS pairing at 128^2 the cost-aware oracle’s median work-unit time to 10^{-8} was 4.472 ms with either corrector and its decision sequence identical on 13 of 16 instances. The gains reported below are thus those of an accurate band-limited spectral correction applied at the right moments; the learned corrector stands in for a spectral inverse that, on these constant-coefficient periodic problems, could also be computed directly. Its cost is dominated by a 4096×4096 matrix–vector product and is therefore almost independent of N ($672 \mu\text{s}$ at 128^2 , [pending] at 512^2 , Table 31), i.e., about 32 Jacobi or 11 GS sweeps at 128^2 and [pending] Jacobi sweeps at 512^2 (Tables 7, 13 and 19). The same corrector is used by HINTS and by all routing policies.

Cost-equalised macro-actions. Per-iteration costs c_j are measured once per equation and grid on the benchmark machine, for all operations of the study in one interleaved session, in the exact form in which the timed replay charges them—the update given the current residual, then the residual of the new iterate and its norm, i.e. the stopping test—as the median over eleven interleaved blocks of the block median of 100 repetitions (the minimum and the spread across blocks are stored with the costs); every pairing and ensemble of that grid therefore shares one corrector cost and one calibration. The unit of cost u is one corrector call; every operation is applied $m_j = \max\{1, \text{round}(u/c_j)\}$ times

per decision, and these macro-actions are compared by their plain error, i.e. the deployed rule is Algorithm 1 on macro-actions; only an operation dearer than the unit ($c_j > u$, hence $m_j = 1$) is compared per unit of cost, $(\|e_j\|/\|e^{(t)}\|)^{u/c_j}$, and the surrogate weights $\tilde{c}_j$ use the same exponent. Which actions are dearer than the unit is read off the cost tables: on the 128^2 grids measured so far (Poisson, convection–diffusion, anisotropic diffusion) none is, so those cells run exactly Algorithm 1 on macro-actions (the rule the exhaustive check of Section E.10 verifies; variable-coefficient diffusion **[pending]**); on the larger grids only a multigrid cycle can be dearer than the unit (**[pending]**), and in the granularity ablation of Section E.6 the corrector itself is. Tables 7, 13 and 19 list c_j , u and m_j for every cell; the equalisation is exact up to rounding ($m_j c_j / u$ lies within $1/(2m_j - 1)$ of 1). Ensembles reuse the cached pairwise costs of their members, so every member has the same macro-action size in the ensemble as in its pairwise run. The cost-aware oracle runs Algorithm 1 on these macro-actions with the true error; the *greedy oracle* rows run the cost-agnostic Algorithm 1 on single operations, as in the main text. (An earlier version of this study applied the exponent $u/(m_j c_j)$ to every action, including the rounding error of cheap ones, which is not Algorithm 1; the deployed rule was changed so that the theory covers it.)

Lightweight router. The learned router selects macro-actions from $6 + K$ scalar features, all $O(1)$ given the residual norm that any stopping test computes: the log relative residual, its per-iteration change over the last macro-action and an exponential moving average thereof, the log epoch index, a one-hot encoding of the previous macro-action, and the log counts of corrector calls so far and of iterations since the last call. It is a two-hidden-layer MLP (64 units) evaluated in numpy, costing $\approx 5 \mu\text{s}$ per decision in a warm loop and $16\text{--}21 \mu\text{s}$ in situ, where every decision after a macro-action finds cold caches (measured inside the timed replays, Section E.8), independently of N . It is trained with the surrogate of Section 5.1, with the per-state weights $\tilde{c}_k$ normalised by $\sum_k \tilde{c}_k$ (which does not change the Bayes decision), on states visited by oracle rollouts with ε -exploration (probability 0.1), followed by two to three DAgger rounds [Ross et al., 2011] in which the current router’s own rollouts are relabelled by the oracle—the batch analogue of the scheduled sampling used in Section C. Training uses 32–128 instances per round (fewer on the larger grids) drawn from the same GRF distribution as the test set but with a different seed; the router of every pairing is trained in seconds to a few minutes. Ensemble routers, which must additionally learn to choose among classical members, use a larger imitation budget (256 rollouts at 128^2 and 128 at 256^2 , six DAgger rounds, hidden width 128, 300 epochs): with the pairwise recipe the router over $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ imitated its oracle imperfectly at tight tolerances (convection–diffusion, 128^2 , $\varepsilon = 10^{-8}$: 3.34 against the oracle’s 2.67 work-unit ms), which the larger budget removes (2.70). At 512^2 the routers are trained from 32 oracle rollouts of at most 400 iterations (stopping error 10^{-8}); for the convection–diffusion Jacobi pairing, where the first such router missed h^2 within the cap on 4 of 16 instances, the reported router uses 64 rollouts of at most 800 iterations—the only per-cell change to the recipe.

Baselines and metric. HINTS applies the corrector every τ -th iteration, first at iteration τ (Section 3). Zhang et al. [2024] use the proportion $1/15$ (one DeepONet call per 14 relaxation sweeps) in all of their two- and three-dimensional experiments and $1/25$ only in the one-dimensional Poisson experiment; since all our problems are two-dimensional, the reference schedule is $\tau = 15$, and $\tau = 25$ is reported as well (an earlier version of this study used it as the reference; the corresponding margins are $0.95 \times \text{--} 7.08 \times$ at h^2 and $1.12 \times \text{--} 4.08 \times$ at 10^{-8} on the 128^2 pairwise cells). We further report the *best fixed schedule*, selected *per cell and per tolerance* on the test instances themselves (an optimistic baseline), from the family of HINTS with $\tau \in \{2, 5, 10, 15, 25, 50\}$, phase-shifted HINTS with the first call at iteration 0 and $\tau \in \{5, 10, 15, 25, 50\}$, and the *one-shot schedule*—the corrector once at iteration 0, then the classical solver only—which is the static rule closest to what the router learns at truncation level and therefore the natural ablation for state-dependent routing ($\tau = 1$, the corrector at every iteration, never reduces the error outside the corrector’s band and is omitted); and the best of three *residual-decay rules*, deterministic controls that decide at the router’s granularity from the same residual norm that the stopping test computes: the corrector at iteration 0, then the solver’s macro-action, and the corrector again whenever the per-iteration contraction of the residual over the last macro-action was worse than $\theta \in \{0.9, 0.95, 0.98\}$ (the threshold is selected per cell and tolerance on the test set, like the best fixed schedule); like the router, the rule is executed live in the timed replay, its decision statistic and threshold test are charged, and its decisions are verified against the untimed trace. The classical methods without corrector run on the same stencil, with the same accounting, *inside the same replay loop as the policies*—same instance, same warm-up, same random order, same three replays: multigrid V(2,2) alone; conjugate gradients, CG

preconditioned by symmetric GS and by a symmetric V-cycle (forward and backward GS, both compiled) for Poisson; BiCGSTAB, multigrid-preconditioned BiCGSTAB and restarted GMRES(20) for convection–diffusion (GMRES is accounted at the granularity of its restart cycles and is not run at 512^2); point-smoothed and line-smoothed multigrid, CG and its two preconditioned variants for anisotropic diffusion; and the sparse direct solve with a cached factorisation together with the multigrid and CG variants for variable-coefficient diffusion. The Krylov methods are SciPy implementations driven by the compiled matrix–vector product and preconditioners: their per-iteration true errors come from an untimed pass with a callback, and the timed pass runs exactly the same number of iterations with a timestamp-only callback (a few hundred nanoseconds per iteration) and verifies that it made exactly that many iterations and that its final iterate reaches the error the untimed pass recorded to within 10% (largest ratio of the two final errors over the study: 1.00; runs failing either check are excluded from the comparisons and counted: 0). An earlier version of this study timed the classical baselines in separate sessions; the same computation differed by 4–9% between sessions on identical instances (paired, $p < 10^{-4}$), so every cross-session p -value was uninformative, which is why all baselines are now timed in-session. All configuration choices of the study were made on a development test set (GRF seed 72, with the earlier numpy kernels, the earlier forcing sampler and $\tau = 25$ as reference; archived as `results_dev/`); the results reported here are a single confirmatory evaluation of the frozen configurations on a fresh test set (seed 73) with three timed replays per instance, and Section E.9 places the two side by side. The test forcings are drawn from the hierarchical GRF of Section 7 as real white noise filtered in Fourier space, which realises exactly the stated covariance with the Hermitian symmetry of a real field (the development runs used a sampler that drew independent complex coefficients on the half-spectrum, which halved the variance of the modes on the $k_2 = 0$ line); the Nyquist modes are excluded, so every forcing is a trigonometric polynomial with $|k| < N/2$ in both directions, and the instances are drawn sequentially so that the first n instances of any script are the benchmark’s. For each test instance (64 at 128^2 ; 32 at 256^2 , except 16 for anisotropic diffusion; 16 at 512^2) we record, after every iteration, the true relative error $\|e_h^{(t)}\|/\|u_h\|$ (untimed, benchmark-only) and the cumulative charged wall-clock time of a replay that executes only the chosen operations (residual and its norm, update, and—for the learned router—every feature and decision computation; oracle decisions are not charged). The reported time to a tolerance is the charged time at the first iteration whose true error is below it, identically for every method: it is a benchmark of the work needed to reach an error level, not of a deployed stopping rule (which would stop on the residual norm that every method computes anyway). We report the median first time at which the error drops below a tolerance ε ; paired speedups—the median over instances of the per-instance time ratio, which is the estimand behind every speedup and test in the speed tables and in Table 1 (the time tables show medians of raw times for scale, which can order differently from the paired median on small samples, and their bold marks compare those raw medians within a pairing)—with bootstrap 95% intervals (2000 resamples over instances) and two-sided Wilcoxon signed-rank tests on the paired log time ratios (the direction is read from the median ratio; “slower” marks a median below one); the study performs several hundred such tests, so the speedup tables mark as significant only comparisons that survive a Holm correction at level 0.05 over all tests of the table. A run that does not reach ε within the iteration cap (60,000 at 128^2 , 40,000 at 256^2 , 8,000 at 512^2 ; 5,000 units for the classical baselines, 3,000 at 512^2) is marked $\dagger$ with the number of censored instances; a median time over runs of which any is censored is the median of the mixed vector of reached times and times-to-cap, a lower bound, and is marked $\geq$ (invalid runs are dropped and counted with $\times$). In the paired comparisons the four censoring classes are decided from the censoring indicators before any substitution: a censored baseline run enters at its time-to-cap (marked $\S$ with its count), so that the median ratio is a lower bound on the true ratio whenever such caps are present and neither of the two following classes occurs (then marked $\geq$); a censored router run enters as a ratio of 0 and as the most extreme negative rank in the test, i.e. a failure penalty counted against the router (marked $\dagger$ with its count); a pair in which both are censored carries no information about the true ratio and is excluded from the median and the test (marked $\ddagger$ with its count); and a pair with an invalid run is dropped (marked $\times$). The estimand is therefore the paired median of capped-time ratios with a failure penalty for the router, not a ratio of ordinary completion times, and the p -values and intervals refer to that convention; a cell with fewer than four informative pairs prints no ratio. Every affected cell carries its class counts. Timed replays are repeated three times per instance in a random order over policies and baselines, each replay preceded by an untimed warm-up (one corrector call and one sweep, or the baseline’s own operation) so that every method starts from the same cache state—the corrector’s matrix exceeds the CPU caches, and without the warm-up a policy’s first call was up to 0.1 ms slower or faster depending on which policy had run

1143 before it; the median over replays is used. The measured per-operation costs set the macro-action
 1144 sizes and define *work units* (measured per-iteration costs times executed operations, plus the measured
 1145 decision cost per decision), a timer-free measure of the decision sequence used wherever runs from
 1146 different sessions must be compared; all reported live times are measured directly. We highlight
 1147 $\varepsilon = h^2$ as a nominal grid-dependent tolerance and checked on the confirmatory instances themselves
 1148 ([pending] instances over all equations and grids) whether it lies below their discretisation error:
 1149 comparing the exact discrete solution with the exact discrete solution on a four times finer grid
 1150 for the same forcing (`discretization_error.py`; for variable coefficients the coefficient is one
 1151 grid-independent function), the relative discretisation error, estimated as the difference between the
 1152 two discrete solutions, has median [pending]–[pending] h^2 over the equations and grids, and its
 1153 smallest ratio to h^2 over all instances is [pending]; wherever that ratio exceeds one, h^2 is a tolerance
 1154 below the (estimated) discretisation error, and driving the algebraic error further is unlikely to improve
 1155 the computed solution materially (the two error components can partly cancel, so “never” would be
 1156 too strong); we also report 10^{-3} and 10^{-8} , and the iteration-based metrics of the main text (AUC
 1157 $= \sum_{t=1}^T \|e_h^{(t)}\| / \|u_h\|$ over $T = 300$ iterations, compared by paired t -tests on $\log \text{AUC}$) on the 128^2
 1158 grid. Every router was additionally retrained from scratch with five independent seeds (different
 1159 training instances, exploration and initialisation; the corrector and the test set are fixed) and re-run on
 1160 the test set; because those trials ran in separate sessions, whose live timings are not comparable with
 1161 the main run’s, they are compared in work units.

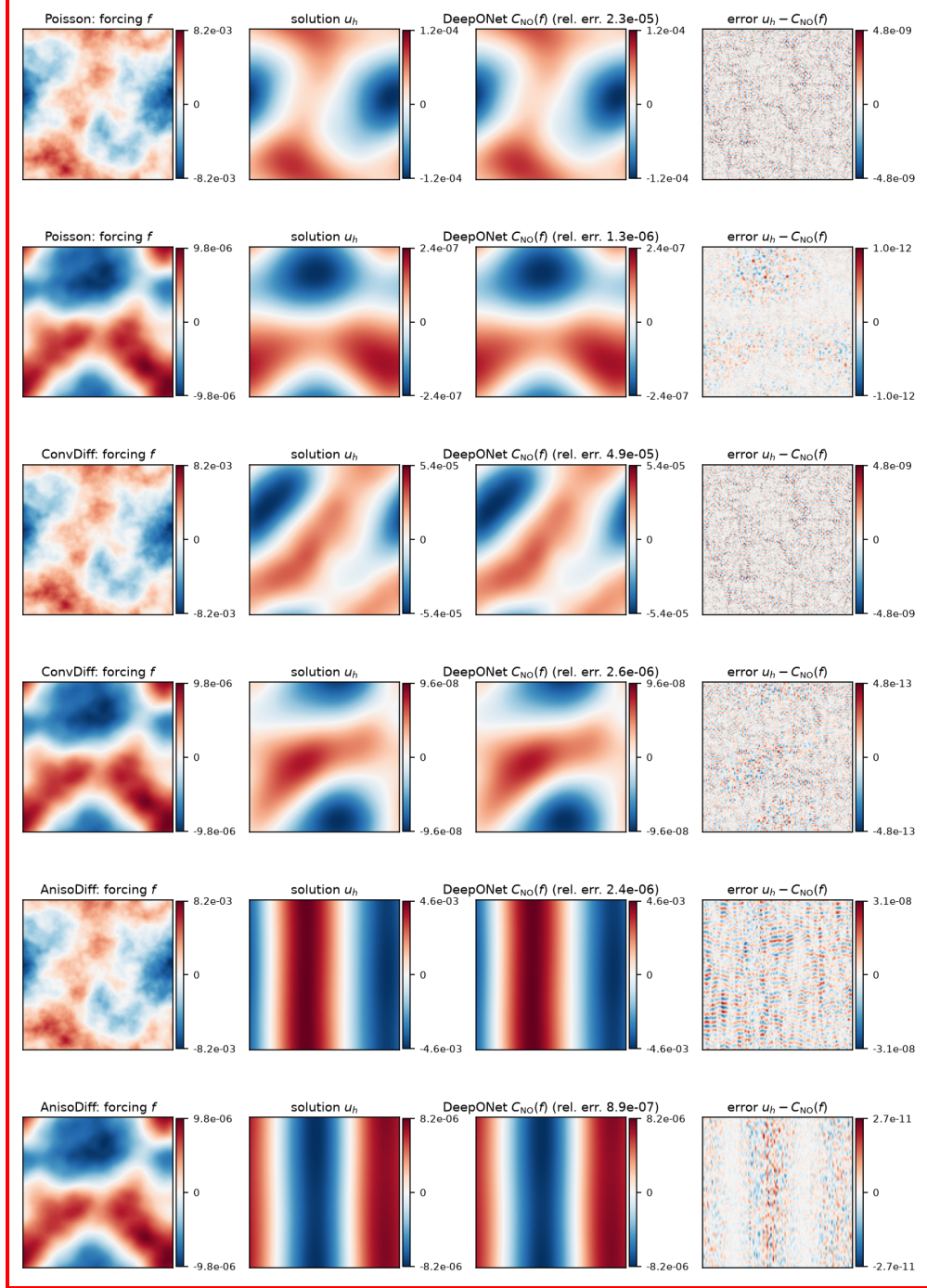


Figure 2: One-shot corrector predictions on the 128×128 grid (two test instances per equation): forcing, exact discrete solution, the DeepONet correction applied to the initial residual ($u_h^{(0)} = 0$), and the remaining error, which is confined to the modes outside the corrector's band (the relative error of the prediction is 10^{-6} – 10^{-5} on the isotropic equations and 10^{-6} on anisotropic diffusion, whose solution is dominated by modes that are smooth in x_2).

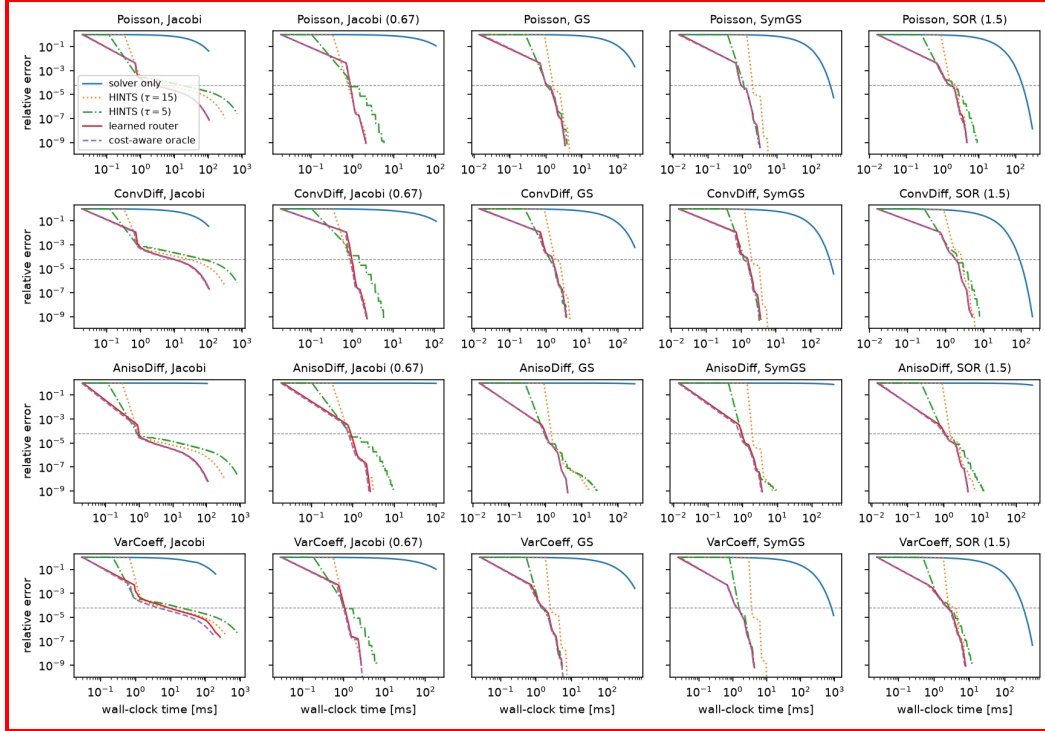


Figure 3: Relative error versus charged wall-clock time on the 128×128 grid for one representative test instance of every pairing (columns) and equation (rows); the dashed line is $\varepsilon = h^2$. The learned router and the cost-aware oracle are indistinguishable.

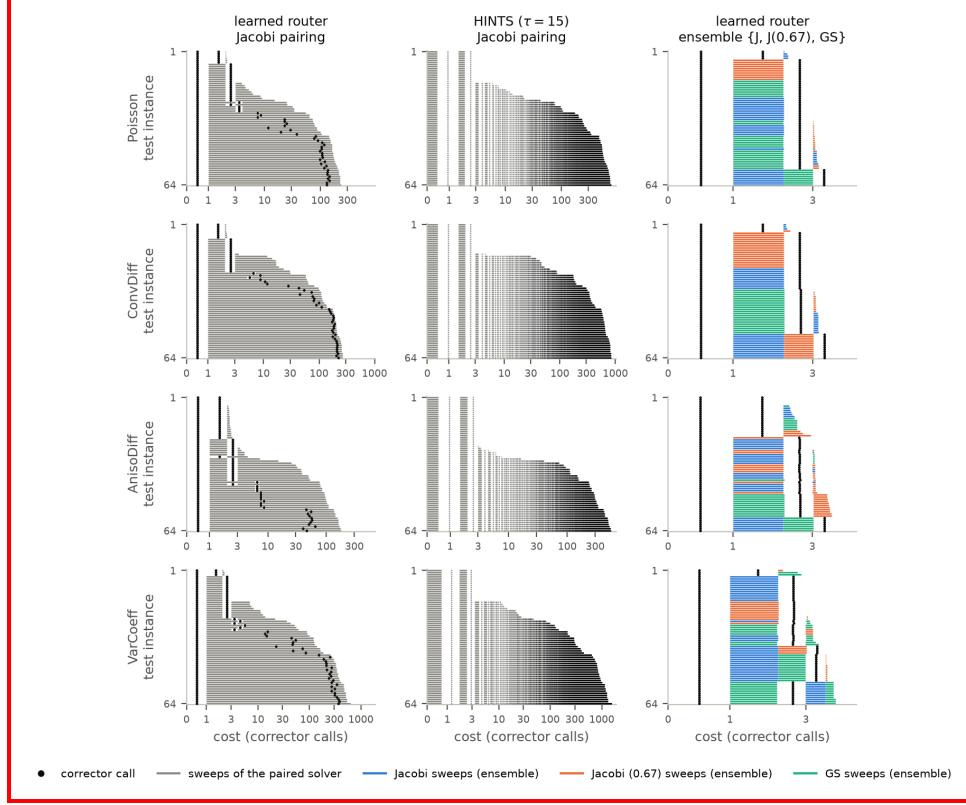


Figure 4: Decisions across test instances on the 128×128 grid: for every equation (rows), the operations of the learned router in the Jacobi pairing (left), of HINTS ($\tau=15$) in the same pairing on the same instances in the same order (middle), and of the learned router in the ensemble $\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$ (right), until the relative error first falls below 10^{-8} , as executed in the timed replays. Each row is one test instance; dots mark corrector calls and lines the solver sweeps between them (coloured by solver in the ensemble); the horizontal axis is the cumulative cost in units of one corrector call, linear below one call and logarithmic above. HINTS calls the corrector at the same iterations on every instance by construction; the learned router’s calls, and its choice of solver in the ensemble, are read off per instance. The per-equation versions for every pairing are Figures 6, 8, 10 and 12.

1162 E.2 Poisson

1163 Table 4 collects the median time to 10^{-3} , h^2 and 10^{-8} for every pairing, every policy and the classical
 1164 baselines on all three grids; Table 5 the paired speedups of the learned router with their p -values;
 1165 Figure 5 and table 6 where the iterations go; Table 7 the measured costs and macro-action sizes;
 1166 Table 8 the iteration-based metrics; and Table 9 the seed trials.

1167 **Two regimes.** At $\varepsilon = h^2$ the router makes one corrector call on 80–100% of the instances (over all
 1168 128^2 pairwise cells) and then smooths (Table 6): the router executes the same decision sequence to
 1169 h^2 as the best static schedule on 67–100% of the instances and as the cost-aware oracle on 50–100%
 1170 (over all 128^2 pairwise cells, every equation; Section 7) and the router’s paired ratio against the best
 1171 fixed schedule is close to one (Table 5; bold marks the comparisons that remain significant after
 1172 the Holm correction), while HINTS with its published period, whose first call comes after $\tau - 1$
 1173 wasted sweeps, is slower by the factors in the table; the ratio of the solver alone to the router ranges
 1174 over $0.95 \times$ – $7083 \times$ over all 128^2 pairwise cells (every equation), the low end being the Multigrid
 1175 pairing (multigrid alone reaches h^2 in a few cycles: $0.94 \times$ – $2.30 \times$ against the $\{\text{NO}, \text{multigrid}\}$
 1176 router). Below h^2 the remaining error is the band the corrector cannot see, and the schedules separate:
 1177 the router calls the corrector again only when the smooth error has re-emerged (two or three calls
 1178 down to 10^{-8} ; Figure 5), the one-shot schedule never calls it again and pays for hundreds of extra

sweeps, and a periodic schedule keeps paying for calls whether or not they help, so at 10^{-8} the paired ratios against HINTS and the one-shot schedule are $0.92\times\text{--}3.07\times$ and $0.97\times\text{--}1373\times$ over all 128^2 pairwise cells (every equation), whereas against the best fixed schedule and the best residual-decay rule—both selected on the test set—the outcome depends on the pairing (Table 5; the counts over all pairings are in Section 7, and Section E.6 tests whether the macro-action granularity explains the losses). The cost-agnostic greedy oracle (Algorithm 1 on single operations) makes the same decisions as its cost-aware counterpart on the path to h^2 (they differ later, in when the corrector is called again); the ratio of the learned router’s median time to the cost-aware oracle’s is $1.01\times\text{--}1.09\times$ and the two execute the same decision sequence to h^2 on 50–100% of the instances (over all 128^2 pairwise cells), and the five retrained routers are compared with the main router in Table 9 **[pending]**.

Classical baselines and larger grids. Against the classical methods without corrector (bottom block of Table 4; timed in the same session as the router of every pairing), multigrid alone has the smallest ratio to the router among the classical methods in the current cells: at 128^2 the paired ratio of multigrid alone to the router (above one favours the router; over all 128^2 pairwise cells) is $0.95\times\text{--}11.2\times$ at $\varepsilon = h^2$ and $0.03\times\text{--}20.5\times$ at 10^{-8} , because one corrector call removes the modes $|k| \leq 31$ at the price of one matrix–vector product whereas a V(2,2)-cycle contracts the whole spectrum by ≈ 0.03 on the first cycle and ≈ 0.08 asymptotically; routing over {NO, multigrid} calls the corrector once and lets the cycles remove the rest ($0.94\times\text{--}2.30\times$ of multigrid alone at h^2 and $1.11\times\text{--}1.63\times$ at 10^{-8}), and the ratio of multigrid-preconditioned CG to the router is $0.95\times\text{--}3.00\times$ at h^2 and $0.03\times\text{--}3.46\times$ at 10^{-8} . The FFT direct solve, exact for this constant-coefficient periodic problem, is the reference the router takes $3.08\times\text{--}3.25\times$ times as long as at 128^2 ; the larger grids are **[pending]**. At 256^2 and 512^2 **[pending]** the same two regimes are expected to hold with shrinking margins for the Jacobi and Gauss–Seidel pairings, because the corrector’s fixed band $|k| \leq 31$ leaves the smoother all modes $k \geq 32$, whose slowest member contracts by only 0.98 per Jacobi sweep at $N = 512$ (against 0.71 at $N = 128$): every schedule then needs the same ≈ 80 sweeps after the first call, the wall-clock time is dominated by sweeps, and the corrector—whose cost is nearly grid-independent—is worth fewer and fewer of them, so the best classical method at 512^2 is multigrid, alone or with one corrector call. This is the expected limit of a corrector whose band does not grow with the grid.

Table 4: Poisson: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids (64, 32 and 16 test instances) for every pairing $\{\text{NO}, \text{solver}\}$ and policy, and for the classical baselines without corrector. “Best fixed schedule” is the best of HINTS ($\tau \in \{2, 5, 10, 15, 25, 50\}$), phase-shifted HINTS ($\tau \in \{5, 10, 15, 25, 50\}$) and the one-shot schedule, and “Best residual-decay rule” the best threshold $\theta \in \{0.9, 0.95, 0.98\}$, both chosen per cell and tolerance on the test set itself (the chosen schedule is the subscript). The greedy oracle is the cost-agnostic Algorithm 1, the cost-aware oracle is Algorithm 1 on cost-equalised macro-actions; both use the true error and are not deployable. Bold marks the router where its median is the smallest among the deployable methods of its pairing (solver alone, every fixed schedule, the one-shot and residual-decay rules); the oracles are in italics; $\dagger n$: n instances did not reach the tolerance within the iteration cap ($>$: lower bound).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	232.1 ms	326.6 ms	623.3 ms
	HINTS ($\tau=15$)	976 μ s	977 μ s	136.8 ms
	Best fixed schedule	671 μ s $_{\tau=25, t_0=0}$	702 μ s $_{\tau=50, t_0=0}$	65.3 ms $_{\text{one-shot}}$
	Best residual-decay rule	690 μ s $_{\theta=0.98}$	704 μ s $_{\theta=0.95}$	85.8 ms $_{\theta=0.9}$
	One-shot schedule	684 μ s	704 μ s	65.3 ms
	Greedy oracle (Alg. 1)	683 μ s	707 μ s	44.0 ms
	Cost-aware oracle	686 μ s	710 μ s	45.9 ms
	Learned router (ours)	703 μ s	723 μ s	45.6 ms
{NO, Jacobi (0.67)}	Solver only	356.7 ms	502.6 ms	958.5 ms
	HINTS ($\tau=15$)	1.01 ms	1.01 ms	2.01 ms
	Best fixed schedule	705 μ s $_{\tau=25, t_0=0}$	718 μ s $_{\tau=50, t_0=0}$	1.77 ms $_{\tau=15, t_0=0}$
	Best residual-decay rule	704 μ s $_{\theta=0.98}$	721 μ s $_{\theta=0.98}$	2.76 ms $_{\theta=0.95}$
	One-shot schedule	712 μ s	724 μ s	78.7 ms
	Greedy oracle (Alg. 1)	700 μ s	727 μ s	1.79 ms
	Cost-aware oracle	701 μ s	716 μ s	2.06 ms
	Learned router (ours)	736 μ s	763 μ s	2.14 ms
{NO, GS}	Solver only	328.2 ms	461.2 ms	877.2 ms
	HINTS ($\tau=15$)	1.53 ms	1.58 ms	4.56 ms
	Best fixed schedule	689 μ s $_{\tau=10, t_0=0}$	712 μ s $_{\tau=10, t_0=0}$	2.01 ms $_{\tau=10, t_0=0}$
	Best residual-decay rule	696 μ s $_{\theta=0.95}$	713 μ s $_{\theta=0.98}$	2.78 ms $_{\theta=0.9}$
	One-shot schedule	699 μ s	715 μ s	114.2 ms
	Greedy oracle (Alg. 1)	693 μ s	733 μ s	2.52 ms
	Cost-aware oracle	696 μ s	728 μ s	2.08 ms
	Learned router (ours)	721 μ s	758 μ s	2.13 ms
{NO, SymGS}	Solver only	247.5 ms	348.0 ms	661.8 ms
	HINTS ($\tau=15$)	1.97 ms	1.99 ms	4.02 ms
	Best fixed schedule	684 μ s $_{\text{one-shot}}$	725 μ s $_{\text{one-shot}}$	1.89 ms $_{\tau=5, t_0=0}$
	Best residual-decay rule	691 μ s $_{\theta=0.98}$	734 μ s $_{\theta=0.95}$	3.33 ms $_{\theta=0.9}$
	One-shot schedule	684 μ s	725 μ s	81.7 ms
	Greedy oracle (Alg. 1)	694 μ s	744 μ s	1.91 ms
	Cost-aware oracle	687 μ s	752 μ s	2.02 ms
	Learned router (ours)	714 μ s	793 μ s	2.08 ms
{NO, SOR (1.5)}	Solver only	110.1 ms	155.2 ms	295.3 ms
	HINTS ($\tau=15$)	1.55 ms	3.06 ms	4.60 ms
	Best fixed schedule	687 μ s $_{\text{one-shot}}$	718 μ s $_{\tau=25, t_0=0}$	2.72 ms $_{\tau=15, t_0=0}$
	Best residual-decay rule	692 μ s $_{\theta=0.98}$	728 μ s $_{\theta=0.98}$	4.08 ms $_{\theta=0.9}$
	One-shot schedule	687 μ s	733 μ s	58.0 ms
	Greedy oracle (Alg. 1)	698 μ s	731 μ s	3.02 ms
	Cost-aware oracle	691 μ s	728 μ s	2.75 ms
	Learned router (ours)	723 μ s	777 μ s	2.86 ms
{NO, Multigrid}	Solver only	698 μ s	1.01 ms	2.00 ms
	HINTS ($\tau=15$)	699 μ s	1.01 ms	2.00 ms
	Best fixed schedule	696 μ s $_{\tau=5, t_0=0}$	850 μ s $_{\tau=10, t_0=0}$	1.71 ms $_{\tau=15, t_0=0}$
	Best residual-decay rule	700 μ s $_{\theta=0.9}$	851 μ s $_{\theta=0.95}$	1.71 ms $_{\theta=0.95}$
	One-shot schedule	705 μ s	872 μ s	1.73 ms
	Greedy oracle (Alg. 1)	697 μ s	855 μ s	1.73 ms
	Cost-aware oracle	696 μ s	867 μ s	1.71 ms
	Learned router (ours)	722 μ s	880 μ s	1.76 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone (point GS)	688 μ s	1.01 ms	2.00 ms
	CG	4.51 ms	5.39 ms	7.59 ms
	PCG (SymGS)	6.08 ms	9.28 ms	15.8 ms
	PCG (multigrid)	708 μ s	982 μ s	1.70 ms
	FFT direct solve (exact; constant coefficients only)	235 μ s	235 μ s	235 μ s

Table 5: Poisson: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$), over the best fixed period for that cell, over multigrid alone and over multigrid-preconditioned CG, with the two-sided Wilcoxon p -value (“slower” marks a median ratio below one) and the bootstrap 95% interval; bold: paired ratio $\geq 1.1\times$ and significant after a Holm correction over all tests of the table.

Pairing	τ	128 $\times$ 128					
		vs. HINTS-15	vs. best schedule	vs. best decay rule	vs. one-shot	vs. multigrid	vs. MG-Krylov
{NO, Jacobi}	h^2	1.40 (1.37, 1.41) ($< 10^{-10}$)	0.97 (0.96, 0.98) (slower, 0.221)	0.97 (0.96, 0.99) (slower, 0.129)	0.96 (0.95, 0.97) (slower, 9×10^{-7})	1.02 (1.00, 1.06) (0.282)	1.03 (1.02, 1.05) (0.097)
	10^{-8}	3.05 (2.85, 3.14) ($< 10^{-10}$)	1.18 (1.04, 1.35) (10^{-7})	1.92 (1.84, 1.95) ($< 10^{-10}$)	1.18 (1.04, 1.35) (10^{-7})	0.04 (0.03, 0.12) (slower, $< 10^{-10}$)	0.04 (0.02, 0.10) (slower, $< 10^{-10}$)
{NO, Jacobi (0.67)}	h^2	1.32 (1.23, 1.36) ($< 10^{-10}$)	0.95 (0.94, 0.96) (slower, 10^{-7})	0.95 (0.94, 0.96) (slower, 10^{-10})	0.96 (0.95, 0.97) (slower, 2×10^{-9})	1.12 (1.03, 1.15) (10^{-7})	1.07 (1.00, 1.13) ($< 10^{-10}$)
	10^{-8}	0.91 (0.91, 0.91) (slower, $< 10^{-10}$)	0.83 (0.81, 0.85) (slower, 5×10^{-10})	1.36 (1.28, 1.42) ($< 10^{-10}$)	3.68 (3.28, 4.01) ($< 10^{-10}$)	0.93 (0.85, 0.95) (slower, 7×10^{-7})	0.79 (0.78, 0.80) (slower, $< 10^{-10}$)
{NO, GS}	h^2	2.09 (1.85, 2.17) ($< 10^{-10}$)	0.96 (0.94, 0.97) (slower, 7×10^{-10})	0.97 (0.95, 0.98) (slower, 5×10^{-10})	0.96 (0.95, 0.97) (slower, 3×10^{-7})	0.98 (0.96, 1.01) (slower, 0.229)	1.01 (0.98, 1.04) (0.288)
	10^{-8}	2.12 (1.64, 2.17) ($< 10^{-10}$)	0.91 (0.91, 0.92) (slower, 9×10^{-10})	1.30 (1.28, 1.32) (10^{-10})	51.9 (42.7, 72.7) ($< 10^{-10}$)	0.90 (0.74, 0.82) (slower, $< 10^{-10}$)	0.66 (0.62, 0.75) (slower, $< 10^{-10}$)
{NO, SymGS}	h^2	2.57 (2.39, 2.66) ($< 10^{-10}$)	0.93 (0.94, 0.95) (slower, $< 10^{-10}$)	0.96 (0.95, 0.97) (slower, $< 10^{-10}$)	0.95 (0.94, 0.96) (slower, $< 10^{-10}$)	1.01 (0.98, 1.08) (0.030)	1.02 (0.99, 1.05) (0.052)
	10^{-8}	1.91 (1.84, 1.96) ($< 10^{-10}$)	0.86 (0.84, 0.88) (slower, 4×10^{-10})	1.39 (1.24, 1.61) ($< 10^{-10}$)	39.8 (33.1, 57.2) ($< 10^{-10}$)	0.83 (0.62, 0.94) (slower, $< 10^{-10}$)	0.72 (0.67, 0.77) (slower, $< 10^{-10}$)
{NO, SOR (1.5)}	h^2	3.65 (2.85, 4.46) ($< 10^{-10}$)	0.96 (0.95, 0.97) (slower, 2×10^{-9})	0.96 (0.95, 0.97) (slower, 6×10^{-10})	0.96 (0.94, 0.98) (slower, 9×10^{-7})	0.96 (0.94, 0.97) (slower, 0.005)	0.92 (0.92, 0.98) (slower, 6×10^{-10})
	10^{-8}	1.60 (1.21, 1.84) ($< 10^{-10}$)	0.97 (0.97, 0.99) (slower, 0.004)	1.42 (1.38, 1.50) ($< 10^{-10}$)	19.8 (14.8, 23.1) ($< 10^{-10}$)	0.66 (0.56, 0.71) (slower, $< 10^{-10}$)	0.59 (0.45, 0.64) (slower, $< 10^{-10}$)
{NO, Multigrid}	h^2	0.97 (0.96, 0.98) (slower, 3×10^{-10})	0.97 (0.96, 0.97) (slower, $< 10^{-10}$)	0.97 (0.97, 0.98) (slower, $< 10^{-10}$)	0.97 (0.97, 0.98) (slower, 3×10^{-10})	0.97 (0.96, 0.97) (slower, 4×10^{-10})	0.96 (0.95, 0.97) (slower, 9×10^{-10})
	10^{-8}	1.15 (1.12, 1.18) ($< 10^{-10}$)	0.98 (0.97, 0.98) (slower, 8×10^{-10})	0.98 (0.97, 0.98) (slower, 10^{-10})	0.98 (0.97, 0.98) (slower, 7×10^{-10})	1.15 (1.12, 1.17) ($< 10^{-10}$)	0.94 (0.85, 0.96) (slower, 2×10^{-10})

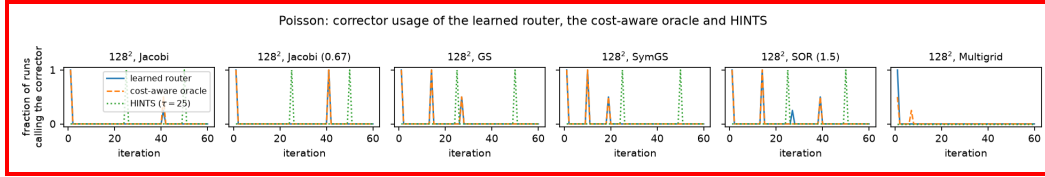


Figure 5: Poisson: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=15$), for every pairing (columns) and grid (rows; all test instances of each grid).

Table 6: Poisson: median iterations (and corrector calls) to $\varepsilon = h^2$ for HINTS ($\tau=15$), the cost-aware oracle and the learned router on every grid.

Pairing	128 $\times$ 128		
	HINTS-15	oracle	router
{NO, Jacobi}	15 (1)	2 (1)	2 (1)
{NO, Jacobi (0.67)}	15 (1)	2 (1)	2 (1)
{NO, GS}	16 (1)	2 (1)	2 (1)
{NO, SymGS}	15 (1)	2 (1)	2 (1)
{NO, SOR (1.5)}	30 (2)	2 (1)	2 (1)
{NO, Multigrid}	3 (0)	2 (1)	2 (1)

Table 7: Poisson: measured cost of one classical iteration c_{solver} and of one corrector call c_{NO} (residual evaluation included), and the resulting macro-action size m_{solver} (sweeps per decision; the corrector is always applied once).

Pairing	128×128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	21 μs	672 μs	32
{NO, Jacobi (0.67)}	21 μs	672 μs	32
{NO, GS}	61 μs	672 μs	11
{NO, SymGS}	92 μs	672 μs	7
{NO, SOR (1.5)}	61 μs	672 μs	11
{NO, Multigrid}	326 μs	672 μs	2

Table 8: Poisson, 128×128 : iteration-based metrics of the main text—relative error after $T = 300$ iterations and AUC over the T iterations (mean and standard error over 64 instances; p : one-sided paired t -test that the method’s AUC exceeds the router’s). Bold: router better than both solver-only and HINTS.

Method	AUC	p
{NO, Jacobi}		
Solver only	2.69e+02 (7.2e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (2.7e-03)	$< 10^{-10}$
Learned router (ours)	6.08e-03 (1.5e-03)	-
Cost-aware oracle	6.08e-03 (1.5e-03)	-
{NO, Jacobi (0.67)}		
Solver only	2.78e+02 (5.7e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (2.4e-03)	$< 10^{-10}$
Learned router (ours)	2.13e-03 (5.0e-04)	-
Cost-aware oracle	2.13e-03 (5.0e-04)	-
{NO, GS}		
Solver only	2.44e+02 (1.0e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.38e+01 (6.1e-03)	$< 10^{-10}$
Learned router (ours)	1.71e-03 (4.1e-04)	-
Cost-aware oracle	1.71e-03 (4.1e-04)	-
{NO, SymGS}		
Solver only	2.05e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.37e+01 (1.1e-02)	$< 10^{-10}$
Learned router (ours)	1.26e-03 (3.0e-04)	-
Cost-aware oracle	1.26e-03 (3.0e-04)	-
{NO, SOR (1.5)}		
Solver only	1.75e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.35e+01 (1.4e-02)	$< 10^{-10}$
Learned router (ours)	2.79e-03 (6.7e-04)	-
Cost-aware oracle	2.80e-03 (6.7e-04)	-

Table 9: Poisson, 128×128 : five independent router-training seeds per pairing—mean and standard deviation over seeds of the median work-unit time to h^2 , the share of test instances on which the seed router’s decision sequence to h^2 coincides with the main router’s, the range over seeds of the paired speedup over HINTS ($\tau=15$), and the number of seeds significantly faster ($p < 0.01$) than both HINTS ($\tau=15$) and the best fixed period.

[results pending: confirmatory run in progress]

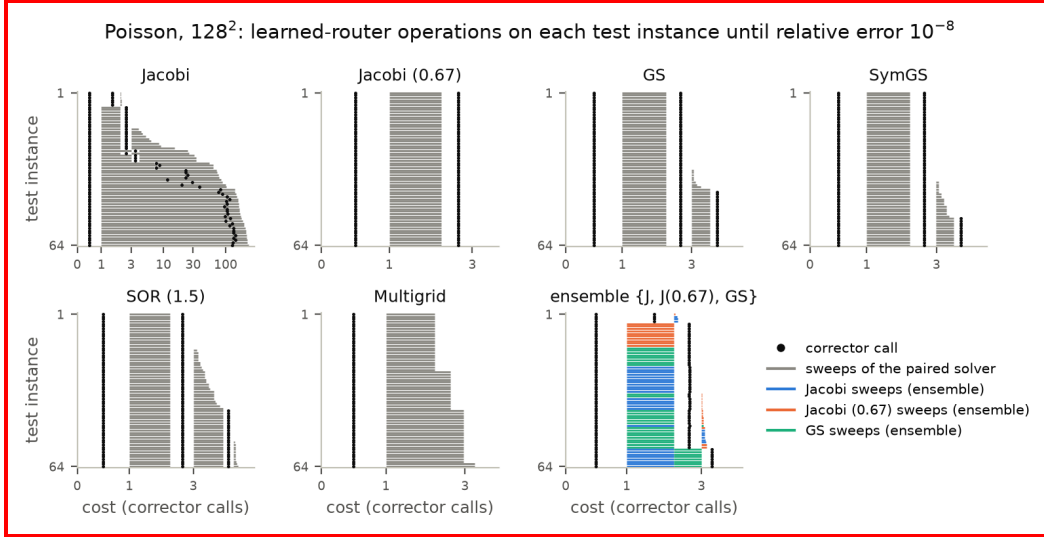


Figure 6: Poisson, 128×128 : operations of the learned router on each of the 64 test instances (rows, sorted by cost) until the relative error first falls below 10^{-8} , for every pairing and for the ensemble $\{\text{Jacobi}, \text{Jacobi (0.67)}, \text{GS}\}$, as executed in the timed replays. Dots mark corrector calls and lines the solver sweeps between them (coloured by solver in the ensemble); the horizontal axis is the cumulative cost in units of one corrector call, linear below one call and logarithmic above.

1208 E.3 Convection–diffusion

1209 The convection–diffusion results (Tables 10 to 15 and fig. 7) mirror the Poisson ones: **[pending]**
 1210 under the revised protocol. In the first confirmatory run (revision 1, reference $\tau = 25$) they mirrored
 1211 the Poisson ones, with the nonsymmetric operator changing only the details: the undamped-Jacobi
 1212 pairing stalled at the near-Nyquist band below 10^{-6} for every policy, and GMRES(20) was far slower
 1213 than the router.

Table 10: Convection–diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids for every pairing and policy, and for the classical baselines without corrector (conventions as in Table 4).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	227.9 ms	323.0 ms	619.5 ms
	HINTS ($\tau=15$)	976 μ s	986 μ s	201.2 ms
	Best fixed schedule	678 μ s _{one-shot}	712 μ s _{one-shot}	67.8 ms _{one-shot}
	Best residual-decay rule	677 μ s $\theta=0.95$	706 μ s $\theta=0.95$	126.7 ms $\theta=0.95$
	One-shot schedule	678 μ s	712 μ s	67.8 ms
	Greedy oracle (Alg. 1)	675 μ s	720 μ s	65.3 ms
	Cost-aware oracle	689 μ s	716 μ s	62.8 ms
	Learned router (ours)	717 μ s	746 μ s	66.4 ms
{NO, Jacobi (0.67)}	Solver only	354.4 ms	502.4 ms	964.9 ms
	HINTS ($\tau=15$)	1.03 ms	1.03 ms	2.05 ms
	Best fixed schedule	724 μ s $\tau=5, t_0=0$	750 μ s $\tau=10, t_0=0$	1.86 ms $\tau=15, t_0=0$
	Best residual-decay rule	721 μ s $\theta=0.95$	753 μ s $\theta=0.98$	2.88 ms $\theta=0.95$
	One-shot schedule	735 μ s	766 μ s	64.2 ms
	Greedy oracle (Alg. 1)	726 μ s	762 μ s	1.88 ms
	Cost-aware oracle	728 μ s	764 μ s	2.15 ms
	Learned router (ours)	764 μ s	813 μ s	2.21 ms
{NO, GS}	Solver only	273.9 ms	388.2 ms	742.4 ms
	HINTS ($\tau=15$)	1.58 ms	1.88 ms	4.70 ms
	Best fixed schedule	726 μ s $\tau=10, t_0=0$	794 μ s _{one-shot}	2.35 ms $\tau=15, t_0=0$
	Best residual-decay rule	736 μ s $\theta=0.98$	805 μ s $\theta=0.98$	2.97 ms $\theta=0.9$
	One-shot schedule	730 μ s	794 μ s	149.7 ms
	Greedy oracle (Alg. 1)	741 μ s	793 μ s	2.76 ms
	Cost-aware oracle	741 μ s	824 μ s	2.32 ms
	Learned router (ours)	752 μ s	853 μ s	2.40 ms
{NO, SymGS}	Solver only	239.7 ms	339.8 ms	652.5 ms
	HINTS ($\tau=15$)	2.05 ms	2.15 ms	4.33 ms
	Best fixed schedule	759 μ s _{one-shot}	807 μ s $\tau=25, t_0=0$	2.14 ms $\tau=5, t_0=0$
	Best residual-decay rule	753 μ s $\theta=0.98$	824 μ s $\theta=0.98$	3.78 ms $\theta=0.9$
	One-shot schedule	759 μ s	822 μ s	118.2 ms
	Greedy oracle (Alg. 1)	761 μ s	812 μ s	2.68 ms
	Cost-aware oracle	766 μ s	830 μ s	2.25 ms
	Learned router (ours)	783 μ s	864 μ s	2.33 ms
{NO, SOR (1.5)}	Solver only	63.5 ms	90.0 ms	172.7 ms
	HINTS ($\tau=15$)	1.59 ms	3.14 ms	4.71 ms
	Best fixed schedule	727 μ s _{one-shot}	847 μ s $\tau=15, t_0=0$	3.14 ms $\tau=25, t_0=0$
	Best residual-decay rule	727 μ s $\theta=0.95$	867 μ s $\theta=0.95$	3.72 ms $\theta=0.9$
	One-shot schedule	727 μ s	869 μ s	47.5 ms
	Greedy oracle (Alg. 1)	727 μ s	852 μ s	3.31 ms
	Cost-aware oracle	728 μ s	851 μ s	3.06 ms
	Learned router (ours)	752 μ s	906 μ s	3.20 ms
{NO, Multigrid}	Solver only	714 μ s	1.04 ms	2.01 ms
	HINTS ($\tau=15$)	714 μ s	1.05 ms	2.04 ms
	Best fixed schedule	713 μ s $\tau=5$	1.04 ms _{one-shot}	1.74 ms $\tau=15, t_0=0$
	Best residual-decay rule	736 μ s $\theta=0.9$	1.05 ms $\theta=0.98$	1.76 ms $\theta=0.95$
	One-shot schedule	743 μ s	1.04 ms	1.76 ms
	Greedy oracle (Alg. 1)	730 μ s	1.06 ms	1.76 ms
	Cost-aware oracle	713 μ s	1.01 ms	1.89 ms
	Learned router (ours)	756 μ s	1.09 ms	1.83 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone (point GS)	688 μ s	1.02 ms	2.00 ms
	BiCGSTAB	18.0 ms	21.5 ms	29.7 ms
	BiCGSTAB (multigrid)	752 μ s	1.46 ms	2.16 ms
	GMRES(20)	178.5 ms	254.9 ms	488.6 ms
	FFT direct solve (exact; constant coefficients only)	235 μ s	235 μ s	235 μ s

Table 11: Convection–diffusion: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$), the best fixed period, multigrid alone and multigrid-preconditioned BiCGSTAB, with two-sided Wilcoxon p -values (conventions as in Table 5).

Pairing	ε	128 × 128						
		vs. HINTS-15	vs. best schedule	vs. best decay rule	vs. one-shot	vs. multigrid	vs. MG-Kayhan	vs. FFT solve
{NO, Jacobi}	h^2	1.39 × [1.18, 1.42] ($< 10^{-10}$)	0.95 × [0.94, 0.97] (closer, 10^{-4})	0.96 × [0.96, 0.96] (closer, 0.051)	0.95 × [0.94, 0.97] (closer, 10^{-4})	1.00 × [0.98, 1.17] (0.959)	1.73 × [1.28, 1.82] (0.014)	0.31 × [0.28, 0.32] (closer, $< 10^{-10}$)
	10^{-8}	3.07 × [2.92, 3.16] ($< 10^{-10}$)	1.03 × [0.99, 1.43] (4×10^{-4})	1.95 × [1.87, 1.99] ($< 10^{-10}$)	1.03 × [0.99, 1.43] (4×10^{-4})	0.03 × [0.02, 0.06] (closer, $< 10^{-10}$)	0.03 × [0.02, 0.06] (closer, 5×10^{-10})	0.00 × [0.00, 0.01] (closer, $< 10^{-10}$)
{NO, Jacobi (0.67)}	h^2	1.26 × [1.18, 1.33] ($< 10^{-10}$)	0.97 × [0.95, 0.98] (closer, 0.007)	0.97 × [0.95, 0.97] (closer, 10^{-4})	0.97 × [0.96, 0.98] (closer, 3×10^{-4})	1.11 × [1.06, 1.18] (7×10^{-4})	1.49 × [1.41, 1.62] ($< 10^{-10}$)	0.31 × [0.29, 0.32] (closer, $< 10^{-10}$)
	10^{-8}	0.92 × [0.91, 0.93] (closer, $< 10^{-10}$)	0.83 × [0.81, 0.87] (closer, 10^{-4})	1.30 × [1.28, 1.31] ($< 10^{-10}$)	25.0 × [26.0, 31.0] ($< 10^{-10}$)	0.90 × [0.80, 0.92] (closer, $< 10^{-10}$)	0.98 × [0.97, 0.99] (closer, 3×10^{-4})	0.11 × [0.11, 0.12] (closer, $< 10^{-10}$)
{NO, GS}	h^2	2.30 × [2.14, 2.37] ($< 10^{-10}$)	0.96 × [0.95, 0.97] (closer, 0.006)	0.97 × [0.96, 0.98] (closer, 0.035)	0.96 × [0.95, 0.97] (closer, 0.006)	0.99 × [0.95, 1.01] (closer, 0.057)	1.57 × [1.43, 1.73] (4×10^{-4})	0.29 × [0.28, 0.33] (closer, $< 10^{-10}$)
	10^{-8}	1.96 × [1.86, 2.07] ($< 10^{-10}$)	1.03 × [1.02, 1.03] (0.001)	1.31 × [1.27, 1.33] ($< 10^{-10}$)	84.9 × [81.5, 97.8] ($< 10^{-10}$)	0.76 × [0.67, 0.77] (closer, $< 10^{-10}$)	0.90 × [0.65, 0.93] (closer, $< 10^{-10}$)	0.10 × [0.08, 0.11] (closer, $< 10^{-10}$)
{NO, SymGS}	h^2	2.51 × [2.30, 2.71] ($< 10^{-10}$)	0.94 × [0.93, 0.96] (closer, 3×10^{-4})	0.95 × [0.94, 0.97] (closer, 2×10^{-4})	0.95 × [0.94, 0.96] (closer, 2×10^{-4})	1.00 × [0.93, 1.11] (0.973)	1.58 × [1.46, 1.73] (2×10^{-4})	0.29 × [0.27, 0.32] (closer, $< 10^{-10}$)
	10^{-8}	1.81 × [1.76, 1.87] ($< 10^{-10}$)	0.84 × [0.81, 0.87] (closer, 9×10^{-4})	1.61 × [1.57, 1.64] ($< 10^{-10}$)	90.3 × [84.7, 99.5] ($< 10^{-10}$)	0.74 × [0.71, 0.77] (closer, $< 10^{-10}$)	0.97 × [0.63, 0.93] (closer, $< 10^{-10}$)	0.11 × [0.09, 0.11] (closer, $< 10^{-10}$)
{NO, SOR (1.5)}	h^2	3.46 × [2.96, 4.12] ($< 10^{-10}$)	0.96 × [0.96, 0.97] (closer, 5×10^{-4})	0.97 × [0.96, 0.98] (closer, 0.032)	0.97 × [0.96, 0.98] (closer, 0.022)	0.95 × [0.90, 1.00] (closer, 0.057)	1.46 × [1.12, 1.69] (8×10^{-4})	0.29 × [0.22, 0.33] (closer, $< 10^{-10}$)
	10^{-8}	1.50 × [1.26, 1.83] ($< 10^{-10}$)	1.24 × [1.23, 1.29] ($< 10^{-10}$)	1.48 × [1.41, 1.52] ($< 10^{-10}$)	14.5 × [12.4, 16.1] ($< 10^{-10}$)	0.61 × [0.54, 0.69] (closer, $< 10^{-10}$)	0.68 × [0.58, 0.84] (closer, $< 10^{-10}$)	0.08 × [0.07, 0.10] (closer, $< 10^{-10}$)
{NO, Multigrid}	h^2	0.95 × [0.93, 0.96] (closer, 0.009)	0.96 × [0.96, 0.97] (closer, 7×10^{-10})	0.97 × [0.96, 0.98] (closer, 2×10^{-4})	0.96 × [0.96, 0.97] (closer, 7×10^{-10})	0.84 × [0.93, 0.95] (closer, 0.004)	1.34 × [1.28, 1.39] ($< 10^{-10}$)	0.23 × [0.22, 0.32] (closer, $< 10^{-10}$)
	10^{-8}	1.11 × [0.96, 1.14] (5×10^{-3})	0.97 × [0.96, 0.97] (closer, $< 10^{-10}$)	0.98 × [0.97, 0.98] (closer, $< 10^{-10}$)	0.97 × [0.96, 0.98] (closer, 7×10^{-10})	1.11 × [0.97, 1.13] ($< 10^{-10}$)	1.19 × [1.08, 1.23] (5×10^{-4})	0.14 × [0.12, 0.15] (closer, $< 10^{-10}$)

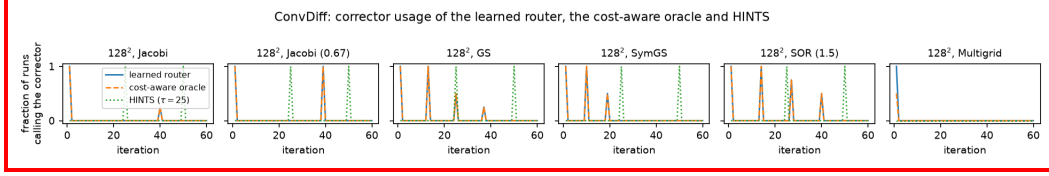


Figure 7: Convection–diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=15$) for every pairing (columns) and grid (rows).

Table 12: Convection–diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$ on every grid.

Pairing	128 × 128		
	HINTS-15	oracle	router
{NO, Jacobi}	15 (1)	2 (1)	2 (1)
{NO, Jacobi (0.67)}	15 (1)	3 (1)	3 (1)
{NO, GS}	20 (1)	2 (1)	2 (1)
{NO, SymGS}	16 (1)	2 (1)	2 (1)
{NO, SOR (1.5)}	30 (2)	4 (1)	4 (1)
{NO, Multigrid}	3 (0)	2 (0)	2 (1)

Table 13: Convection–diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	21 μs	626 μs	30
{NO, Jacobi (0.67)}	19 μs	626 μs	34
{NO, GS}	54 μs	626 μs	12
{NO, SymGS}	81 μs	626 μs	8
{NO, SOR (1.5)}	54 μs	626 μs	12
{NO, Multigrid}	288 μs	626 μs	2

Table 14: Convection–diffusion, 128×128 : iteration-based metrics (conventions as in Table 8).

Method	AUC	p
{NO, Jacobi}		
Solver only	2.60e+02 (1.4e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (5.7e-03)	$< 10^{-10}$
Learned router (ours)	1.38e-02 (3.2e-03)	-
Cost-aware oracle	<i>1.38e-02 (3.2e-03)</i>	-
{NO, Jacobi (0.67)}		
Solver only	2.71e+02 (1.1e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (5.1e-03)	$< 10^{-10}$
Learned router (ours)	4.83e-03 (1.0e-03)	-
Cost-aware oracle	<i>4.83e-03 (1.0e-03)</i>	-
{NO, GS}		
Solver only	2.24e+02 (1.8e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.37e+01 (1.4e-02)	$< 10^{-10}$
Learned router (ours)	3.76e-03 (8.1e-04)	-
Cost-aware oracle	<i>3.76e-03 (8.1e-04)</i>	-
{NO, SymGS}		
Solver only	1.89e+02 (2.2e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.35e+01 (2.2e-02)	$< 10^{-10}$
Learned router (ours)	2.89e-03 (6.3e-04)	-
Cost-aware oracle	<i>2.89e-03 (6.3e-04)</i>	-
{NO, SOR (1.5)}		
Solver only	1.21e+02 (1.5e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.31e+01 (3.6e-02)	$< 10^{-10}$
Learned router (ours)	5.86e-03 (1.3e-03)	-
Cost-aware oracle	<i>5.86e-03 (1.3e-03)</i>	-

Table 15: Convection–diffusion, 128×128 : five-seed router-training trials (conventions as in Table 9).

[results pending: confirmatory run in progress]

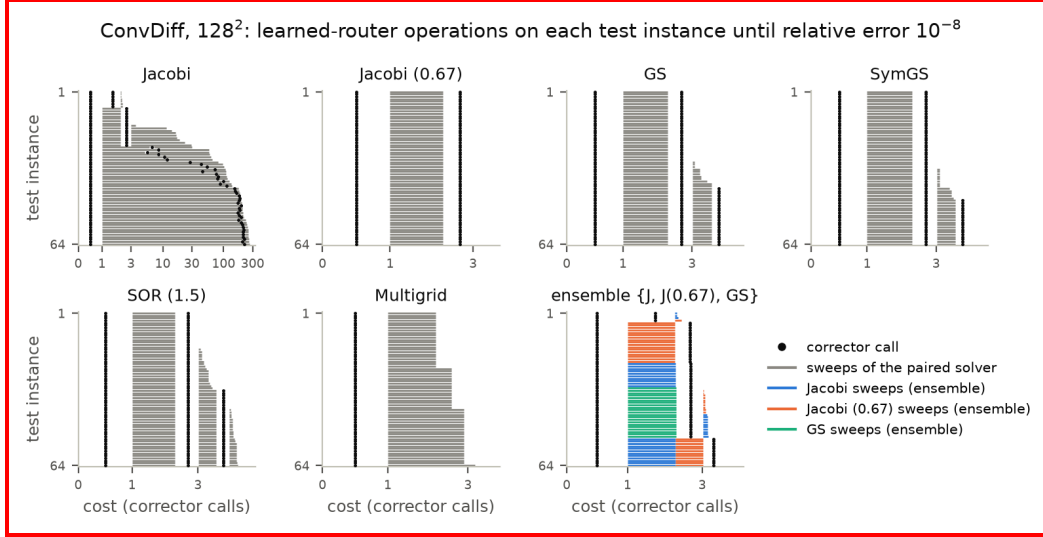


Figure 8: Convection–diffusion, 128×128 : operations of the learned router on each of the 64 test instances (rows, sorted by cost) until the relative error first falls below 10^{-8} , for every pairing and for the ensemble $\{\text{Jacobi}, \text{Jacobi (0.67)}, \text{GS}\}$, as executed in the timed replays. Dots mark corrector calls and lines the solver sweeps between them (coloured by solver in the ensemble); the horizontal axis is the cumulative cost in units of one corrector call, linear below one call and logarithmic above.

1214 E.4 Anisotropic diffusion

1215 The anisotropic equation is where the two components of the hybrid are most complementary and
 1216 where the point smoothers are weakest: they damp the x_1 -oscillatory modes only by $(1 - \epsilon)/(1 + \epsilon) \approx$
 1217 0.98 per sweep (in the first confirmatory run none of the five point smoothers alone reached h^2 within
 1218 the iteration cap on either grid; the revised cells are **[pending]**), whereas the corrector, whose
 1219 sensor grid keeps full resolution along x_1 , removes exactly those modes together with the smooth
 1220 ones in a single call and leaves the smoother only the x_2 -oscillatory band, which it damps at the
 1221 isotropic rate. The corrector’s $N \times N/4$ band is thus the spectral counterpart of semi-coarsening,
 1222 and the honest classical comparison is the method a practitioner would use on this problem: *line*
 1223 Gauss–Seidel along x_2 (which contracts the x_1 -oscillatory modes by 0.35 – 0.45 per sweep, measured
 1224 on the modes $(N/2, 0)$ and $(N/4, 0)$ at 128^2 , where point Jacobi contracts them by 0.98) and line-
 1225 smoothed multigrid, which regains grid-independent convergence (asymptotically ≈ 0.04 per cycle
 1226 on one 128^2 instance, against ≈ 0.86 with point smoothing). Both are included, line GS as the sixth
 1227 pairing $\{\text{NO}, \text{line GS}\}$ and, together with point-smoothed multigrid and the CG variants, as classical
 1228 baselines timed in the same session **[pending]**. For the point-smoother pairings the router reaches h^2
 1229 in the time of one corrector call plus a few sweeps, as does the one-shot schedule, while HINTS with
 1230 $\tau = 25$, the reference of the first confirmatory run (revision 1), was 1.7 – $3.9\times$ slower there; in that
 1231 run the best fixed schedule, which is exactly one call at $t = 0$ followed by sweeps, was 3 – 7% faster
 1232 than the router at h^2 , the cost of the router’s per-step decision. At 10^{-8} the router was significantly
 1233 faster than the best fixed schedule in two of the five point-smoother pairings (Jacobi and damped
 1234 Jacobi) and at parity in the other three, within 4 – 7% of the cost-aware oracle; these numbers are
 1235 being regenerated under the revised protocol **[pending]**, and the five-seed trials for this equation are
 1236 **[pending]** (Table 21). The 512^2 grid is not run: the anisotropic corrector keeps full resolution along
 1237 x_1 ($N \times N/4$ sensors), so at 512^2 its least-squares branch would be a dense $65,536 \times 65,536$ matrix
 1238 (17 GB in single precision), which the current design cannot hold; a coarser x_1 resolution would
 1239 leave the point smoothers modes they cannot damp (Section E.7 discusses exactly this regime).

Table 16: Anisotropic diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 (64 instances, six pairings including line GS) and 256^2 (16 instances) grids for every policy, and for the classical baselines without corrector (conventions as in Table 4; the point-smoother-only runs are capped at 60,000 and 40,000 iterations; censored runs are marked †).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	≥ 1.23 s ^{†64}	≥ 1.23 s ^{†64}	≥ 1.23 s ^{†64}
	HINTS ($\tau=15$)	1.04 ms	1.04 ms	53.1 ms
	Best fixed schedule	715 μ s $\tau=10, t_0=0$	727 μ s $\tau=10, t_0=0$	26.0 ms $\tau=50$
	Best residual-decay rule	721 μ s $\theta=0.98$	732 μ s $\theta=0.9$	32.6 ms $\theta=0.95$
	One-shot schedule	730 μ s	738 μ s	≥ 1.23 s ^{†51}
	Greedy oracle (Alg. 1)	722 μ s	744 μ s	17.2 ms
	Cost-aware oracle	717 μ s	720 μ s	17.5 ms
	Learned router (ours)	759 μ s	764 μ s	17.0 ms
{NO, Jacobi (0.67)}	Solver only	≥ 1.21 s ^{†64}	≥ 1.21 s ^{†64}	≥ 1.21 s ^{†64}
	HINTS ($\tau=15$)	970 μ s	970 μ s	2.00 ms
	Best fixed schedule	668 μ s $\tau=15, t_0=0$	676 μ s $\tau=15, t_0=0$	1.92 ms $\tau=25, t_0=0$
	Best residual-decay rule	665 μ s $\theta=0.9$	679 μ s $\theta=0.9$	2.57 ms $\theta=0.9$
	One-shot schedule	683 μ s	688 μ s	≥ 1.22 s ^{†59}
	Greedy oracle (Alg. 1)	672 μ s	684 μ s	1.89 ms
	Cost-aware oracle	668 μ s	680 μ s	1.97 ms
	Learned router (ours)	695 μ s	700 μ s	2.03 ms
{NO, GS}	Solver only	≥ 3.40 s ^{†64}	≥ 3.40 s ^{†64}	≥ 3.40 s ^{†64}
	HINTS ($\tau=15$)	1.54 ms	1.54 ms	7.06 ms
	Best fixed schedule	665 μ s $\tau=15, t_0=0$	672 μ s $\tau=10, t_0=0$	2.30 ms $\tau=15, t_0=0$
	Best residual-decay rule	669 μ s $\theta=0.98$	674 μ s $\theta=0.98$	3.29 ms $\theta=0.9$
	One-shot schedule	687 μ s	699 μ s	≥ 3.37 s ^{†43}
	Greedy oracle (Alg. 1)	670 μ s	688 μ s	2.95 ms
	Cost-aware oracle	665 μ s	678 μ s	2.33 ms
	Learned router (ours)	695 μ s	710 μ s	2.52 ms
{NO, SymGS}	Solver only	≥ 5.05 s ^{†64}	≥ 5.05 s ^{†64}	≥ 5.05 s ^{†64}
	HINTS ($\tau=15$)	1.98 ms	1.98 ms	3.98 ms
	Best fixed schedule	669 μ s $\tau=50, t_0=0$	677 μ s $\tau=50, t_0=0$	2.24 ms $\tau=10, t_0=0$
	Best residual-decay rule	668 μ s $\theta=0.98$	679 μ s $\theta=0.95$	3.41 ms $\theta=0.9$
	One-shot schedule	683 μ s	700 μ s	≥ 3.21 s ^{†19}
	Greedy oracle (Alg. 1)	666 μ s	678 μ s	2.69 ms
	Cost-aware oracle	669 μ s	683 μ s	2.15 ms
	Learned router (ours)	701 μ s	714 μ s	2.24 ms
{NO, SOR (1.5)}	Solver only	≥ 3.39 s ^{†64}	≥ 3.39 s ^{†64}	≥ 3.39 s ^{†64}
	HINTS ($\tau=15$)	1.53 ms	1.53 ms	4.79 ms
	Best fixed schedule	656 μ s $\tau=10, t_0=0$	663 μ s $\tau=15, t_0=0$	2.67 ms $\tau=15, t_0=0$
	Best residual-decay rule	658 μ s $\theta=0.95$	664 μ s $\theta=0.95$	3.56 ms $\theta=0.9$
	One-shot schedule	667 μ s	678 μ s	≥ 1.51 s ^{†6}
	Greedy oracle (Alg. 1)	658 μ s	663 μ s	3.38 ms
	Cost-aware oracle	656 μ s	664 μ s	2.68 ms
	Learned router (ours)	687 μ s	690 μ s	2.95 ms
{NO, Line GS}	Solver only	248.6 ms	349.2 ms	662.9 ms
	HINTS ($\tau=15$)	2.00 ms	2.00 ms	3.99 ms
	Best fixed schedule	652 μ s $\tau=10, t_0=0$	656 μ s $\tau=10, t_0=0$	1.67 ms $\tau=5, t_0=0$
	Best residual-decay rule	655 μ s $\theta=0.9$	658 μ s $\theta=0.95$	3.33 ms $\theta=0.9$
	One-shot schedule	656 μ s	660 μ s	48.5 ms
	Greedy oracle (Alg. 1)	654 μ s	657 μ s	1.48 ms
	Cost-aware oracle	652 μ s	657 μ s	1.96 ms
	Learned router (ours)	682 μ s	686 μ s	2.03 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone (point GS)	2.70 ms	8.19 ms	40.1 ms
	Multigrid V(2,2) alone (line GS)	1.05 ms	1.06 ms	2.55 ms
	CG	17.6 ms	20.0 ms	25.7 ms
	PCG (SymGS)	23.5 ms	26.6 ms	49.8 ms
	PCG (multigrid)	1.05 ms	2.08 ms	6.95 ms
	FFT direct solve (exact; constant coefficients only)	245 μ s	245 μ s	245 μ s

Table 17: Anisotropic diffusion: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$) and the best fixed period, with two-sided Wilcoxon p -values and bootstrap intervals; the classical baselines (point- and line-smoothed multigrid, CG variants) are timed in the same session.

Pairing	ε	128 × 128									
		vs. HINTS-15	vs. best schedule	vs. best decay rule	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. FFT solve			
{NO, Jacobi}	h^2	1.97 × [1.24, 3.26] ($< 10^{-10}$)	0.95 × [0.94, 0.97] (down, 7×10^{-10})	0.95 × [0.94, 0.97] (down, 7×10^{-10})	0.97 × [0.95, 0.99] (down, 2×10^{-10})	11.1 × [9.78, 13.4] ($< 10^{-10}$)	2.88 × [2.44, 3.52] ($< 10^{-10}$)	0.32 × [0.31, 0.33] (down, $< 10^{-10}$)			
	10^{-8}	2.91 × [1.43, 5.86] ($< 10^{-10}$)	1.69 × [1.65, 1.73] ($< 10^{-10}$)	1.90 × [1.75, 2.05] ($< 10^{-10}$)	72.2 × [12.5, 440] ($< 10^{-10}$)	2.27 × [1.15, 44.0] ($< 10^{-10}$)	0.41 × [0.20, 2.03] (down, 0.006)	0.01 × [0.01, 0.11] (down, $< 10^{-10}$)			
{NO, Jacobi (0.67)}	h^2	1.38 × [1.36, 1.40] ($< 10^{-10}$)	0.95 × [0.95, 0.96] (down, 2×10^{-10})	0.96 × [0.95, 0.96] (down, 8×10^{-10})	0.97 × [0.96, 0.98] (down, 9×10^{-10})	11.0 × [10.1, 14.0] ($< 10^{-10}$)	2.96 × [2.46, 3.67] ($< 10^{-10}$)	0.33 × [0.32, 0.33] (down, $< 10^{-10}$)			
	10^{-8}	1.61 × [0.95, 1.15] (0.000)	0.91 × [0.90, 1.00] (down, 0.030)	1.25 × [1.22, 1.27] ($< 10^{-10}$)	3.98 × [100, 630] ($< 10^{-10}$)	26.5 × [20.1, 20.6] ($< 10^{-10}$)	3.46 × [3.43, 3.50] ($< 10^{-10}$)	0.12 × [0.11, 0.12] (down, $< 10^{-10}$)			
{NO, GS}	h^2	2.18 × [2.15, 2.21] ($< 10^{-10}$)	0.95 × [0.95, 0.96] (down, $< 10^{-10}$)	0.96 × [0.95, 0.97] (down, 2×10^{-10})	0.98 × [0.96, 0.99] (down, 0.010)	11.0 × [9.78, 13.2] ($< 10^{-10}$)	2.93 × [2.50, 3.06] ($< 10^{-10}$)	0.33 × [0.32, 0.34] (down, $< 10^{-10}$)			
	10^{-8}	2.91 × [2.62, 3.18] ($< 10^{-10}$)	0.99 × [0.97, 1.03] (down, 0.374)	1.36 × [1.28, 1.39] ($< 10^{-10}$)	3.24 × [109, 1310] ($< 10^{-10}$)	15.9 × [15.2, 16.1] ($< 10^{-10}$)	2.76 × [2.66, 2.86] ($< 10^{-10}$)	0.10 × [0.09, 0.11] (down, $< 10^{-10}$)			
{NO, SymGS}	h^2	2.72 × [2.69, 2.79] ($< 10^{-10}$)	0.95 × [0.94, 0.96] (down, 5×10^{-10})	0.95 × [0.94, 0.96] (down, 10^{-10})	0.96 × [0.95, 0.96] (down, 10^{-10})	18.5 × [16.7, 13.3] ($< 10^{-10}$)	2.88 × [2.42, 3.30] ($< 10^{-10}$)	0.32 × [0.31, 0.33] (down, $< 10^{-10}$)			
	10^{-8}	1.81 × [1.74, 1.87] ($< 10^{-10}$)	1.02 × [1.00, 1.04] (0.031)	1.48 × [1.41, 1.54] ($< 10^{-10}$)	3.170 × [1270, 1432] ($< 10^{-10}$)	17.1 × [16.7, 17.6] ($< 10^{-10}$)	2.96 × [2.84, 3.06] ($< 10^{-10}$)	0.10 × [0.10, 0.11] (down, $< 10^{-10}$)			
{NO, SOR (1.5)}	h^2	2.20 × [2.16, 2.22] ($< 10^{-10}$)	0.96 × [0.95, 0.96] (down, 2×10^{-10})	0.96 × [0.95, 0.97] (down, $< 10^{-10}$)	0.97 × [0.96, 0.98] (down, 3×10^{-10})	10.8 × [9.93, 13.4] ($< 10^{-10}$)	2.89 × [2.56, 3.24] ($< 10^{-10}$)	0.33 × [0.32, 0.33] (down, $< 10^{-10}$)			
	10^{-8}	1.68 × [1.55, 1.78] ($< 10^{-10}$)	0.95 × [0.91, 0.98] (down, 0.055)	1.28 × [1.17, 1.33] ($< 10^{-10}$)	3.597 × [497, 633] ($< 10^{-10}$)	12.7 × [11.7, 13.6] ($< 10^{-10}$)	2.26 × [2.03, 2.44] ($< 10^{-10}$)	0.08 × [0.07, 0.09] (down, $< 10^{-10}$)			
{NO, Line GS}	h^2	2.81 × [2.75, 2.86] ($< 10^{-10}$)	0.95 × [0.94, 0.96] (down, $< 10^{-10}$)	0.95 × [0.95, 0.96] (down, $< 10^{-10}$)	0.95 × [0.94, 0.97] (down, 10^{-10})	11.2 × [10.1, 13.6] ($< 10^{-10}$)	3.00 × [2.47, 3.67] ($< 10^{-10}$)	0.32 × [0.32, 0.33] (down, $< 10^{-10}$)			
	10^{-8}	1.96 × [1.95, 1.98] ($< 10^{-10}$)	0.83 × [0.82, 0.83] (down, $< 10^{-10}$)	1.65 × [1.64, 1.67] ($< 10^{-10}$)	24.0 × [18.2, 26.7] ($< 10^{-10}$)	19.0 × [17.1, 20.8] ($< 10^{-10}$)	3.29 × [3.03, 3.55] ($< 10^{-10}$)	0.11 × [0.11, 0.11] (down, $< 10^{-10}$)			

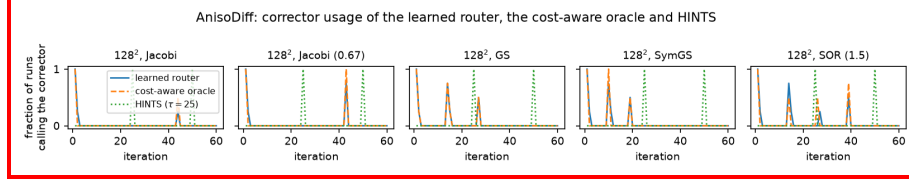


Figure 9: Anisotropic diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=15$) for every pairing (columns) and grid (rows).

Table 18: Anisotropic diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

Pairing	128 × 128		
	HINTS-15	oracle	router
{NO, Jacobi}	15 (1)	1 (1)	1 (1)
{NO, Jacobi (0.67)}	15 (1)	1 (1)	1 (1)
{NO, GS}	15 (1)	1 (1)	1 (1)
{NO, SymGS}	15 (1)	1 (1)	1 (1)
{NO, SOR (1.5)}	15 (1)	1 (1)	1 (1)
{NO, Line GS}	15 (1)	1 (1)	1 (1)

Table 19: Anisotropic diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	21 μs	628 μs	30
{NO, Jacobi (0.67)}	21 μs	628 μs	30
{NO, GS}	54 μs	628 μs	12
{NO, SymGS}	82 μs	628 μs	8
{NO, SOR (1.5)}	54 μs	628 μs	12
{NO, Line GS}	84 μs	628 μs	7

Table 20: Anisotropic diffusion, 128×128 : iteration-based metrics (conventions as in Table 8).

Method	AUC	p
{NO, Jacobi}		
Solver only	2.99e+02 (4.6e-02)	$< 10^{-10}$
HINTS ($\tau=15$)	1.40e+01 (1.3e-04)	$< 10^{-10}$
Learned router (ours)	7.15e-04 (2.0e-04)	-
Cost-aware oracle	7.16e-04 (2.0e-04)	-
{NO, Jacobi (0.67)}		
Solver only	3.00e+02 (3.2e-02)	$< 10^{-10}$
HINTS ($\tau=15$)	1.40e+01 (8.1e-05)	$< 10^{-10}$
Learned router (ours)	2.94e-04 (7.6e-05)	-
Cost-aware oracle	2.95e-04 (7.6e-05)	-
{NO, GS}		
Solver only	2.99e+02 (8.4e-02)	$< 10^{-10}$
HINTS ($\tau=15$)	1.40e+01 (2.4e-04)	$< 10^{-10}$
Learned router (ours)	2.04e-04 (5.4e-05)	-
Cost-aware oracle	2.05e-04 (5.4e-05)	-
{NO, SymGS}		
Solver only	2.97e+02 (1.5e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.40e+01 (4.6e-04)	$< 10^{-10}$
Learned router (ours)	1.31e-04 (3.4e-05)	-
Cost-aware oracle	1.31e-04 (3.4e-05)	-
{NO, SOR (1.5)}		
Solver only	2.96e+02 (2.1e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.40e+01 (6.7e-04)	$< 10^{-10}$
Learned router (ours)	2.97e-04 (7.9e-05)	-
Cost-aware oracle	3.01e-04 (8.0e-05)	-
{NO, Line GS}		
Solver only	2.09e+02 (1.4e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.37e+01 (1.4e-02)	$< 10^{-10}$
Learned router (ours)	7.84e-05 (2.1e-05)	-
Cost-aware oracle	7.82e-05 (2.1e-05)	-

Table 21: Anisotropic diffusion, 128×128 : five-seed router-training trials (conventions as in Table 9).

[results pending: confirmatory run in progress]

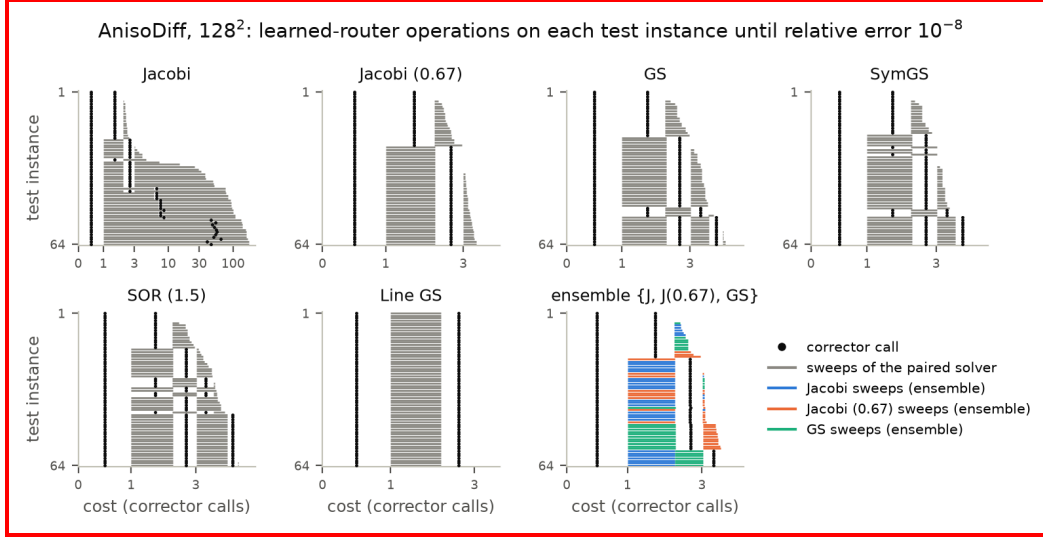


Figure 10: Anisotropic diffusion, 128×128 : operations of the learned router on each of the 64 test instances (rows, sorted by cost) until the relative error first falls below 10^{-8} , for every pairing and for the ensemble $\{\text{Jacobi}, \text{Jacobi (0.67)}, \text{GS}\}$, as executed in the timed replays. Dots mark corrector calls and lines the solver sweeps between them (coloured by solver in the ensemble); the horizontal axis is the cumulative cost in units of one corrector call, linear below one call and logarithmic above.

1240 E.5 Variable-coefficient diffusion

1241 The three equations above have constant coefficients on a periodic domain, so their discrete operators
1242 are diagonalised by the FFT: the exact band-limited inverse that the corrector approximates has
1243 a closed form, and a direct FFT solve is available, which at 128^2 takes $1/3.25 \times -1/3.08 \times$ of
1244 the best router's time (Tables 4 and 10). To evaluate the hybrid where neither exists we add
1245 $-\nabla \cdot (a(\mathbf{x}) \nabla u) = f$ on the same periodic domain with a fixed, smooth, positive coefficient field
1246 $a(\mathbf{x}) = \exp(\kappa z(\mathbf{x}))$, where z is a band-limited random field ($|k| \leq 4$, one fixed realisation, scaled by
1247 its maximum on a fixed 1024^2 reference grid, so that a is one grid-independent function sampled
1248 on every grid) and $\kappa = \frac{1}{2} \ln 10$, so that the contrast $\max a / \min a$ is at most 10 by construction
1249 (9.99 measured on the grids used); the forcing distribution is unchanged. The discretisation is
1250 the conservative 5-point scheme with face-averaged coefficients (symmetric positive semidefinite,
1251 constants in its null space); reference solutions come from a sparse LU factorisation with one pinned
1252 unknown, which is also reported as a direct-solve baseline (its solve time with the factorisation
1253 cached, the factorisation itself costing ≈ 0.5 s at 128^2). Geometric multigrid rediscratises the coarse
1254 operators with block-averaged coefficients and contracts the error by ≈ 0.1 per $V(2,2)$ -cycle. The
1255 corrector is the same least-squares DeepONet as before—a linear map on the 64×64 band—but fitted
1256 to (residual, error) pairs of *this* operator, whose band-limited inverse $R(\mathcal{L}_h)^{-1}P$ is not diagonal in
1257 the Fourier basis and has no closed form. It is nevertheless a fixed 4096×3969 matrix that could be
1258 assembled directly from 4096 sparse solves, one per sensor basis function; the least-squares fit on
1259 random pairs recovers it to the same $\approx 10^{-6}$ as on the constant-coefficient equations. What changes
1260 is that this matrix is no longer the inverse of the band-restricted error: a variable coefficient couples
1261 the band to the modes outside it, so an out-of-band residual leaves band error that the corrector does
1262 not see, and a band correction leaves out-of-band error behind. That coupling, not fit error, is why the
1263 corrector is called repeatedly along a solve and why the router's decisions matter beyond the first call;
1264 what this experiment tests is routing on an operator without Fourier structure (no shared eigenbasis
1265 with the smoothers), not that the correction must be learned. The coefficient field is fixed rather than
1266 sampled, so this experiment tests a corrector learned for one operator without an analytic inverse, not
1267 generalisation across coefficient fields; the latter would require the coefficient-conditioned branch
1268 of the original DeepONet and is left to future work. Tables 22 to 27 and fig. 11 report the same
1269 quantities as for the other equations on the 128^2 and 256^2 grids.

Table 22: Variable-coefficient diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 and 256^2 grids for every pairing and policy, and for the classical baselines without corrector, including the sparse direct solve (conventions as in Table 4).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	453.9 ms	654.4 ms	1.28 s
	HINTS ($\tau=15$)	1.24 ms	1.24 ms	179.2 ms
	Best fixed schedule	706 $\mu s_{\tau=25, t_0=0}$	730 $\mu s_{\tau=5, t_0=0}$	103.0 ms _{one-shot}
	Best residual-decay rule	709 $\mu s_{\theta=0.95}$	728 $\mu s_{\theta=0.95}$	149.8 ms _{$\theta=0.9$}
	One-shot schedule	719 μs	732 μs	103.0 ms
	Greedy oracle (Alg. 1)	706 μs	728 μs	78.7 ms
	Cost-aware oracle	707 μs	742 μs	79.0 ms
	Learned router (ours)	743 μs	784 μs	79.4 ms
{NO, Jacobi (0.67)}	Solver only	668.1 ms	957.4 ms	1.86 s
	HINTS ($\tau=15$)	1.20 ms	1.20 ms	2.37 ms
	Best fixed schedule	677 $\mu s_{\tau=5, t_0=0}$	705 $\mu s_{\tau=50, t_0=0}$	2.04 ms _{$\tau=10$}
	Best residual-decay rule	676 $\mu s_{\theta=0.98}$	708 $\mu s_{\theta=0.95}$	2.92 ms _{$\theta=0.9$}
	One-shot schedule	678 μs	727 μs	102.4 ms
	Greedy oracle (Alg. 1)	680 μs	723 μs	2.05 ms
	Cost-aware oracle	679 μs	734 μs	2.13 ms
	Learned router (ours)	709 μs	752 μs	2.20 ms
{NO, GS}	Solver only	722.9 ms	1.03 s	2.01 s
	HINTS ($\tau=15$)	2.47 ms	2.58 ms	7.33 ms
	Best fixed schedule	674 $\mu s_{\text{one-shot}}$	742 $\mu s_{\tau=50, t_0=0}$	2.76 ms _{$\tau=10, t_0=0$}
	Best residual-decay rule	672 $\mu s_{\theta=0.98}$	751 $\mu s_{\theta=0.98}$	4.47 ms _{$\theta=0.9$}
	One-shot schedule	674 μs	756 μs	255.5 ms
	Greedy oracle (Alg. 1)	674 μs	740 μs	3.34 ms
	Cost-aware oracle	679 μs	772 μs	3.09 ms
	Learned router (ours)	699 μs	792 μs	3.14 ms
{NO, SymGS}	Solver only	581.8 ms	833.8 ms	1.62 s
	HINTS ($\tau=15$)	3.49 ms	3.59 ms	7.12 ms
	Best fixed schedule	676 $\mu s_{\tau=10, t_0=0}$	780 $\mu s_{\text{one-shot}}$	2.56 ms _{$\tau=5, t_0=0$}
	Best residual-decay rule	674 $\mu s_{\theta=0.98}$	778 $\mu s_{\theta=0.98}$	5.49 ms _{$\theta=0.9$}
	One-shot schedule	682 μs	780 μs	182.3 ms
	Greedy oracle (Alg. 1)	681 μs	789 μs	2.55 ms
	Cost-aware oracle	680 μs	804 μs	2.56 ms
	Learned router (ours)	703 μs	831 μs	2.57 ms
{NO, SOR (1.5)}	Solver only	243.4 ms	347.3 ms	674.2 ms
	HINTS ($\tau=15$)	2.44 ms	4.90 ms	7.33 ms
	Best fixed schedule	673 $\mu s_{\tau=15, t_0=0}$	798 $\mu s_{\tau=15, t_0=0}$	4.28 ms _{$\tau=15, t_0=0$}
	Best residual-decay rule	677 $\mu s_{\theta=0.98}$	808 $\mu s_{\theta=0.95}$	5.97 ms _{$\theta=0.9$}
	One-shot schedule	675 μs	820 μs	119.4 ms
	Greedy oracle (Alg. 1)	682 μs	806 μs	4.62 ms
	Cost-aware oracle	677 μs	805 μs	4.29 ms
	Learned router (ours)	708 μs	848 μs	4.40 ms
{NO, Multigrid}	Solver only	1.94 ms	2.56 ms	4.46 ms
	HINTS ($\tau=15$)	1.92 ms	2.53 ms	4.46 ms
	Best fixed schedule	681 $\mu s_{\tau=10, t_0=0}$	989 $\mu s_{\tau=10, t_0=0}$	2.65 ms _{one-shot}
	Best residual-decay rule	679 $\mu s_{\theta=0.98}$	1.01 ms _{$\theta=0.95$}	2.66 ms _{$\theta=0.95$}
	One-shot schedule	681 μs	998 μs	2.65 ms
	Greedy oracle (Alg. 1)	685 μs	1.00 ms	2.66 ms
	Cost-aware oracle	682 μs	1.03 ms	2.65 ms
	Learned router (ours)	707 μs	1.06 ms	2.73 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone (point GS)	2.01 ms	2.66 ms	4.61 ms
	CG	13.9 ms	19.7 ms	35.8 ms
	PCG (SymGS)	17.2 ms	24.7 ms	41.3 ms
	PCG (multigrid)	1.36 ms	2.01 ms	3.91 ms
	Sparse LU direct solve (cached factorisation)	932 μs	932 μs	932 μs

Table 23: Variable-coefficient diffusion: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$), the best fixed schedule, the one-shot schedule, multigrid alone and multigrid-preconditioned CG, with Holm-corrected two-sided Wilcoxon significance (conventions as in Table 5).

Pairing	ε	128 × 128					
		vs. HINTS-15	vs. best schedule	vs. best decay rule	vs. one-shot	vs. multigrid	vs. MG-Krylov
{NO, Jacobi}	h^2	1.65 × [1.60, 1.71] ($< 10^{-10}$)	0.99 × [0.97, 1.03] (downer, 0.179)	0.96 × [0.94, 0.98] (downer, 0.016)	0.97 × [0.95, 0.98] (downer, 2×10^{-3})	3.26 × [2.93, 3.50] (2×10^{-8})	2.55 × [2.32, 2.65] (7×10^{-5})
	10^{-8}	2.19 × [1.98, 2.22] ($< 10^{-10}$)	1.19 × [1.05, 2.75] (5×10^{-10})	1.82 × [1.76, 1.86] ($< 10^{-10}$)	1.19 × [1.05, 2.75] (5×10^{-10})	0.05 × [0.03, 0.16] (downer, 5×10^{-4})	0.05 × [0.02, 0.14] (downer, 10^{-4})
{NO, Jacobi (0.67)}	h^2	1.59 × [1.39, 1.60] ($< 10^{-10}$)	0.95 × [0.94, 0.96] (downer, 2×10^{-10})	0.95 × [0.95, 0.96] (downer, $< 10^{-10}$)	0.95 × [0.94, 0.96] (downer, $< 10^{-10}$)	3.40 × [2.97, 3.87] ($< 10^{-10}$)	2.57 × [2.29, 3.72] ($< 10^{-10}$)
	10^{-8}	1.08 × [1.05, 1.08] (10^{-9})	0.94 × [0.93, 0.96] (downer, 8×10^{-4})	1.33 × [1.32, 1.35] ($< 10^{-10}$)	45.5 × [40.4, 52.8] ($< 10^{-10}$)	2.04 × [2.01, 2.06] ($< 10^{-10}$)	1.73 × [1.70, 1.76] ($< 10^{-10}$)
{NO, GS}	h^2	3.19 × [2.51, 3.56] ($< 10^{-10}$)	0.96 × [0.96, 0.97] (downer, 3×10^{-4})	0.96 × [0.95, 0.96] (downer, 3×10^{-4})	0.96 × [0.95, 0.96] (downer, 3×10^{-4})	2.94 × [2.51, 3.40] ($< 10^{-10}$)	2.44 × [1.88, 2.66] ($< 10^{-10}$)
	10^{-8}	2.21 × [1.92, 2.76] ($< 10^{-10}$)	1.03 × [1.00, 1.06] (0.039)	1.67 × [1.57, 1.72] ($< 10^{-10}$)	78.8 × [59.7, 104] ($< 10^{-10}$)	1.44 × [1.19, 1.72] ($< 10^{-10}$)	1.19 × [1.00, 1.49] (9×10^{-4})
{NO, SymGS}	h^2	4.32 × [2.57, 4.83] ($< 10^{-10}$)	0.96 × [0.95, 0.97] (downer, 2×10^{-4})	0.96 × [0.94, 0.97] (downer, 5×10^{-4})	0.96 × [0.95, 0.97] (downer, 2×10^{-4})	2.83 × [2.51, 3.42] ($< 10^{-10}$)	2.25 × [1.88, 2.66] ($< 10^{-10}$)
	10^{-8}	2.78 × [2.26, 3.01] ($< 10^{-10}$)	0.98 × [0.97, 0.99] (downer, 3×10^{-4})	2.07 × [1.93, 2.18] ($< 10^{-10}$)	60.6 × [45.1, 80.8] ($< 10^{-10}$)	1.69 × [1.46, 1.85] ($< 10^{-10}$)	1.42 × [1.18, 1.61] (7×10^{-4})
{NO, SOR (1.5)}	h^2	5.11 × [3.92, 6.66] ($< 10^{-10}$)	0.96 × [0.95, 0.97] (downer, 10^{-4})	0.97 × [0.96, 0.98] (downer, 2×10^{-4})	0.96 × [0.95, 0.97] (downer, 3×10^{-4})	2.73 × [2.06, 3.40] ($< 10^{-10}$)	2.24 × [1.48, 2.66] (2×10^{-4})
	10^{-8}	1.66 × [1.25, 1.80] ($< 10^{-10}$)	0.97 × [0.97, 0.99] (downer, 0.28)	1.42 × [1.36, 1.51] ($< 10^{-10}$)	26.9 × [18.7, 32.3] ($< 10^{-10}$)	1.03 × [0.83, 1.14] (0.446)	0.84 × [0.64, 1.01] (downer, 0.017)
{NO, Multigrid}	h^2	2.45 × [1.87, 3.57] ($< 10^{-10}$)	0.97 × [0.96, 0.97] (downer, $< 10^{-10}$)	0.97 × [0.96, 0.98] (downer, 10^{-10})	0.96 × [0.95, 0.97] (downer, $< 10^{-10}$)	2.30 × [1.83, 3.55] ($< 10^{-10}$)	1.97 × [1.38, 2.65] ($< 10^{-10}$)
	10^{-8}	1.62 × [1.57, 1.82] ($< 10^{-10}$)	0.97 × [0.97, 0.98] (downer, 4×10^{-10})	0.98 × [0.97, 0.98] (downer, 3×10^{-10})	0.97 × [0.97, 0.98] (downer, 4×10^{-10})	1.63 × [1.56, 1.76] ($< 10^{-10}$)	1.38 × [1.25, 1.43] ($< 10^{-10}$)

(figure pending)

Figure 11: Variable-coefficient diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=15$) for every pairing (columns) and grid (rows).

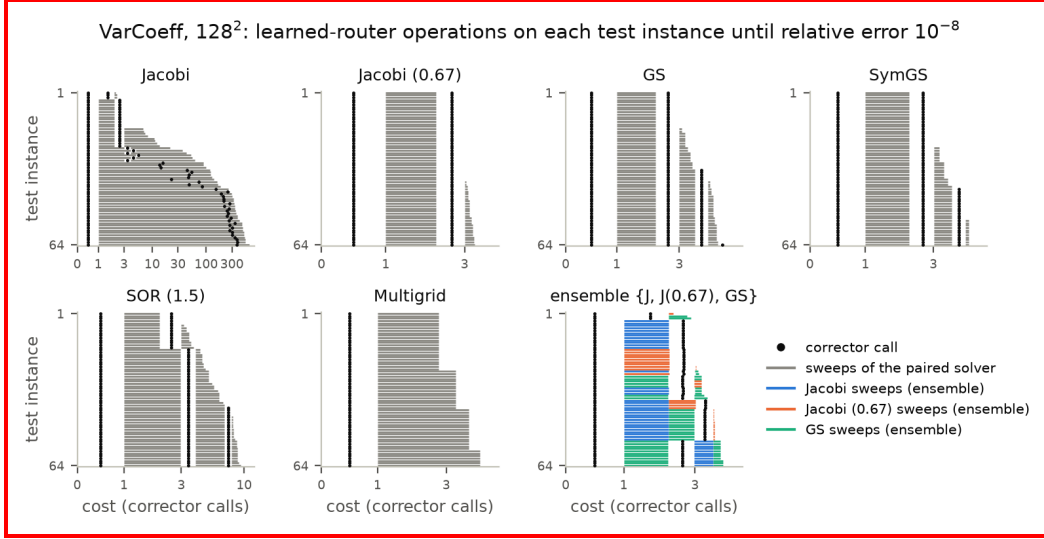


Figure 12: Variable-coefficient diffusion, 128×128 : operations of the learned router on each of the 64 test instances (rows, sorted by cost) until the relative error first falls below 10^{-8} , for every pairing and for the ensemble $\{\text{Jacobi}, \text{Jacobi (0.67)}, \text{GS}\}$, as executed in the timed replays. Dots mark corrector calls and lines the solver sweeps between them (coloured by solver in the ensemble); the horizontal axis is the cumulative cost in units of one corrector call, linear below one call and logarithmic above.

Table 24: Variable-coefficient diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

Pairing	128×128		
	HINTS-15	oracle	router
$\{\text{NO}, \text{Jacobi}\}$	15 (1)	2 (1)	2 (1)
$\{\text{NO}, \text{Jacobi (0.67)}\}$	15 (1)	2 (1)	2 (1)
$\{\text{NO}, \text{GS}\}$	16 (1)	2 (1)	2 (1)
$\{\text{NO}, \text{SymGS}\}$	16 (1)	2 (1)	2 (1)
$\{\text{NO}, \text{SOR (1.5)}\}$	30 (2)	2 (1)	2 (1)
$\{\text{NO}, \text{Multigrid}\}$	4 (0)	2 (1)	2 (1)

Table 25: Variable-coefficient diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128×128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	37 μs	749 μs	20
{NO, Jacobi (0.67)}	36 μs	749 μs	21
{NO, GS}	121 μs	749 μs	6
{NO, SymGS}	192 μs	749 μs	4
{NO, SOR (1.5)}	122 μs	749 μs	6
{NO, Multigrid}	619 μs	749 μs	1

Table 26: Variable-coefficient diffusion, 128×128 : iteration-based metrics (conventions as in Table 8).

Method	AUC	p
{NO, Jacobi}		
Solver only	2.66e+02 (9.5e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (3.7e-03)	$< 10^{-10}$
Learned router (ours)	6.91e-03 (1.7e-03)	-
Cost-aware oracle	6.91e-03 (1.7e-03)	-
{NO, Jacobi (0.67)}		
Solver only	2.76e+02 (7.5e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (3.1e-03)	$< 10^{-10}$
Learned router (ours)	2.49e-03 (5.9e-04)	-
Cost-aware oracle	2.49e-03 (5.9e-04)	-
{NO, GS}		
Solver only	2.40e+02 (1.3e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.38e+01 (8.1e-03)	$< 10^{-10}$
Learned router (ours)	1.93e-03 (4.6e-04)	-
Cost-aware oracle	1.93e-03 (4.6e-04)	-
{NO, SymGS}		
Solver only	2.01e+02 (1.6e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.36e+01 (1.4e-02)	$< 10^{-10}$
Learned router (ours)	1.45e-03 (3.5e-04)	-
Cost-aware oracle	1.45e-03 (3.5e-04)	-
{NO, SOR (1.5)}		
Solver only	1.71e+02 (1.7e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.35e+01 (1.9e-02)	$< 10^{-10}$
Learned router (ours)	3.27e-03 (7.9e-04)	-
Cost-aware oracle	3.27e-03 (7.9e-04)	-

Table 27: Variable-coefficient diffusion: five-seed router-training trials (conventions as in Table 9).

[results pending: confirmatory run in progress]

1270 E.6 Decision granularity

1271 The cost-aware router decides once per macro-action, and the macro-action of a cheap solver is as
 1272 expensive as one corrector call—32 Jacobi or 11 Gauss–Seidel sweeps at 128^2 with the compiled
 1273 kernels. Below truncation level the best fixed schedules at 10^{-8} are $\tau=15$ (first call at 0) (9), one-shot
 1274 (4), $\tau=5$ (first call at 0) (4), $\tau=10$ (first call at 0) (3), $\tau=25$ (first call at 0) (2), $\tau=50$ (1), $\tau=10$

(1) (pairings in parentheses), the periodic ones among which call the corrector more often than a cost-equalised macro-action of 11 Gauss–Seidel or 32 Jacobi sweeps allows the router to, a plausible cause of the router’s losses in several pairings at 10^{-8} (Tables 5, 11 and 17). The granularity is a free parameter of the construction of Section 4: the unit of cost can be any fraction of a corrector call, actions dearer than the unit (here the corrector itself) being compared per unit of cost. Table 28 repeats the 128^2 study with the unit set to one quarter of a corrector call (about a quarter of those sweeps per decision; the corrector is then the one action dearer than the unit and is scored per unit of cost), routers retrained with the same recipe, on the same confirmatory instances, for every pairing and for the nested two-member Jacobi ensemble.

Table 28: Decision granularity at 128×128 : median time to h^2 and 10^{-8} of the cost-aware oracle and the learned router with the unit of cost equal to one corrector call (main results) and to one quarter of it, and, in both cases, the router’s paired work-unit ratio at 10^{-8} against the main run’s best fixed schedule on the same instances (the ablation runs a reduced schedule family and reuses the main run’s fixed-schedule work units).

[results pending: confirmatory run in progress]

E.7 Solver ensembles

When can a second classical solver help? The cost-aware oracle is a per-decision comparison of macro-actions, so a router over $\text{NO} \cup \mathcal{W}$ can only beat the best pairwise router of its members if, along the way to the tolerance, the error passes through regimes in which *different* members give the largest reduction per unit cost. We screened this ceiling exhaustively before training any router: for every subset $\mathcal{W}$ of up to three members of {Jacobi, Jacobi (0.67), GS, SymGS, SOR (1.5), multigrid} (multigrid on the isotropic equations only), every equation and grid, we ran the cost-aware oracle over $\text{NO} \cup \mathcal{W}$ on the test instances and compared its work-unit time to that of the oracle over each member alone (Table 29). Two facts emerged in the development screening ([**provisional**]). At $\varepsilon = h^2$ no ensemble beat the best single solver anywhere: one corrector call followed by one or two macro-actions of the cheapest smoother already reaches truncation-level accuracy, so there is no second regime to exploit. At $\varepsilon = 10^{-8}$, after the corrector has removed the smooth band, the smoother has to damp the whole band $|k| > 31$, whose sub-bands favour different solvers: undamped Jacobi is the cheapest per unit cost on the modes near the band edge (it contracts them by $\cos(2\pi \cdot 32/N)$ -like factors that damped Jacobi cannot match), but it does not damp the near-Nyquist modes at all, which damped Jacobi and GS remove in a few sweeps. In the development screening ([**provisional**]: Table 29 is computed with the development-run macro-action sizes; the confirmatory screening with the compiled kernels’ sizes is pending) the oracle over {Jacobi, Jacobi (0.67)}—a *nonstationary* Jacobi iteration whose damping parameter is chosen per macro-action—was therefore faster than every single solver, by 13–28% on four of the six (equation, grid) settings at 128^2 and 256^2 , and adding a third member (GS) changed nothing, i.e., the gain saturates. Whether this ceiling survives the coarser macro-actions of the compiled kernels is exactly what the nested-ensemble runs below measure. At 512^2 (development screening) multigrid dominated every subset (its cycle removes all sub-bands at once), so the oracle over any ensemble containing it equalled the oracle over {NO, multigrid}: adding members neither helped nor hurt. The “difficulty” that matters for ensembles is thus not the equation but the width of the spectrum left to the classical members after the corrector, which grows with the grid (fixed band) and with the tolerance.

Nested ensembles. Table 30 evaluates the learned router over the nested sets {Jacobi}, {Jacobi (0.67)}, {Jacobi, Jacobi (0.67)} and {Jacobi, Jacobi (0.67), GS} on every equation at 128^2 and 256^2 , and at 128^2 also over the four ensembles of Section 7 ({Jacobi, GS} up to all five solvers); the single-member routers are the pairwise routers of Sections E.2 to E.4, evaluated in the same session on the same instances, and all members keep the macro-action sizes of their pairwise runs. The table reports live and work-unit times to h^2 , 10^{-6} and 10^{-8} for the router and its oracle, and the paired speedup at 10^{-8} over the best single-member router (and of the ensemble oracle over the best single-member oracle) with its p -value; work units are the primary measure, since they are determined by the decision sequence alone. All cells of this table are [**pending**] under the revised protocol; the description below is of the first confirmatory run (revision 1, 39-sweep Jacobi macro-actions), and the macros will report the revised values. *Separation*: at $\varepsilon = 10^{-8}$ the ensemble router is faster than

every single-member router by at least 10% and significantly so (bold in the table) in 0 of the 20 ensemble cells, and the ensemble *oracle* is $0.95 \times - 1.02 \times$ of the best single-member oracle in work units (in the first run: no wins, and oracles within 6%, i.e., the greedy rule itself found no second regime to exploit). That differed from the development runs, in which the two-member Jacobi set was 8–15% faster than damped Jacobi alone on convection–diffusion and anisotropic diffusion (Table 33), and the difference is plausibly the decision granularity. With the numpy sweeps a Jacobi macro-action was about ten sweeps, and the oracle alternated undamped Jacobi—the cheapest member on the band-edge modes $|k| \approx 32$, which it contracts by about 0.5 per sweep at 128^2 but which leaves the Nyquist modes untouched—with damped Jacobi, which removes them. With the compiled sweeps a damped-Jacobi macro-action is 32 sweeps: it contracts the band-edge modes by $0.665^{32} \approx 2 \times 10^{-6}$ and the Nyquist modes by $0.34^{32} \approx 1 \times 10^{-15}$ (single-mode factors at 128^2), which suggests that the band the corrector leaves is removed within one or two macro-actions, leaving a nonstationary schedule little to gain; the granularity ablation tests this hypothesis. Section E.6 repeats these cells with a four times finer unit and tests directly whether the gain returns **[pending]**; the 256^2 grids, where the band left to the smoothers is twice as wide and needs several macro-actions, are the other place it could. *Saturation*: in the first run, adding GS to the Jacobi pair changed the oracle’s schedule by at most a few percent, because GS never won a cost-aware comparison against the Jacobi pair. *Monotonicity*: the ensemble routers’ ratios against the best single-member router in work units are $1.00 \times - 1.00 \times$ at h^2 and $0.74 \times - 1.00 \times$ at 10^{-8} , with 9 of the 20 cells more than 5% slower (in the first run these were sets containing both GS and SymGS, where the router’s imitation of its oracle degraded among near-tied members while the oracles themselves stayed within 6% of the best single oracle). A greedy rule is not monotone in its action set in general, and its learned imitation is less so; the table reports how far the losses go.

Table 29: Oracle-level screening of ensembles (**[provisional: development-run costs and instances]**): for every equation and grid, the fastest single solver and the fastest ensemble of up to three members under the cost-aware oracle (median work-unit time over 16 test instances), at $\varepsilon = h^2$ and 10^{-8} ; the ratio is the speedup of the ensemble over the best single solver, and the last column is the ratio for {Jacobi, Jacobi (0.67)}.

[results pending: confirmatory run in progress]

Table 30: Nested ensembles: median time to $\varepsilon \in \{h^2, 10^{-6}, 10^{-8}\}$ of the learned router (live and work units) and of the cost-aware oracle (work units) over $\text{NO} \cup \mathcal{W}$, for the single members and the nested two- and three-member sets, all evaluated in one session on the same instances (64 at 128^2 , 32 at 256^2 , 16 for anisotropic diffusion at 256^2). The last columns give the paired speedup at 10^{-8} over the best single-member router (resp. oracle) with the two-sided Wilcoxon *p*-value; bold: $\geq 1.1 \times$ and significant after a Holm correction over all tests of the table.

Equation	<i>N</i>	<i>W</i>	$\varepsilon = h^2$			$\varepsilon = 10^{-6}$			$\varepsilon = 10^{-8}$			$\varepsilon = 10^{-8}$, vs. best single solver	
			router	router (WU)	oracle (WU)	router	router (WU)	oracle (WU)	router	router (WU)	oracle (WU)	router	oracle
Poisson	128^2	{Jacobi}	896 μs	707 μs	697 μs	1.70 ms	1.49 ms	1.48 ms	44.9 ms	45.8 ms	43.5 ms	–	–
		{GS}	869 μs	727 μs	717 μs	1.61 ms	1.53 ms	1.69 ms	2.44 ms	2.08 ms	2.06 ms	–	–
		{SymGS}	966 μs	743 μs	713 μs	1.33 ms	1.07 ms	1.15 ms	2.38 ms	2.02 ms	2.00 ms	(best single)	–
		{Jacobi(0.67)}	880 μs	707 μs	697 μs	1.05 ms	874 μs	886 μs	2.39 ms	2.04 ms	2.03 ms	–	–
		{SOR(1.5)}	918 μs	727 μs	717 μs	1.76 ms	1.68 ms	1.69 ms	3.03 ms	2.82 ms	2.79 ms	–	–
		{Jacobi, GS}	860 μs	708 μs	697 μs	1.33 ms	1.22 ms	1.39 ms	2.74 ms	2.18 ms	2.10 ms	$0.98 \times$ (slower, 8×10^{-8})	$0.98 \times$ (slower, 0.236)
		{Jacobi, Jacobi(0.67)}	897 μs	708 μs	697 μs	1.09 ms	887 μs	886 μs	2.71 ms	2.05 ms	2.03 ms	$0.99 \times$ (slower, 0.941)	$1.01 \times$ (0.001)
		{Jacobi, GS, SymGS}	792 μs	708 μs	733 μs	1.34 ms	1.24 ms	1.15 ms	2.82 ms	2.70 ms	2.64 ms	$0.94 \times$ (slower, $< 10^{-10}$)	$1.02 \times$ (5×10^{-4})
		{Jacobi, Jacobi(0.67), GS}	877 μs	728 μs	697 μs	1.13 ms	967 μs	886 μs	2.53 ms	2.06 ms	2.03 ms	$0.98 \times$ (slower, 0.883)	$1.01 \times$ (0.001)
		{Jacobi, GS, SymGS, Jacobi(0.67)}	913 μs	707 μs	697 μs	1.14 ms	906 μs	886 μs	2.73 ms	2.08 ms	2.03 ms	$0.98 \times$ (slower, 0.043)	$1.01 \times$ (0.001)
		{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)}	885 μs	707 μs	697 μs	1.41 ms	1.25 ms	886 μs	2.86 ms	2.51 ms	2.03 ms	$0.94 \times$ (slower, $< 10^{-10}$)	$1.01 \times$ (0.001)
Convection-diffusion	128^2	{Jacobi}	914 μs	682 μs	672 μs	3.85 ms	2.95 ms	2.92 ms	91.6 ms	65.8 ms	65.5 ms	–	–
		{GS}	994 μs	731 μs	721 μs	2.42 ms	1.93 ms	1.91 ms	2.72 ms	2.07 ms	2.05 ms	–	–
		{SymGS}	978 μs	731 μs	721 μs	1.81 ms	1.58 ms	1.59 ms	2.58 ms	1.98 ms	1.96 ms	–	–
		{Jacobi(0.67)}	936 μs	687 μs	677 μs	1.13 ms	854 μs	863 μs	2.59 ms	1.91 ms	1.90 ms	(best single)	–
		{SOR(1.5)}	1.07 ms	785 μs	775 μs	2.41 ms	1.93 ms	1.91 ms	3.36 ms	2.82 ms	2.79 ms	–	–
		{Jacobi, GS}	969 μs	692 μs	688 μs	1.52 ms	1.40 ms	1.36 ms	2.61 ms	2.03 ms	2.00 ms	$0.94 \times$ (slower, $< 10^{-10}$)	$0.95 \times$ (slower, $< 10^{-10}$)
		{Jacobi, Jacobi(0.67)}	955 μs	689 μs	677 μs	1.23 ms	874 μs	863 μs	2.79 ms	1.92 ms	1.90 ms	$1.00 \times$ (slower, 8×10^{-7})	$1.00 \times$ (10^{-4})
		{Jacobi, GS, SymGS}	878 μs	682 μs	721 μs	1.42 ms	1.36 ms	1.59 ms	3.10 ms	2.57 ms	1.94 ms	$0.74 \times$ (slower, $< 10^{-10}$)	$0.98 \times$ (slower, 2×10^{-10})
		{Jacobi, Jacobi(0.67), GS}	926 μs	732 μs	677 μs	1.46 ms	1.32 ms	863 μs	2.62 ms	1.94 ms	1.90 ms	$0.98 \times$ (slower, 10^{-7})	$1.00 \times$ (10^{-4})
		{Jacobi, GS, SymGS, Jacobi(0.67)}	935 μs	692 μs	677 μs	1.68 ms	1.37 ms	863 μs	2.58 ms	2.02 ms	1.90 ms	$0.95 \times$ (slower, $< 10^{-10}$)	$1.00 \times$ (6×10^{-4})
		{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)}	981 μs	693 μs	677 μs	1.71 ms	1.43 ms	863 μs	2.62 ms	2.00 ms	1.90 ms	$0.96 \times$ (slower, $< 10^{-10}$)	$1.00 \times$ (9×10^{-4})
Anisotropic diffusion	128^2	{Jacobi}	887 μs	652 μs	642 μs	1.33 ms	1.07 ms	854 μs	22.0 ms	17.3 ms	17.2 ms	–	–
		{GS}	884 μs	652 μs	642 μs	1.47 ms	1.14 ms	1.15 ms	2.89 ms	2.29 ms	2.14 ms	–	–
		{SymGS}	881 μs	652 μs	642 μs	1.36 ms	1.06 ms	1.05 ms	2.61 ms	2.06 ms	1.92 ms	–	–
		{Jacobi(0.67)}	897 μs	652 μs	642 μs	1.29 ms	958 μs	937 μs	2.63 ms	1.92 ms	1.90 ms	(best single)	–
		{SOR(1.5)}	878 μs	652 μs	642 μs	1.72 ms	1.28 ms	1.27 ms	3.35 ms	2.67 ms	2.49 ms	–	–
		{Jacobi, GS}	882 μs	653 μs	642 μs	1.30 ms	1.03 ms	854 μs	2.76 ms	2.32 ms	2.07 ms	$0.85 \times$ (slower, $< 10^{-10}$)	$0.96 \times$ (slower, 3×10^{-5})
		{Jacobi, Jacobi(0.67)}	873 μs	653 μs	642 μs	1.25 ms	949 μs	937 μs	2.52 ms	1.95 ms	1.94 ms	$1.00 \times$ (slower, 0.774)	$1.00 \times$ (0.008)
		{Jacobi, GS, SymGS}	899 μs	652 μs	642 μs	1.39 ms	1.11 ms	968 μs	2.66 ms	2.13 ms	1.96 ms	$0.87 \times$ (slower, $< 10^{-10}$)	$0.99 \times$ (slower, 4×10^{-4})
		{Jacobi, Jacobi(0.67), GS}	896 μs	653 μs	642 μs	1.21 ms	896 μs	937 μs	2.57 ms	1.98 ms	1.94 ms	$0.97 \times$ (slower, 10^{-7})	$1.00 \times$ (0.008)
		{Jacobi, GS, SymGS, Jacobi(0.67)}	875 μs	652 μs	642 μs	1.25 ms	941 μs	937 μs	2.55 ms	1.98 ms	1.92 ms	$0.98 \times$ (slower, 3×10^{-4})	$1.00 \times$ (4×10^{-4})
		{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)}	883 μs	652 μs	642 μs	1.19 ms	922 μs	948 μs	2.58 ms	2.03 ms	1.97 ms	$0.85 \times$ (slower, $< 10^{-10}$)	$1.00 \times$ (0.754)
Variable-coefficient diffusion	128^2	{Jacobi}	923 μs	805 μs	795 μs	$\geq 2.54 \text{ ms}^{11}$	$\geq 1.86 \text{ ms}^{11}$	$\geq 1.84 \text{ ms}^{11}$	$\geq 87.7 \text{ ms}^{124}$	$\geq 81.8 \text{ ms}^{124}$	$\geq 81.3 \text{ ms}^{124}$	–	–
		{Jacobi(0.67)}	939 μs	804 μs	794 μs	1.25 ms	1.11 ms	1.10 ms	2.63 ms	2.31 ms	2.29 ms	(best single)	–
		{GS}	1.02 ms	847 μs	837 μs	1.98 ms	1.90 ms	1.88 ms	3.46 ms	3.32 ms	3.29 ms	–	–
		{Jacobi, Jacobi(0.67)}	919 μs	806 μs	794 μs	1.45 ms	1.13 ms	1.10 ms	2.94 ms	2.37 ms	2.32 ms	$1.00 \times$ (slower, 6×10^{-4})	$1.00 \times$ (0.021)
		{Jacobi, GS, SymGS}	915 μs	805 μs	794 μs	1.30 ms	1.15 ms	1.10 ms	2.96 ms	2.54 ms	2.32 ms	$0.91 \times$ (slower, 9×10^{-5})	$1.00 \times$ (0.021)

1345 E.8 Overheads and amortisation

1346 Table 31 lists the measured cost of every operation as a function of N , including one decision of the
1347 LSTM router of Section 7 (full-state input of size $4N^2$, hidden size 256, 4 layers): at 128^2 one LSTM
1348 decision costs [pending] ms, about [pending] Jacobi sweeps, which is why the wall-clock study uses
1349 the scalar-feature router ($\approx 5 \mu\text{s}$, independent of N). In situ, inside the timed replays, the router’s
1350 decisions cost 16–21 μs each (the first decision of a solve and every decision after a macro-action
1351 find cold caches; the 5 μs figure is a warm-loop median), i.e. 2.3–3.3% of the router’s time to h^2 (1–2
1352 decisions) and 0.8–2.5% of its time to 10^{-8} , as instrumented directly inside the replays. Separately,
1353 the paired difference between the router’s and the cost-aware oracle’s time at h^2 is 28–50 μs per solve;
1354 the two run as separate replays with decisions that agree on 50–100% of the instances, so this gap is
1355 consistent with, but not a measurement of, the decision cost (an earlier version of this appendix put
1356 the decisions at “well under 1%” on the basis of the warm-loop figure). The corrector calls account
1357 for an estimated 57–100% of the router’s time to h^2 (number of calls times the calibrated per-call
1358 cost; per-cell medians over the 128^2 pairwise cells). Two further accounting checks: the charged
1359 time of a router solve is 99–99% of the outer elapsed time of its replay, the remainder being the
1360 Python driver’s loop and timer overhead, which every method pays per iteration; and the router’s
1361 live time to h^2 is 0.92–1.17 times its work-unit time, the per-operation costs being measured in the
1362 same form as the replay charges them. Table 32 accounts for training: the corrector (shared with
1363 HINTS) is fit in minutes, the router in seconds to a few minutes per pairing, and the break-even
1364 number of solves after which the router’s own training cost is recovered relative to HINTS ($\tau=15$)
1365 with the same corrector is [pending]–[pending] solves for the GS pairing at each equation and grid
1366 (Table 32; [pending] of those cells have no positive saving, and [pending] have a positive saving but
1367 no recorded router-training time).

Table 31: Measured single-thread cost of one operation as a function of the grid size (Poisson; median of repeated calls). Multigrid uses a direct solve below 64^2 and is not listed there.

[results pending: confirmatory run in progress]

Table 32: Training-cost amortisation for the GS pairing: corrector training (data generation and least-squares fit, CPU), router training (oracle rollouts and DAGger rounds), median wall-clock saved per solve at $\varepsilon = h^2$ relative to HINTS ($\tau=15$) and to the solver alone, and the number of solves after which the router’s training cost is recovered relative to HINTS.

[results pending: confirmatory run in progress]

1368 E.9 Development versus confirmatory runs

1369 Several choices of this study were made after looking at test results: the corrector band and the
1370 anisotropic sensor grid, the router of the convection–diffusion Jacobi pairing at 512^2 (retrained with
1371 more rollouts after the first router missed h^2 on 4 of 16 development instances, Section E.1), and
1372 the imitation budget of the ensemble routers (raised after the pairwise recipe imitated its oracle
1373 imperfectly on one development cell, Section E.7); the optimistic best-schedule baseline is tuned on
1374 the test set by design and labelled as such. All of these were made on the development set (seed 72),
1375 with the earlier numpy/scipy sweeps, the earlier forcing sampler and $\tau = 25$ as the HINTS reference
1376 (Section E.1), which is why the table compares against $\tau = 25$ in both runs. Every result in this
1377 appendix comes from a single evaluation of the frozen configurations on the confirmatory set (seed
1378 73) with the compiled kernels and three timed replays. Table 33 lists, for every cell present in both,
1379 the development and the confirmatory speedups: the two differ both through the instances and through
1380 the kernels—the compiled sweeps are $4\text{--}5\times$ faster, so the corrector is worth $2\text{--}3\times$ more sweeps than
1381 in the development runs and the margins over fixed schedules at h^2 shrink accordingly—and the table
1382 shows exactly how much of the development-phase margin survives.

Table 33: Development (seed 72, numpy/scipy sweeps, one timed replay) versus confirmatory (seed 73, compiled kernels, three timed replays) paired median speedups of the learned router, for every pairwise cell at $\varepsilon = h^2$ against HINTS ($\tau=25$, the reference of the development runs) and against the best fixed schedule available in that run, and for every nested ensemble at 10^{-8} in work units against the best single-member router and against its oracle; two-sided Wilcoxon p -values in parentheses, censoring classes as in the speed tables; bold: ratio $\geq 1.1\times$ and significant after a Holm correction over the table.

Equation	N	Pairing / ensemble	development (seed 72, numpy kernels)		confirmatory (seed 73, compiled kernels)	
			vs. HINTS-25 or best single	vs. best schedule or oracle	vs. HINTS-25 or best single	vs. best schedule or oracle
Poisson	128^2	{NO, Jacobi}	2.68 ($< 10^{-10}$)	1.21 (4×10^{-8})	1.66 ($< 10^{-10}$)	0.97 $\times$ (slower, 0.221)
	128^2	{NO, Jacobi (0.67)}	2.90 ($< 10^{-10}$)	1.28 (4×10^{-8})	1.61 ($< 10^{-10}$)	0.95 $\times$ (slower, 10^{-7})
	128^2	{NO, GS}	6.75 ($< 10^{-10}$)	1.94 ($< 10^{-10}$)	2.90 ($< 10^{-10}$)	0.96 $\times$ (slower, 7×10^{-9})
	128^2	{NO, SymGS}	9.21 ($< 10^{-10}$)	2.34 ($< 10^{-10}$)	3.72 ($< 10^{-10}$)	0.95 $\times$ (slower, $< 10^{-10}$)
	128^2	{NO, SOR (1.5)}	10.3 ($< 10^{-10}$)	3.22 ($< 10^{-10}$)	4.17 ($< 10^{-10}$)	0.96 $\times$ (slower, 2×10^{-9})
	128^2	{NO, Multigrid}	1.96 ($< 10^{-10}$)	1.88 ($< 10^{-10}$)	0.98 $\times$ (slower, 10^{-4})	0.97 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, GS} (10^{-8} , work units)	—	0.99 $\times$ (slower, $< 10^{-10}$)	1.00 $\times$ (slower, 0.072)	0.90 $\times$ (slower, 5×10^{-8})
	128^2	{Jacobi, GS, SymGS} (10^{-8} , work units)	—	0.99 $\times$ (slower, 0.012)	0.94 $\times$ (slower, $< 10^{-10}$)	0.76 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, Jacobi(0.67)} (10^{-8} , work units)	1.00 $\times$ (0.194)	0.99 $\times$ (slower, $< 10^{-10}$)	1.00 $\times$ (slower, 9×10^{-7})	0.99 $\times$ (slower, 2×10^{-7})
	128^2	{Jacobi, Jacobi(0.67), GS} (10^{-8} , work units)	0.92 $\times$ (slower, 2×10^{-7})	0.98 $\times$ (slower, 6×10^{-8})	0.99 $\times$ (slower, 10^{-7})	0.99 $\times$ (slower, 10^{-7})
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67)} (10^{-8} , work units)	—	0.91 $\times$ (slower, $< 10^{-10}$)	0.98 $\times$ (slower, 0.043)	0.96 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	0.79 $\times$ (slower, $< 10^{-10}$)	0.94 $\times$ (slower, $< 10^{-10}$)	0.74 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
Convection-diffusion	128^2	{NO, Jacobi}	2.95 ($< 10^{-10}$)	1.34 (10^{-10})	1.67 ($< 10^{-10}$)	0.95 $\times$ (slower, 10^{-8})
	128^2	{NO, Jacobi (0.67)}	3.14 ($< 10^{-10}$)	1.32 (10^{-9})	1.51 ($< 10^{-10}$)	0.97 $\times$ (slower, 0.007)
	128^2	{NO, GS}	6.76 ($< 10^{-10}$)	2.60 ($< 10^{-10}$)	3.01 ($< 10^{-10}$)	0.96 $\times$ (slower, 0.006)
	128^2	{NO, SymGS}	8.70 ($< 10^{-10}$)	2.40 ($< 10^{-10}$)	3.65 ($< 10^{-10}$)	0.94 $\times$ (slower, 3×10^{-7})
	128^2	{NO, SOR (1.5)}	8.89 ($< 10^{-10}$)	2.40 ($< 10^{-10}$)	4.84 ($< 10^{-10}$)	0.96 $\times$ (slower, 5×10^{-9})
	128^2	{NO, Multigrid}	2.07 ($< 10^{-10}$)	2.01 ($< 10^{-10}$)	0.95 $\times$ (slower, 0.011)	0.96 $\times$ (slower, 7×10^{-10})
	128^2	{Jacobi, GS} (10^{-8} , work units)	—	0.78 $\times$ (slower, $< 10^{-10}$)	1.01 $\times$ (9×10^{-5})	0.99 $\times$ (slower, 7×10^{-5})
	128^2	{Jacobi, GS, SymGS} (10^{-8} , work units)	—	0.96 $\times$ (slower, 6×10^{-8})	1.00 $\times$ (0.003)	0.75 $\times$ (slower, 10^{-10})
	128^2	{Jacobi, Jacobi(0.67)} (10^{-8} , work units)	1.08 $\times$ (10^{-10})	0.99 $\times$ (slower, 2×10^{-10})	1.00 $\times$ (slower, 8×10^{-7})	0.99 $\times$ (slower, 10^{-8})
	128^2	{Jacobi, Jacobi(0.67), GS} (10^{-8} , work units)	0.99 $\times$ (slower, 0.309)	0.90 $\times$ (slower, $< 10^{-10}$)	0.98 $\times$ (slower, 10^{-7})	0.97 $\times$ (slower, 10^{-7})
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67)} (10^{-8} , work units)	—	0.82 $\times$ (slower, $< 10^{-10}$)	0.95 $\times$ (slower, $< 10^{-10}$)	0.94 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	0.93 $\times$ (slower, 3×10^{-10})	0.96 $\times$ (slower, $< 10^{-10}$)	0.93 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
Anisotropic diffusion	128^2	{NO, Jacobi}	3.16 ($< 10^{-10}$)	1.25 (6×10^{-9})	1.63 ($< 10^{-10}$)	0.95 $\times$ (slower, 10^{-6})
	128^2	{NO, Jacobi (0.67)}	3.31 ($< 10^{-10}$)	1.34 ($< 10^{-10}$)	1.68 ($< 10^{-10}$)	0.95 $\times$ (slower, 2×10^{-9})
	128^2	{NO, GS}	7.61 ($< 10^{-10}$)	1.99 ($< 10^{-10}$)	3.04 ($< 10^{-10}$)	0.95 $\times$ (slower, $< 10^{-10}$)
	128^2	{NO, SymGS}	11.8 ($< 10^{-10}$)	2.72 ($< 10^{-10}$)	4.08 ($< 10^{-10}$)	0.95 $\times$ (slower, 9×10^{-9})
	128^2	{NO, SOR (1.5)}	7.36 ($< 10^{-10}$)	2.04 ($< 10^{-10}$)	3.06 ($< 10^{-10}$)	0.96 $\times$ (slower, 2×10^{-7})
	128^2	{Jacobi, GS} (10^{-8} , work units)	—	0.99 $\times$ (slower, 2×10^{-10})	1.00 $\times$ (slower, 3×10^{-5})	0.88 $\times$ (slower, $< 10^{-10}$)
	128^2	{Jacobi, GS, SymGS} (10^{-8} , work units)	—	0.99 $\times$ (slower, 0.003)	0.98 $\times$ (slower, 10^{-8})	0.88 $\times$ (slower, 4×10^{-9})
	128^2	{Jacobi, Jacobi(0.67)} (10^{-8} , work units)	1.14 $\times$ ($< 10^{-10}$)	0.99 $\times$ (slower, $< 10^{-10}$)	1.00 $\times$ (slower, 0.774)	0.99 $\times$ (slower, 0.053)
	128^2	{Jacobi, Jacobi(0.67), GS} (10^{-8} , work units)	1.15 $\times$ ($< 10^{-10}$)	0.99 $\times$ (slower, 2×10^{-6})	0.97 $\times$ (slower, 10^{-7})	0.97 $\times$ (slower, 10^{-6})
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67)} (10^{-8} , work units)	—	0.95 $\times$ (slower, 4×10^{-9})	0.98 $\times$ (slower, 2×10^{-7})	0.96 $\times$ (slower, 5×10^{-8})
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	0.95 $\times$ (slower, 2×10^{-9})	0.85 $\times$ (slower, $< 10^{-10}$)	0.89 $\times$ (slower, 4×10^{-10})
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—
	128^2	{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)} (10^{-8} , work units)	—	—	—	—

1383 E.10 Verification of the theoretical assumptions

1384 Theorem 4.1 and Propositions 4.2–4.3 assume that every error propagation map $G_j = I - C_j \circ \mathcal{L}_h^a$ is
1385 Lipschitz with constant $\rho_j \leq 1$ (with at least one strict contraction in the optimal sequence, respec-
1386 tively a shared unitary eigenbasis), zero-preserving, and—for postfix monotonicity—simultaneously
1387 diagonalisable by a unitary matrix; Theorem 5.1 assumes that at every state at least one solver leaves
1388 a positive fraction of the error. Table 34 checks these conditions numerically for every operation
1389 of the study—finite diagnostics (a power-iteration estimate of the Lipschitz constant, commutators
1390 on random vectors, exhaustive search over short horizons), none of which is a certificate over the
1391 whole input space— on the mean-free subspace (the constant mode is the null space of the pe-
1392 riodic operator and is excluded from the data and from every iteration): the Euclidean Lipschitz
1393 constant $\|G_j\|_2$ (power iteration on $G_j^T G_j$; exact from the Fourier symbol for Jacobi), the Lipschitz
1394 constant $\|G_j\|_A = \|A_s^{1/2} G_j A_s^{-1/2}\|_2$ in the energy norm of the symmetric part A_s of the discrete
1395 operator, the same for the macro-action $G_j^{m_j}$, the spectral radius, the smallest observed contraction
1396 $\sigma_{\min} = \min \|G_j x\|/\|x\|$, $\|C_j(0)\|$, and the commutator $\|G_j G_{\text{NO}} - G_{\text{NO}} G_j\|$ on random vectors.
1397 Table 36 evaluates the quantities of Theorem 4.1 and Theorem 5.1 along the cost-aware oracle’s own
1398 path to $\varepsilon = 10^{-8}$ (a horizon of tens of macro-actions; to h^2 the path has one or two steps).

1399 **What holds, and in which norm.** (i) Every classical solver is zero-preserving exactly and, on the
1400 mean-free Nyquist-free subspace on which the forcing lives (undamped Jacobi has eigenvalue -1
1401 on the checkerboard mode, which the forcing excludes and the Jacobi-type maps never generate;
1402 the table’s Jacobi values include the Nyquist modes as its caption states), contracts in the sense that
1403 matters for convergence: the spectral radius of G_j is below 1 in every setting (at most [pending]), and
1404 so is the energy-norm Lipschitz constant of the solver and of its macro-action (at most [pending]).
1405 The *Euclidean* Lipschitz constant, however, exceeds 1 for Gauss–Seidel, SOR and symmetric GS (up
1406 to [pending] for SOR), whose iteration matrices are non-normal; the proof of Proposition 4.2 uses
1407 only the triangle inequality and the composition rule for Lipschitz constants, so it holds verbatim
1408 with $\|\cdot\|_2$ replaced by the energy norm, in which the condition is satisfied by all classical solvers
1409 in all settings—but that statement concerns the greedy rule and objective defined with the energy
1410 norm, whereas the oracle and the router of this study minimise the Euclidean error. For the evaluated
1411 Euclidean rule, the strict-contraction premise of Proposition 4.2 therefore holds for the Jacobi-type
1412 solvers and fails for the non-normal sweeps, whose Euclidean expansion is confined to a single

1413 sweep (their spectral radii and macro-action contraction show the iteration is convergent). For
 1414 (damped) Jacobi both constants agree (the maps are Fourier-diagonal); the values are close to 1
 1415 because the slowest modes (the smoothest ones, and the near-Nyquist mode for undamped Jacobi)
 1416 are barely damped by a point smoother—which is exactly why the corrector is needed. (ii) The
 1417 corrector’s map is zero-preserving, contracts every band mode to **[pending]** or better (on average to
 1418 10^{-4} – 10^{-6}) and leaves the modes outside the band untouched, so its Lipschitz constant is exactly 1
 1419 (not < 1): Theorem 4.2 in its strict form does not apply to the corrector, but Theorem 4.3 does, and
 1420 its second condition—a shared unitary eigenbasis—holds exactly for the exact band-limited inverse
 1421 together with (damped) Jacobi, all of which are diagonal in the Fourier basis, and, for the learned
 1422 corrector, the measured commutators are $\approx 10^{-6}$, at the corrector’s single-precision accuracy (a
 1423 diagnostic; replacing the learned corrector by the exact inverse did not change the oracle’s decisions
 1424 in the development run, Section E.1), so for the ensembles $\{\text{NO}, \text{Jacobi}\}$, $\{\text{NO}, \text{Jacobi} (0.67)\}$ and
 1425 $\{\text{NO}, \text{Jacobi}, \text{Jacobi} (0.67)\}$ with the exact band-limited inverse the objective is supermodular and
 1426 postfix monotone and Theorem 4.1 applies with $\alpha = \mu = 1$; for the fitted corrector we measure
 1427 the fit error and the commutators but do not propagate them through a perturbation bound (a small
 1428 commutator is a diagnostic, not a certificate), so no guarantee is stated for it, and the exhaustive short-
 1429 horizon check below is the evidence that the fitted maps behave like the exact ones. For GS, SOR,
 1430 symmetric GS and multigrid the commutators are small but nonzero (10^{-3} – 10^{-1}) and no guarantee
 1431 is claimed at any horizon. (iii) Invertibility, which an earlier version of the theory used for postfix
 1432 monotonicity, is neither needed for the corrected result nor sufficient in general (Section A.3); for the
 1433 record, a Gauss–Seidel or symmetric-GS sweep annihilates an error concentrated at the first grid point
 1434 ($\sigma_{\min} = 0$), SOR leaves $(1 - \omega)$ of it ($\sigma_{\min} = 0.72$ – 0.83 in Table 34), undamped Jacobi annihilates
 1435 the mode $(N/4, N/4)$, and the corrector annihilates the band. Postfix monotonicity ($\mu = 1$) is
 1436 therefore guaranteed only for the Jacobi-type ensembles with the exact band-limited inverse, and is
 1437 measured for the others. (iv) Table 35 checks Theorem 4.1 by exhaustive enumeration (in floating
 1438 point, with errors clipped at a relative floor of 10^{-11}) for short horizons on the evaluated ensembles,
 1439 in the Euclidean objective the oracle optimises and for the deployed cost-aware rule: for every pairing
 1440 (including line GS on anisotropic diffusion), for $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$, for the two non-commuting
 1441 sweeps $\{\text{GS}, \text{SOR}\}$, for $\{\text{GS}, \text{multigrid}\}$ and for all point smoothers together, the optimal sequence
 1442 O of T macro-actions ($T = 4$, or 3 for ensembles with more than four actions) is found by exhaustive
 1443 search on 8 test instances (4 for ensembles with more than four actions), and μ (over the greedy
 1444 prefixes), $\alpha(O)$ (over the greedy prefixes, and over every sequence of length below T) and the
 1445 bound itself are computed. Over all **[pending]** instance–ensemble pairs the largest μ is **[pending]**,
 1446 the largest $\alpha(O)$ over the greedy prefixes is **[pending]** and over all short prefixes **[pending]**; the
 1447 numerical bound holds on **[pending]** pairs, and the on-path premises that the recurrence proof of
 1448 Theorem 4.1 uses—prefix monotonicity and the weak-supermodularity inequality at the reported α
 1449 on the greedy prefixes (not the global definition), the deployed rule equal to Algorithm 1, and an
 1450 exhaustive optimum resolved above the round-off floor—are verified, as a finite numerical check, on
 1451 **[pending]** pairs (on **[pending]** the optimum is at the floor and the instance is numerically unresolved
 1452 rather than certified; the deployed rule differs from Algorithm 1 on **[pending]**, and the clipping would
 1453 have changed the deployed path on **[pending]**); **[pending]** greedy prefixes and **[pending]** prefixes
 1454 overall show an expansion of the error by a single action of O , i.e. a failure of prefix monotonicity in
 1455 the Euclidean norm, and **[pending]** greedy prefixes (**[pending]** prefixes overall) violate the inequality
 1456 at the reported α (the largest candidate α over greedy prefixes, **[pending]**, includes the pairs that fail).
 1457 The greedy sequence’s error is within **[pending]**× of the exhaustive optimum on every one of the
 1458 **[pending]** pairs with a resolved optimum (on the others the ratio is one of clipped objectives; the table
 1459 gives medians and the worst cases per ensemble). The check supports the optimality of the greedy rule
 1460 over a few macro-actions; the horizons at which the oracle runs to 10^{-8} are 10–400 macro-actions
 1461 (Table 36), where no constant is established for the non-normal sweeps: Proposition 4.2 would
 1462 need contraction in the Euclidean norm, which they lack (Table 34), and the worst-case constant
 1463 in Table 36 is computed with energy-norm constants for orientation only. The worst-case $\alpha(O)$ of
 1464 Theorem 4.2, which only uses the Lipschitz constants, is loose in comparison: along the oracle’s
 1465 path to 10^{-8} (Table 36) $\sum_i \rho_{O_i}^2$ is close to T because each ρ is dominated by the slowest mode,
 1466 so the bound ranges from **[pending]** (the multigrid pairing, whose cycle contracts everything) to
 1467 10^2 – 10^4 for the point smoothers, and is infinite on the few instances where $\sum_i \rho_{O_i}^2 \geq T$. (The last
 1468 two columns of Table 36 evaluate the ratio of Equation (7) with the oracle’s own suffix in place of
 1469 the unknown long-horizon O ; this is only a diagnostic, and its maximum is attained trivially at the
 1470 last prefix.) (v) Theorem 5.1 is stated for the per-state normalised weights $\tilde{c}_j / \sum_k \tilde{c}_k$ that the router
 1471 is trained with; its one hypothesis is that at every visited state at least one solver leaves a positive

1472 fraction of the current error, i.e. $\max_j \tilde{c}_j / \|e^{(t)}\|^2 > 0$ (last column of Table 36; the column before it
 1473 shows that the per-step errors never exceed the current error), so the surrogate’s Bayes consistency
 1474 applies at every visited state.

Table 34: Numerical check of the assumptions of Propositions 4.2–4.3 for every operation of the study (mean-free subspace). **[Provisional: development run; the confirmatory run regenerates this table with the compiled kernels’ macro-action sizes.]** $\|\cdot\|_2$: Euclidean Lipschitz constant of the error propagation map (power-iteration estimate; exact from the symbol for Jacobi); $\|\cdot\|_A$: Lipschitz constant in the energy norm of the symmetric part of $\mathcal{L}_h$, for one iteration and for the macro-action of m_j iterations; ρ : spectral radius; $\sigma_{\min}$: smallest observed contraction (a unit error at the first grid point for the triangular sweeps, the minimum over Fourier modes otherwise; for the corrector, the largest remaining fraction of a band mode); $\|C_j(0)\|$: zero preservation; last column: commutator with the corrector’s map on random unit vectors (Proposition 4.3). For (damped) Jacobi the constants are exact from the Fourier symbol (the value with the Nyquist modes included; without them undamped Jacobi has $\|G\|_2 = \rho = 0.9994$ at 128^2).

[results pending: confirmatory run in progress]

Table 35: Exhaustive short-horizon check of Theorem 4.1 (128×128 , n test instances per row, T macro-actions, K actions including the corrector, deployed cost-aware rule): the optimal sequence O is found by exhaustive search; $\mu = \max_{t < T} g(S^t \oplus O) / g(O)$ over the greedy prefixes; $\alpha(O)$ over the greedy prefixes and over all sequences of length below T ; whether $g(S^T) \leq \mu(1 - \phi_T)g(O) + \phi_T g(\emptyset)$ holds; the ratio $g(S^T) / g(O)$ of the greedy to the optimal error (median and maximum over instances); the number of instances on which the greedy sequence is the optimum; and the numbers of instances on which the exhaustive optimum is at the round-off floor (numerically unresolved), on which the deployed rule coincides with Algorithm 1, of greedy prefixes and of all prefixes at which the weak-supermodularity inequality fails at the reported α , and of instances on which the on-path premises of the proof were verified. Errors below 10^{-11} relative are clipped to that floor.

[results pending: confirmatory run in progress]

Table 36: Quantities of Theorem 4.1, Proposition 4.2 and Theorem 5.1 along the cost-aware oracle’s path to $\varepsilon = 10^{-8}$ (8 test instances; medians unless stated): number of macro-actions T , $\sum_i \rho_{O_i}^2$ with the energy-norm constants of the chosen macro-actions, the per-instance margin $T - \sum_i \rho_{O_i}^2$ (median) and the resulting per-instance $\alpha(O)$ of Proposition 4.2 (median and range over the instances on which the margin is positive; ∞ otherwise), the empirical supermodularity ratio $\hat{\alpha}$ of Equation (7) evaluated with S the oracle’s prefix and S' its suffix (maximum and median over prefixes and instances), and the normalised bounds of Theorem 5.1: the largest ratio $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ over visited states ($\bar{E}$) and the smallest value of $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ ($E_{\min}$).

[results pending: confirmatory run in progress]

1475 F Limitations

1476 A primary limitation of our approach is the higher on-time computational cost requires to learn the
 1477 greedy routing strategy. In contrast, the simpler baselines such as HINTS incur no training overhead.
 1478 While this cost is amortized during inference time and leads to improved solution quality, it may be
 1479 prohibitive in settings with limited computational resources or when rapid deployment is required.
 1480 On the deployment side, Section E shows what survives cost accounting: with a corrector that covers
 1481 the smooth half of the spectrum and a classical smoother that covers the rest, the router makes
 1482 one corrector call on 80–100% of the instances at truncation level and executes the same decision

sequence as the best static schedule on 67–100% of them, without a search; below truncation level the comparison with the best fixed schedule is pairing-dependent, and the paired ratio of multigrid alone to the router is $0.03\times$ – $20.5\times$ at 128^2 . The corrector we use is specific to the constant-coefficient periodic setting (a fixed Fourier trunk basis on a coarse sensor grid), its cost grows with the fourth power of the sensor-grid size, and undamped Jacobi remains limited far below truncation level by the near-Nyquist modes that neither component damps. Our wall-clock study is restricted to single-thread CPU execution; with a corrector band that does not grow with the grid, the gains over fixed schedules shrink at 512^2 for the point smoothers, whereas pairing the corrector with geometric multigrid is expected to restore them (Section E; [512² **pending**]). Ensembles of classical solvers can pay off only where the error passes through regimes that favour different members; in the development screening a single well-chosen smoother sufficed at truncation level (Section E.7). For the constant-coefficient periodic test problems the discrete operator is diagonalised by the FFT, and the resulting direct solve, reported in every table, took $1/3.25\times$ – $1/3.08\times$ of the best router’s time at 128^2 ; the hybrid is meant for problems on which no such direct diagonalisation exists—where the band-limited inverse has to be assembled or fitted rather than read off a symbol, and the corrector is then inexact—which is why Section E also reports a variable-coefficient diffusion problem.

Additionally, the effectiveness of the learned router depends on the neural operator exhibiting heterogeneous performance across samples. When the neural operators produces predictions of relatively uniform quality, the benefit of adaptive routing diminishes and it is more practical to set a fixed schedule based on the quality of the neural operator. In practice, achieving such heterogeneity can be sensitive to training choices, and neural operators themselves can be finicky to train. In the wall-clock study the number of corrector calls to truncation level is one on 80–100% of the instances (Section 7); the instance dependence of the router’s decisions appears below truncation level, in when the corrector is worth calling again (Figure 1).

1507 G LLM Usage

LLMs were used substantially in preparing this paper, and we disclose this precisely. The text of the original submission was drafted by the authors, with ChatGPT and Gemini used for feedback on grammar and clarity. The revision that responds to the reviews was produced with Claude Code (Anthropic), operated and directed by the authors: the wall-clock benchmark pipeline (the compiled stencil kernels, the cost-aware macro-action environment, the scalar-feature router and its training loop, the corrector fitting, the baseline, table and figure generators, and the assumption and theorem checkers) was written largely by the assistant under the authors’ direction and was reviewed, tested and run by the authors; the corrections to the theory in this revision (the μ -slack in Theorem 4.1, the corrected constant and hypotheses of Proposition 4.2, the counterexample to postfix monotonicity, the unitary-eigenbasis condition, and the restatement of Theorem 5.1 for normalised weights) were drafted with the assistant’s help in response to reviewer findings and were verified by the authors; and much of the new text of Section 7 and Section E was drafted by the assistant from the generated tables and edited by the authors. The theorems and proof strategies of the original submission predate the use of the assistant. The research questions, the framing of the method, and the decisions about what to run and what to claim are the authors’. Independent reviews of the manuscript and code were also obtained from LLM-based reviewers; their findings are addressed in this revision.

1524 NeurIPS Paper Checklist

1525 1. Claims

1526 Question: Do the main claims made in the abstract and introduction accurately reflect the
1527 paper’s contributions and scope?

1528 Answer: [Yes]

1529 Justification: The abstract and introduction state the two-regime empirical outcome (parity
1530 with the best static schedule at truncation level; mixed results against the best fixed schedule
1531 below it; multigrid alone faster below truncation level) with counts and ranges computed
1532 from the tables, and the theoretical claims with their hypotheses (unitary eigenbasis for
1533 postfix monotonicity, measured slack otherwise; per-decision consistency for the full-
1534 information router). At the time of this draft several confirmatory cells are still being
1535 regenerated and are marked as pending in the text.

1536 Guidelines:

- 1537 • The answer [N/A] means that the abstract and introduction do not include the claims
1538 made in the paper.
- 1539 • The abstract and/or introduction should clearly state the claims made, including the
1540 contributions made in the paper and important assumptions and limitations. A [No] or
1541 [N/A] answer to this question will not be perceived well by the reviewers.
- 1542 • The claims made should match theoretical and experimental results, and reflect how
1543 much the results can be expected to generalize to other settings.
- 1544 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1545 are not attained by the paper.

1546 2. Limitations

1547 Question: Does the paper discuss the limitations of the work performed by the authors?

1548 Answer: [Yes]

1549 Justification: The paper discusses its limitations in the Limitations section (Appendix F),
1550 including the constant-coefficient periodic scope of the corrector, the single-thread CPU
1551 setting, the test-set-selected optimistic baselines and the pairing-dependent results below
1552 truncation level.

1553 Guidelines:

- 1554 • The answer [N/A] means that the paper has no limitation while the answer [No] means
1555 that the paper has limitations, but those are not discussed in the paper.
- 1556 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1557 • The paper should point out any strong assumptions and how robust the results are to
1558 violations of these assumptions (e.g., independence assumptions, noiseless settings,
1559 model well-specification, asymptotic approximations only holding locally). The authors
1560 should reflect on how these assumptions might be violated in practice and what the
1561 implications would be.
- 1562 • The authors should reflect on the scope of the claims made, e.g., if the approach was
1563 only tested on a few datasets or with a few runs. In general, empirical results often
1564 depend on implicit assumptions, which should be articulated.
- 1565 • The authors should reflect on the factors that influence the performance of the approach.
1566 For example, a facial recognition algorithm may perform poorly when image resolution
1567 is low or images are taken in low lighting. Or a speech-to-text system might not be
1568 used reliably to provide closed captions for online lectures because it fails to handle
1569 technical jargon.
- 1570 • The authors should discuss the computational efficiency of the proposed algorithms
1571 and how they scale with dataset size.
- 1572 • If applicable, the authors should discuss possible limitations of their approach to
1573 address problems of privacy and fairness.
- 1574 • While the authors might fear that complete honesty about limitations might be used by
1575 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
1576 limitations that aren’t acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: All theoretical results include clearly stated assumptions in the statement of theorems. Complete proofs are provided in Appendices A and B (greedy guarantee) and C (surrogate consistency) with cross-references from the main text.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper provides sufficient detail to reproduce all experiments, including the protocol, training procedures and implementation details described in Section 7 and Appendix E.1, and the code, run scripts and README that produce every table and figure from scratch.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to code with detailed instructions in a comprehensive README, including environment setup and commands to train routers and reproduce the main experimental results. These files will automatically generate the data needed for training. Data generation has also been documented in Appendix C.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies all training and evaluation details in Section 7 and Appendix E.1 (protocol, corrector fitting, router training, baselines, statistics); Appendix C documents the pipeline of the original submission whose experiment was withdrawn.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Every speedup in the appendix speed tables is a paired median ratio over test instances with a bootstrap 95% interval and a two-sided Wilcoxon signed-rank test, Holm-corrected within each table (Appendix E.1); the main-text table repeats two of those ratios per cell with their censoring marks but without intervals or tests (bold: both ratios at least one), the tests being in the appendix; the nested-ensemble, granularity and development-versus-confirmatory tables use the same tests and Holm correction without intervals; the AUC tables report means with standard errors and one-sided paired t-tests on log AUC; the seed trials report means and standard deviations over five router-training seeds and count seeds significant in the router's direction at $p < 0.01$.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper provides details on compute resources (single-thread CPU timings on one machine, training times, amortisation) in Appendix E.1 and E.8.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This paper does not involve any human subjects and all data has been simulated. This research focuses on numerical methods for physical systems and does not raise concerns related to safety, security, discrimination, surveillance, deception & harassment, environment, human rights, and bias & fairness.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses broader impacts in the introduction section. No direct negative societal impacts are anticipated.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release any models or datasets at all. It focuses on methodological contributions for numerical solvers using simulated data. Therefore, safeguards are not applicable.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- 1789 • Datasets that have been scraped from the Internet could pose safety risks. The authors
1790 should describe how they avoided releasing unsafe images.
1791 • We recognize that providing effective safeguards is challenging, and many papers do
1792 not require this, but we encourage authors to take this into account and make a best
1793 faith effort.

1794 12. Licenses for existing assets

1795 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1796 the paper, properly credited and are the license and terms of use explicitly mentioned and
1797 properly respected?

1798 Answer: [N/A]

1799 Justification: The paper does not use any external datasets or pretrained models. All
1800 experiments are conducted using simulated data and models trained in-house.

1801 Guidelines:

- 1802 • The answer [N/A] means that the paper does not use existing assets.
1803 • The authors should cite the original paper that produced the code package or dataset.
1804 • The authors should state which version of the asset is used and, if possible, include a
1805 URL.
1806 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
1807 • For scraped data from a particular source (e.g., website), the copyright and terms of
1808 service of that source should be provided.
1809 • If assets are released, the license, copyright information, and terms of use in the
1810 package should be provided. For popular datasets, `paperswithcode.com/datasets`
1811 has curated licenses for some datasets. Their licensing guide can help determine the
1812 license of a dataset.
1813 • For existing datasets that are re-packaged, both the original license and the license of
1814 the derived asset (if it has changed) should be provided.
1815 • If this information is not available online, the authors are encouraged to reach out to
1816 the asset's creators.

1817 13. New assets

1818 Question: Are new assets introduced in the paper well documented and is the documentation
1819 provided alongside the assets?

1820 Answer: [Yes]

1821 Justification: We release code for our method which is documented with a detailed
1822 README.

1823 Guidelines:

- 1824 • The answer [N/A] means that the paper does not release new assets.
1825 • Researchers should communicate the details of the dataset/code/model as part of their
1826 submissions via structured templates. This includes details about training, license,
1827 limitations, etc.
1828 • The paper should discuss whether and how consent was obtained from people whose
1829 asset is used.
1830 • At submission time, remember to anonymize your assets (if applicable). You can either
1831 create an anonymized URL or include an anonymized zip file.

1832 14. Crowdsourcing and research with human subjects

1833 Question: For crowdsourcing experiments and research with human subjects, does the paper
1834 include the full text of instructions given to participants and screenshots, if applicable, as
1835 well as details about compensation (if any)?

1836 Answer: [N/A]

1837 Justification: The paper does not involve crowdsourcing or any research with human subjects.
1838 All experiments are conducted using simulated data.

1839 Guidelines:

1840 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1841 with human subjects.

1842 • Including this information in the supplemental material is fine, but if the main contribu-
1843 tion of the paper involves human subjects, then as much detail as possible should be
1844 included in the main paper.

1845 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
1846 or other labor should be paid at least the minimum wage in the country of the data
1847 collector.

1848 **15. Institutional review board (IRB) approvals or equivalent for research with human**
1849 **subjects**

1850 Question: Does the paper describe potential risks incurred by study participants, whether
1851 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1852 approvals (or an equivalent approval/review based on the requirements of your country or
1853 institution) were obtained?

1854 Answer: [N/A]

1855 Justification: This paper does not involve human subjects. All experiments are conducted
1856 using simulated data. Therefore, IRB approval is not applicable.

1857 Guidelines:

1858 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1859 with human subjects.

1860 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1861 may be required for any human subjects research. If you obtained IRB approval, you
1862 should clearly state this in the paper.

1863 • We recognize that the procedures for this may vary significantly between institutions
1864 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1865 guidelines for their institution.

1866 • For initial submissions, do not include any information that would break anonymity (if
1867 applicable), such as the institution conducting the review.

1868 **16. Declaration of LLM usage**

1869 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1870 non-standard component of the core methods in this research? Note that if the LLM is used
1871 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1872 scientific rigor, or originality of the research, declaration is not required.

1873 Answer: [Yes]

1874 Justification: LLMs are not a component of the method, but an LLM coding assistant was
1875 used substantially in this revision to implement the benchmark pipeline, to help draft the
1876 corrections to the theory and to draft text from the generated tables, under the authors’
1877 direction and with the authors’ verification; the LLM Usage section (Appendix G) describes
1878 this precisely.

1879 Guidelines:

1880 • The answer [N/A] means that the core method development in this research does not
1881 involve LLMs as any important, original, or non-standard components.

1882 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1883 be described.